chapter : 01

Embedd system overview

✓. Defination of Embedded system

An embedded system is a special purpose system in which the computer is completely encapsulated by the device it controls.
An embedded system can be an independent system or it can be a part of a large system. It is microprocessor or microcontroller based system which is designed to perform specific tasks.
eg: fire alarm, traffic light system.

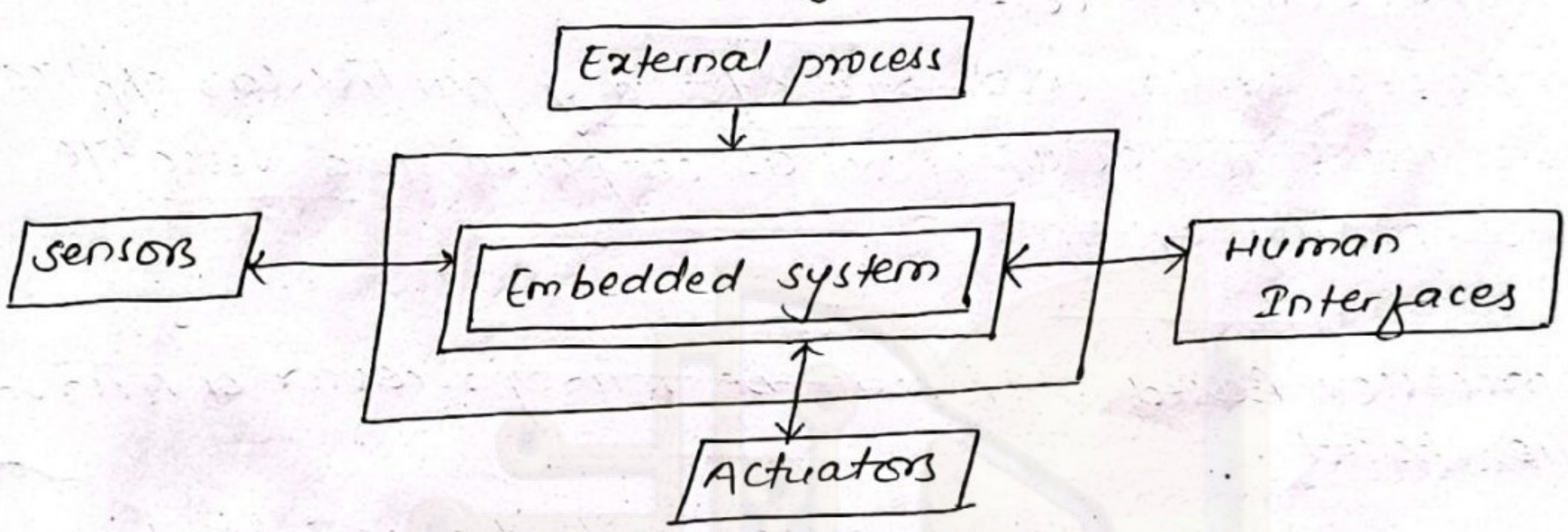


fig: Embedded system basic block diagram.

2015 spring
2016 Fall
2017 spring
2019 spring

what are the major characteristics of embedded system? Explain with example.

ans: The main characteristics of embedded system are:

notes.arpanadhikari7.com.np

i) is single functioned: An embedded system usually executes a specific program. The functionality of no two embedded system application is bounded to be the same. The functionality of a washing machine is different from the microwave.

ii) Real Time operation: It doesn't mean live operation. It means the software programs hardware to operate in a time bound fashion. It could also have two modes: hard and soft

CamScanner
CamScanner
Available on notes.arpanadhikari7.com.np

Hard mode indicates the task has to be completed within the allocated time (eg: clock), but in soft mode, the system could use additional time over the allotted time (eg: microwave)

iii) Tightly Design constraint

There are many design constraints to consider around the cost, performance, size and power of an embedded system to realize its absolute performance. These design factors are kept to a minimum to justify their simple function.

iv) Low manufacturing cost

As an embedded system design aims for any particular applicati, it involves less manufacturing cost as compared to a versatile general computing system.

v) Microprocessor Based : It must be microprocessor or micro-controller based.

vi) Connected : It must be connected with the peripheral of input and output devices.

vii) Hardware and software System

Software is used for more features and flexibility. Hardware is used for performance and security.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2014 fall ✓. **classification of embedded system.**

Embedded systems are classified into three categories and they are based on performance and microcontroller used.

i) small scale embedded systems
ii) medium scale embedded systems
iii) sophisticated embedded systems

1) small scale Es: small scale embedded systems have following specifications:

- designed with a single 8 or 16 bit microcontroller
- operated with a battery
- main programming tools:
  - * an editor
  - * assembler
  - * IDE (Integrated Development Environment)
  - * cross assembler.

2) medium scale Es: medium scale embedded systems have following specifications:

- designed with a single or few 16 or 32 bit microcontroller
- use of DSPs (Digital signal processor)
  RISCs (Reduced instruction set Computer)
- programming tools used:
  - * C
  - * C++
  - * Visual C++
  - * Java
  - * RTOS

3) Sophisticated Embedded systems: Sophisticated embedded systems have following specifications:

- huge hardware and software complexities
- use of PLA (programmable logic Array), IPs
- designed with multiple 32 or 64 bit microprocessor

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

· 2015 Fall
· 2016 spring
· 2019 Fall

V. Components of Embedded system

An Embedded system has three main components:

i's Embedded system hardware
ii's Embedded system software
iii's Real time operating system.

The basic block diagram of embedded system is shown below.

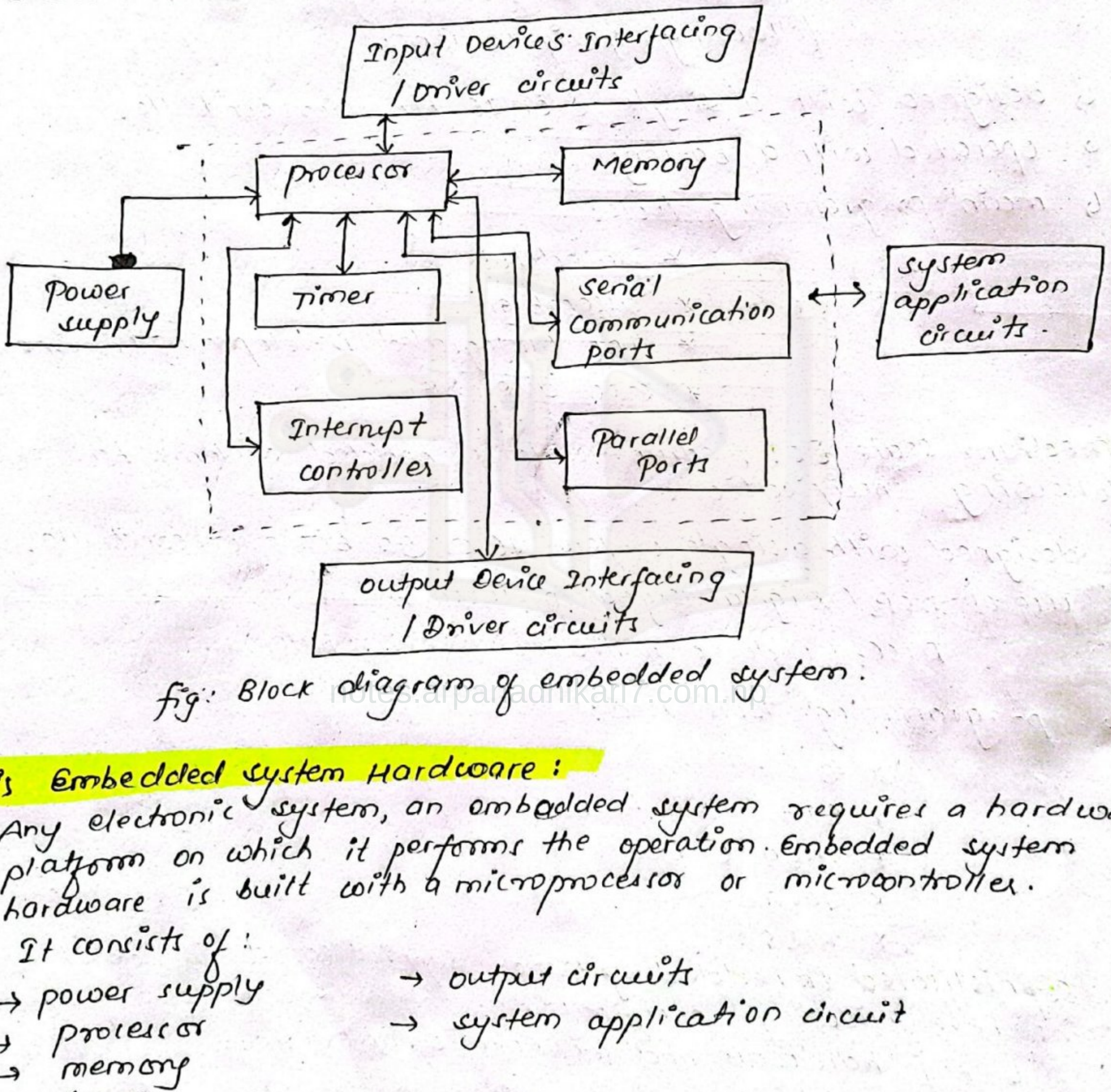


fig: Block diagram of embedded system.

notes.arpanadhikari7.com.np

i's Embedded system Hardware:

Any electronic system, an embedded system requires a hardware platform on which it performs the operation. Embedded system hardware is built with a microprocessor or microcontroller.

It consists of:

→ power supply
→ processor
→ memory
→ Timers
→ serial communication ports
→ output circuits
→ system application circuit

CamScanner

CamScanner

Available on notes.arpanadhikari7.com.np

(ii) Embedded system software

The embedded system software is written to perform a specific function. It is typically written in higher level format and then compiled down to provide code that can be lodged within a non-volatile memory within the hardware.

Embedded system software must take care of:

→ Availability of system memory
→ Availability of processor's speed
→ when the system runs continuously, there is a need to limit power dissipation

Assembler, compiler, emulator are some of the software tools used in embedded system.

(iii) Real Time operating system

→ A system is said to be real time, if it is essential to compl- ete its work and delivery its service on time

→ Real time operating system manages the application software and affords a mechanism to let the processor run.

→ The Real time operating system is responsible for handling the hardware resources of computer and host applications which run on the computer.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2014 Fall
2015 Spring
2020 Fall

Embedded systems Applications:

Embedded systems have different applications. A few select applications of embedded systems are smart cards, telecommunications, satellites, missiles, digital consumer electronics, computer networking etc.

Embedded systems in Automobiles

→ motor control system
→ Robotics in Assembly line
→ car multimedia
→ mobile and E-Com Access

Embedded systems in Telecommunications

→ mobile computing
→ Networking
→ wireless communications

Embedded systems in smart cards

→ Banking
→ Telephone
→ security systems

Embedded systems in Missiles and satellites

→ Defense
→ Aerospace
→ Communication

Embedded systems in Computer Networking

→ Networking system
→ Image processing
→ Printers
→ Network cards
→ monitors and displays

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Chapter : 8

VHDL

↳ VHDL stands for Very high speed integrated circuit hardware description language.

↳ It is a programming language used to model a digital system by data flow, behavioral and structural sytle of modeling.

↳ modeling style in VHDL:

i's Behavioural style
ii's Data flow modeling style
iii's structural modeling style
iv's mixed modeling style.

1. structural Modeling style:

↳ In structural style of modeling an entity is described as a set of interconnected components.

↳ The top level design entitie's architecture describes the interconnection of lower-level design entities.

↳ when components are used each must be declared.
↳ port map tells how a design entity is connected in the enclosed architechure.

eg:
```
library ieee;
use ieee.std_logic_1164.all;
entity sag_ha is
port (a,b : in std_logic;
      sum, carry : out std_logic);
end sag_ha;

architecture ha_sag of sag_ha is
component sag_xor is
  port (a,b : in std_logic;
        y : out std_logic);
```

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

```
end component;

component sag_and is
 port (a,b : in std_logic;
       z : out std_logic);
 end component;
begin
 X1 : sag_xor port map (a,b, sum);
 X2 : sag_and port map (c,d, carry);
end ha_sag;
```

2. Data flow modeling style:

- Data flow architecture uses only concurrent signal assignment statements.
- Dataflow style describes a system in terms of how dataflow through system.

Lets take an example.

```
library ieee;
use ieee.std_logic_1164.all;
entity half_adder is
 port (a,b : in std_logic;
       sum, carry : out std_logic);
end half_adder;

architecture dataflow of half_adder is
begin
    sum <= a xor b;
    carry <= a and b;
end dataflow;
```

notes.arpanadhikari7.com.np

CamScanner

CamScanner

Available on notes.arpanadhikari7.com.np

3. Behavioural modeling style:

→ A behavioural description describes a systems behaviour or function in an algorithmic fashion.
→ It is the most abstract style.
→ It consists of one or more process statements.
→ The entity decleration is same as for the dataflow architecture. However the architecture body is quit different. This architecture consists of a single process statement.

eg:

```
liberary ieee;
use ieee.std_logic_1164.all;
entity sag_pa is
port (A,B,C,D: in std_logic;
       Y: out std_logic);
end sag_pa;

architecture pa_sag of sag_pa is
begin
   process (A,B,C,D)                    [process statement]
   variable temp1, temp2,: std_logic;
                           temp3
begin
      temp1 := A and B;
      temp2 := C and D;
      temp3 := temp1 or temp2;
       Y <= not temp3;

end process;
end pa_sag;
```

Expression:
$Y = \overline{AB+CD}$

[Note: All the processes within the architecture body run concurrently but the statements within process run sequently]

notes.arpanadhikari7.com.np

CamScanner

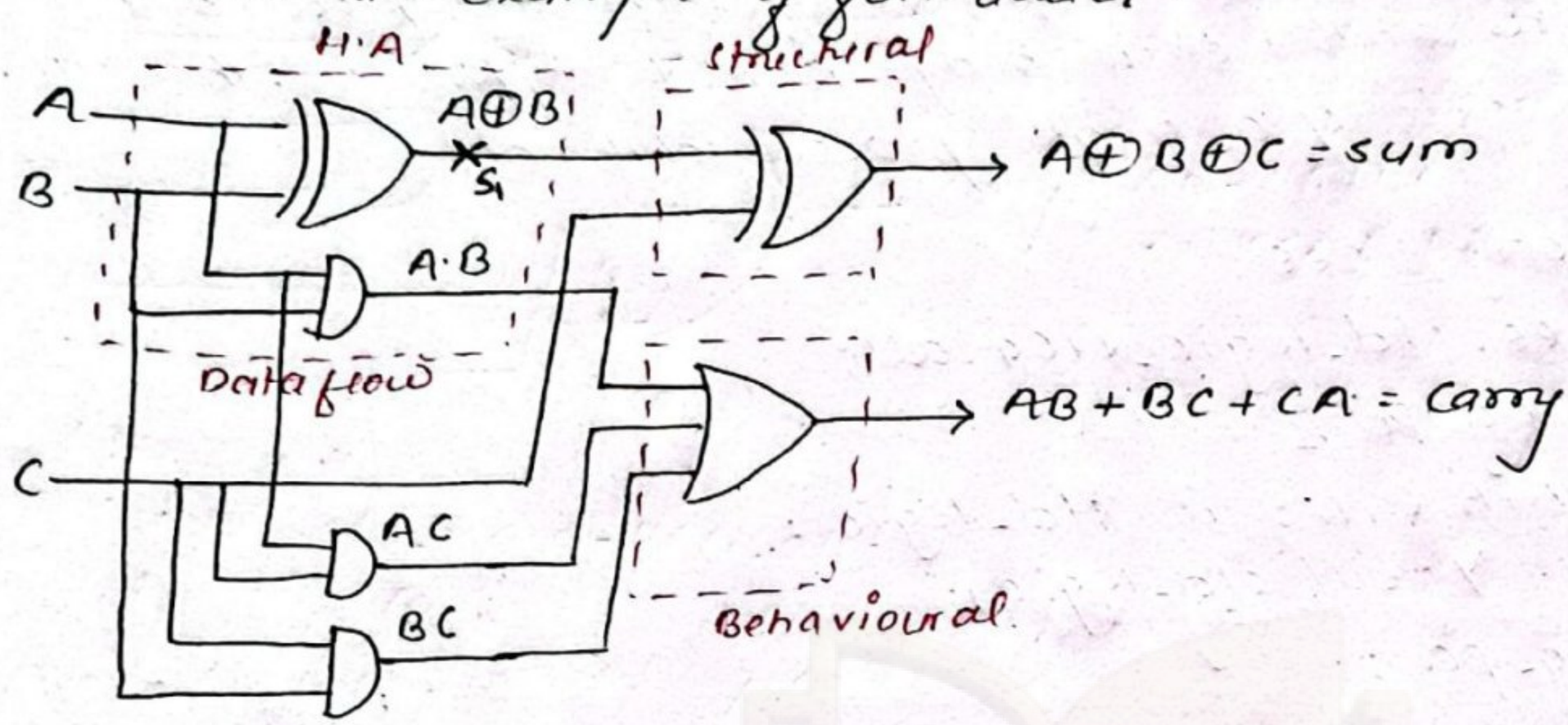
Available on notes.arpanadhikari7.com.np

4. mixed modeling style

The combination of atleast any two modeling style is called mixed modeling style.

eg: combination of structural, Dataflow and Behavioural modeling style.

Lets take an example of full adder.




```
library ieee;
use ieee.std_logic_1164.all;
entity sag_add is
port (A,B,C : in std_logic;
      sum, carry : out std_logic);
end sag_add;

architecture sag_mixed of mixed_sag is
component sag_XOR
port (A,B : in std_logic
      Y : out std_logic);
end component
signal S1 : std_logic;
begin
   S1 <= A XOR B
   X1 : sag_XOR port map (S1, C, sum);
```

} Data flow modeling style

} structural modeling style.

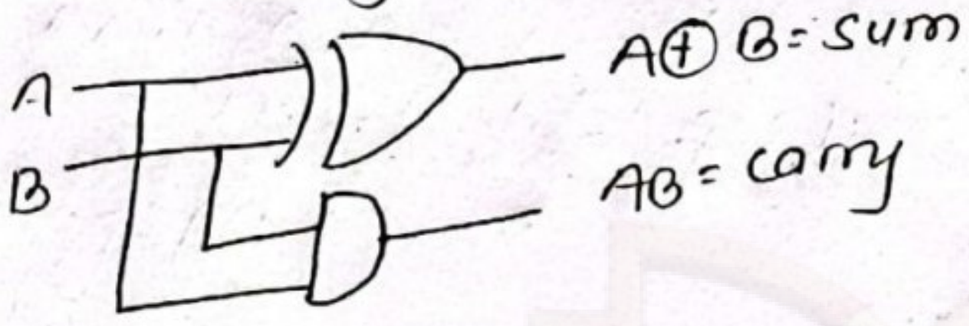
notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

```
process (A,B,C)
variable  V1, V2, V3 : in std_logic;
begin
  V1 = A and B;
  V2 = A and C;
  V3 = B and C;
carry <= V1 or V2 or V3;
end process;
end sag_mixed;
```

1. Half adder using VHDL.



input		output	
		carry	sum
A	B		
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

sum = A'B + B'A

= A⊕B

carry = AB

```
library ieee;
use ieee.std_logic_1164.all;
entity sag_ha is
 port (A,B :in std_logic;
     sum, carry : out std_logic);
 end sag_ha;
architecture ha_sag of sag_ha is
 component sag_xor
 port (A,B : in std_logic;
       sum : out std_logic);
 component sag_and
 port (A,B: in std_logic;
       carry: out std_logic);
 begin
 X1 : sag_xor portmap (A,B, sum);
 X2 : sag_and portmap (A,B, carry);
 end ha_sag;
```

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2. Write VHDL code for full adder using Half adder.

Sum = $A \oplus B \oplus C$

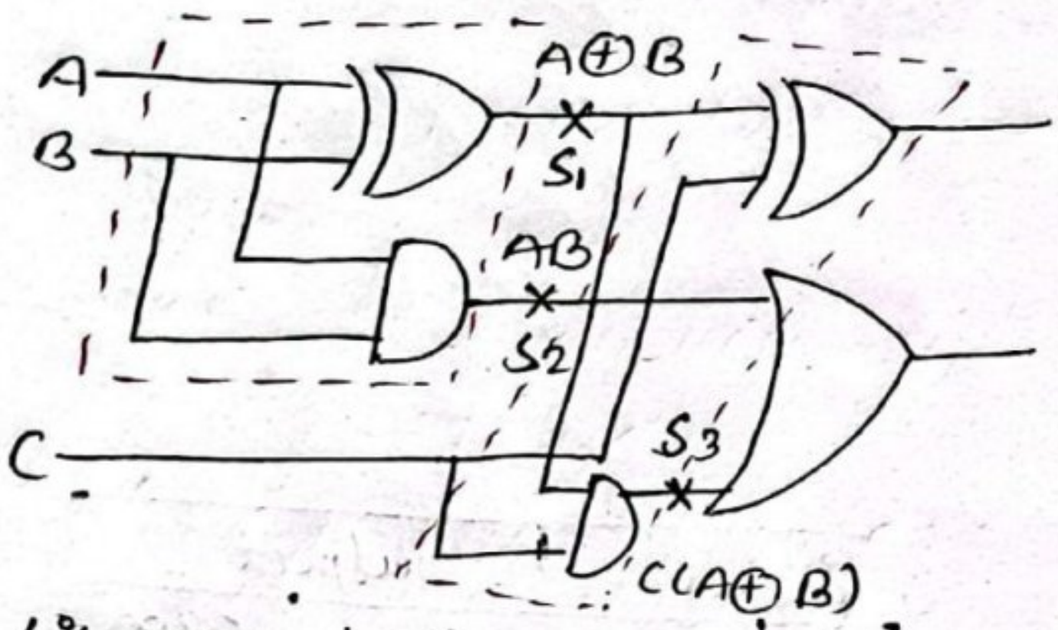
Carry = $AB + AC + BC$

$= AB + C(A \oplus B)$

Carry = $AB + AC + BC$

$= AB\bar{C} + A\bar{B}C + \bar{A}BC + AB'C + \bar{A}BC$

$= AB + C(A \oplus B)$



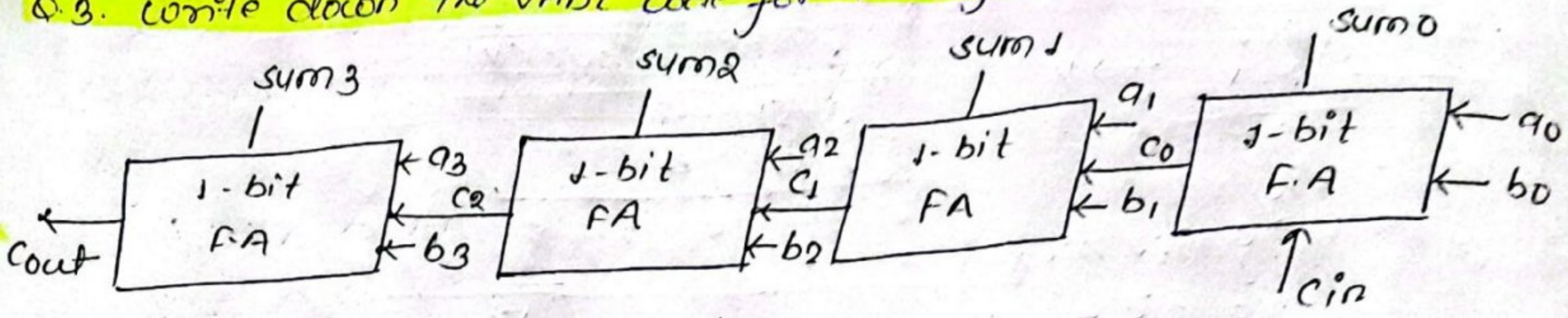

```
library ieee;
use ieee.std_logic_1164.all;
entity sag_fa is
port (a,b,c : in std_logic;
      sum,carry : out std_logic);
end sag_fa;

architecture fa_sag of sag_fa is
component sag_ha
port (a,b : in std_logic;
      y : out std_logic);
end component;

component sag_or
port (a,b: in std_logic;
      z : out std_logic);
end component;

signal s1, s2, s3 : std_logic;
begin
x1 : sag_ha port map (a,b, s1, s2);
x2 : sag_ha port map (s1, c, s3, sum);
x3 : sag_or port map (s2, s3, carry);
end fa_sag;
```

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Q.3. Write down the VHDL code for 4 bit full adder using VHDL.

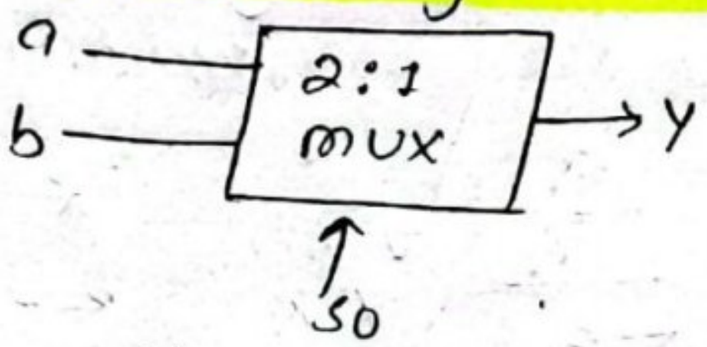



```
library ieee;
use ieee.std_logic_1164.all;
entity sag_fa is
port (a,b : in std_logic_vector (3 down to 0);
      sum : out std_logic_vector (3 down to 0);
      cin : in std_logic;
      Cout : out std_logic);

end sag_fa;
architecture fa_sag of sag_fa is
component sag_fa
port (a,b,cin : in std_logic;
      sum, carry : out std_logic);

end component;
signal c0, c1, c2 : std_logic;
begin
X1 : sag_fa portmap (a(0), b(0), cin, c0, sum(0));
X2 : sag_fa portmap (a(1), c0, b(1), sum(1), c1);
X3 : sag_fa portmap (a(2), b(2), c1, sum(2), c2);
X4 : sag_fa portmap (a(3), b(3), c2, cout, sum(3));
end fa_sag;
```

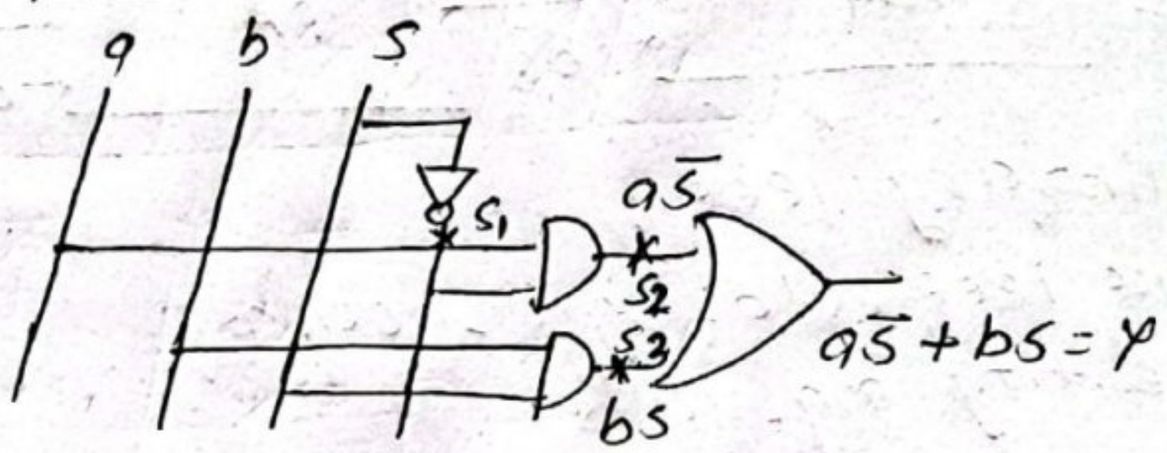
CamScanner

Available on notes.arpanadhikari7.com.np

4. VHDL code for 2:1 mux



S	output (Y)
0 ($\bar{S}$)	a
1 (S)	b



the eqn of 2:1 mux is

$$Y = a\bar{S} + bS$$

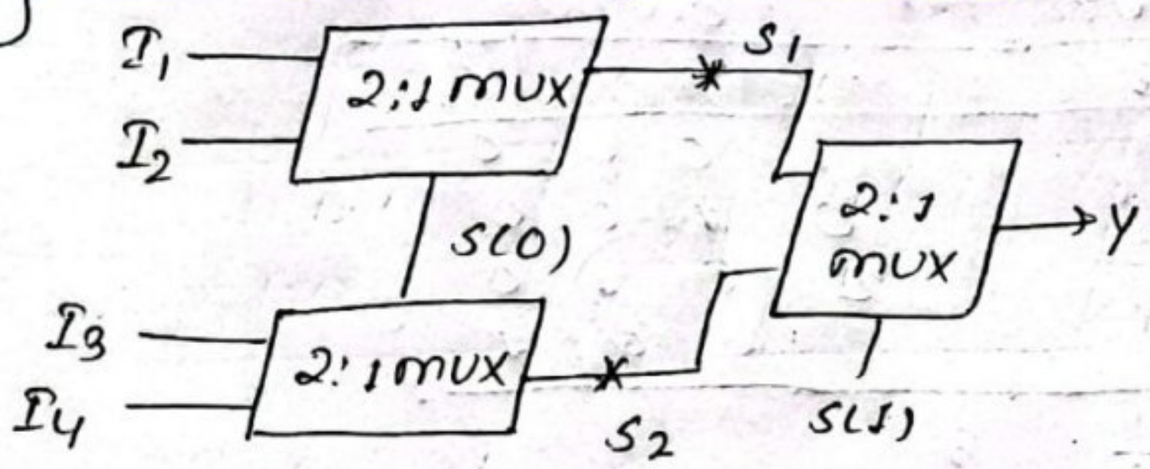
```
library ieee;
use ieee.std_logic_1164.all;
entity sag_21mux is
port(a,b,s: in std_logic;
        Y: out std_logic);
end sag_21mux;
architecture 21mux_sag of sag_21mux is
component sag_and
 port (a,b: in std_logic;
         x: out std_logic);
 end component;
component sag_or
 port (a,b: in std_logic;
         y: out std_logic);
 end component;
component sag_not
 port (p: in std_logic;
         p1: out std_logic);
 end component;
signal s1,s2,s3: std_logic;
begin
x1: sag_and portmap (a, s1, s2);
x2: sag_and portmap (b, s, s3);
x3: sag_or portmap (s2, s3, Y);
x4: sag_not portmap (s0, s1);
end 2imux_sag;
```

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2018 Spring 5. 4:1 mux using 2:1 mux.




```
library ieee;
use ieee.std_logic_1164.all;
entity sag_4x1mux is
port ( I1, I2, I3, I4 , S(0), S(1) : in std_logic;
       Y : out std_logic);
end sag_4x1mux
architecture 4x1mux_sag of sag_4x1mux is
 component sag_2x1mux
 port ( I1, I2, S(0) : in std_logic;
        X : out std_logic);
end component;
signal s1, s2 : std_logic;
begin
 x1 : sag_2x1mux portmap ( I1, I2, S(0), s1);
 x2 : sag_2x1mux portmap ( I3, I4, S(0), s2);
 x3 : sag_2x1mux portmap ( s1, s2, S(1), Y);
end 4x1mux_sag;
```

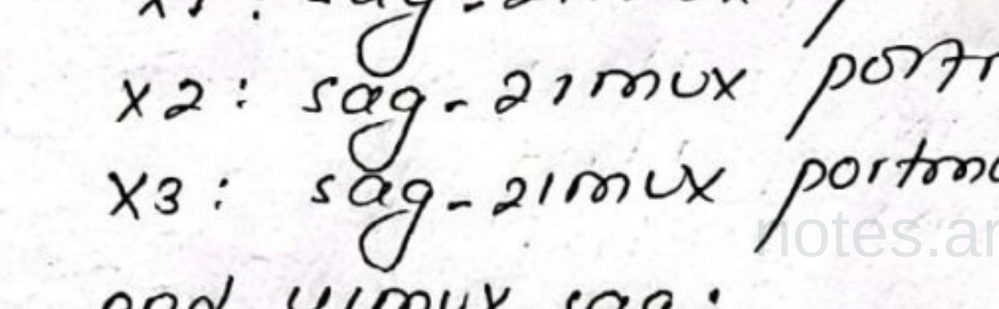
notes.arpanadhikari7.com.np

De-multiplexer

→ 1:2^n format

→ n = no. of select line

→ Block diagram:



E = Enable line.

→	i/p		o/p			
E	S_0	S_1	a	b	c	d
1	0	0	I	0	0	0
1	0	1	0	I	0	0
1	1	0	0	0	I	0
1	1	1	0	0	0	I
0	x	x	0	0	0	0

Here,
$a = \bar{S_0}\bar{S_1}I$
$b = \bar{S_0}S_1I$
$c = S_0\bar{S_1}I$
$d = S_0S_1I$

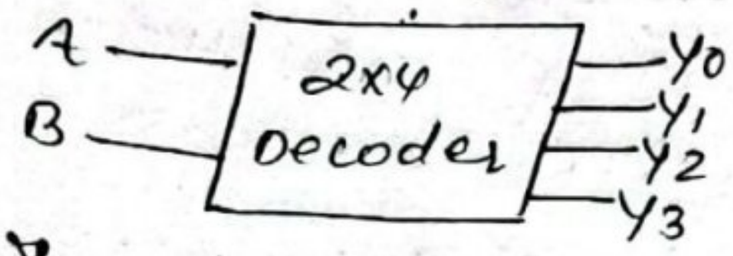
When Enable = 0, all, abcd = 0000
When Enable = 1, then only, S_0 and S_1 work

CamScanner

CamScanner

Available on notes.arpanadhikari7.com.np

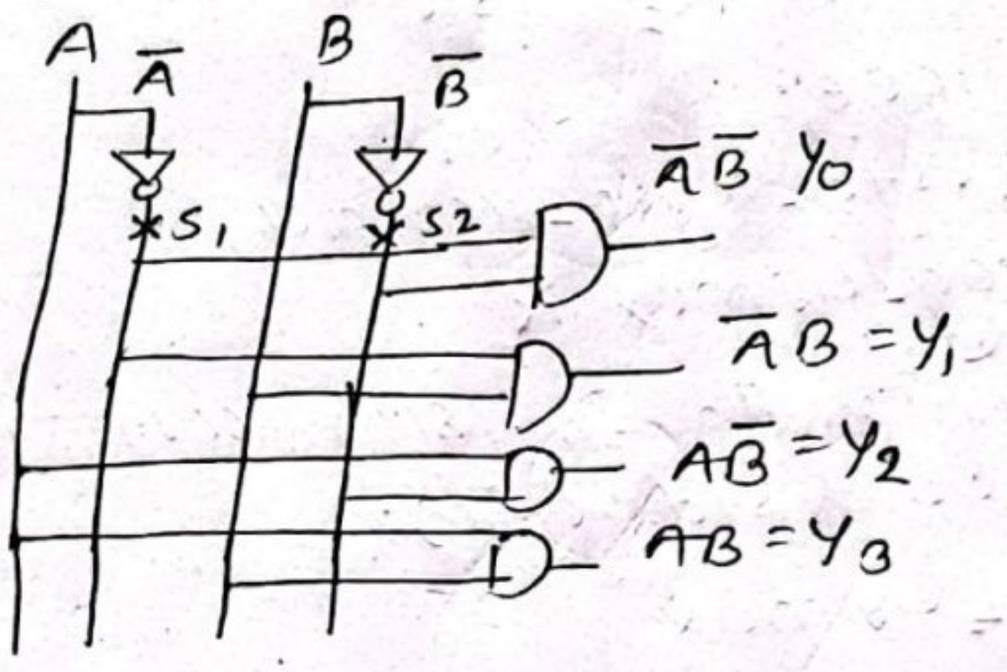
6. VHDL code for 2x4 decoder.



$Y_0 = \bar{A}\bar{B}$ $Y_2 = A\bar{B}$

$Y_1 = \bar{A}B$ $Y_3 = AB$

input		output			
A	B	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1




```
library ieee;
use ieee.std_logic_1164.all;
entity sag_2xdecoder is
port (a,b : in std_logic;
      Y0,Y1,Y2,Y3 : out std_logic);
end sag_2xdecoder;
architecture 2xdecoder_sag of sag_2xdecoder is
 component sag_and
  port (a,b : in std_logic;
        X : out std_logic);
  end component;

component sag_not
  port (a : in std_logic;
        Y : out std_logic);
  end component;
  signal s1, s2 : std_logic;
  begin
 X1 : sag_and portmap (s1, s2, Y0);
 X2 : sag_and portmap (s1, B, Y0);
 X3 : sag_and portmap (A, s2, Y2);
 X4 : sag_and portmap (A, B, Y3);
 X5 : sag_not portmap (A, s1);
 X6 : sag_not portmap (B, s2);
  end 2xdecoder_sag;
```

notes.arpanadhikari7.com.np

CamScanner

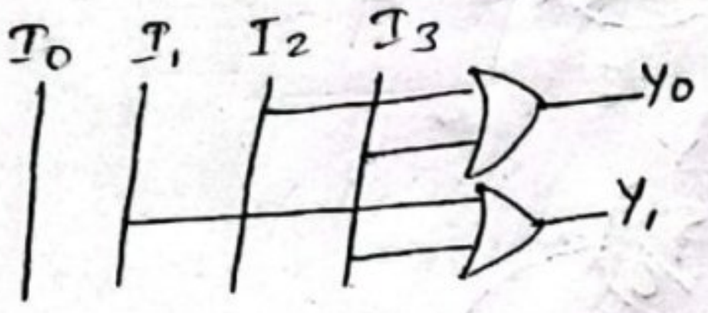
Available on notes.arpanadhikari7.com.np

7. 4x2 Encoder, write VHDL code.

Inputs				outputs	
I_0	I_1	I_2	I_3	Y_0	Y_1
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

$Y_0 = I_2 + I_3$

$Y_1 = I_1 + I_3$




```
library ieee;
use ieee.std_logic_1164.all;
entity sag_u2encoder is
port (I0, I1, I2, I3 : in std_logic;
      Y0, Y1 : out std_logic);
end sag_u2encoder;
architecture u2encoder_sag of sag_u2encoder is
 component sag_or
  port ( I0, I1 : in std_logic;
         Y : out std_logic);
  end component;

 begin
X1: sag_or portmap (I2, I3, Y0);
X2: sag_or portmap (I1, I3, Y1);
end u2encoder_sag;
```

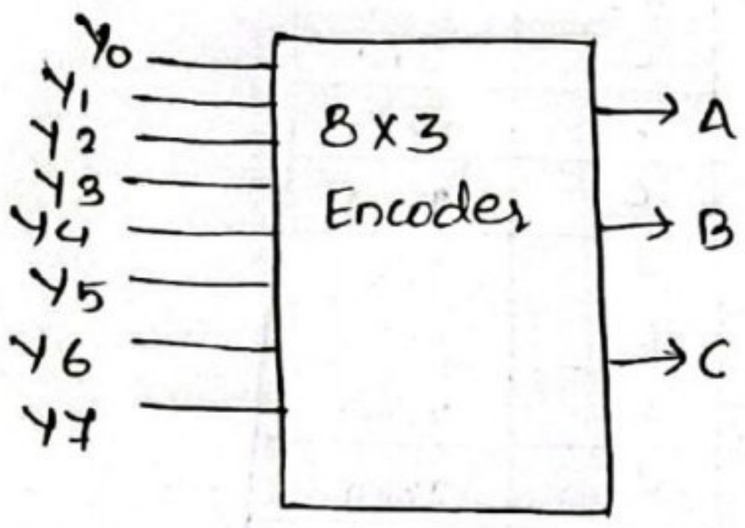
notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2015 Fall write VHDL code for octal to binary encode (8x3) encoder

Solution:



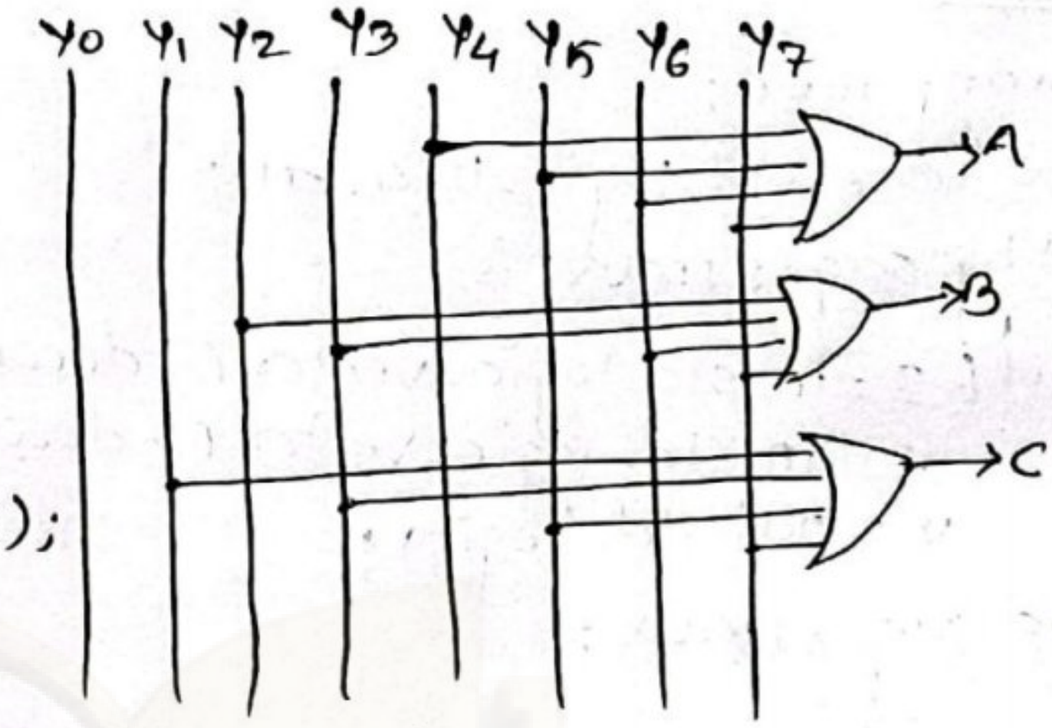
inputs								outputs		
Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7	A	B	C
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Here,

$A = Y4 + Y5 + Y6 + Y7$

$B = Y2 + Y3 + Y6 + Y7$

$C = Y1 + Y3 + Y5 + Y7$



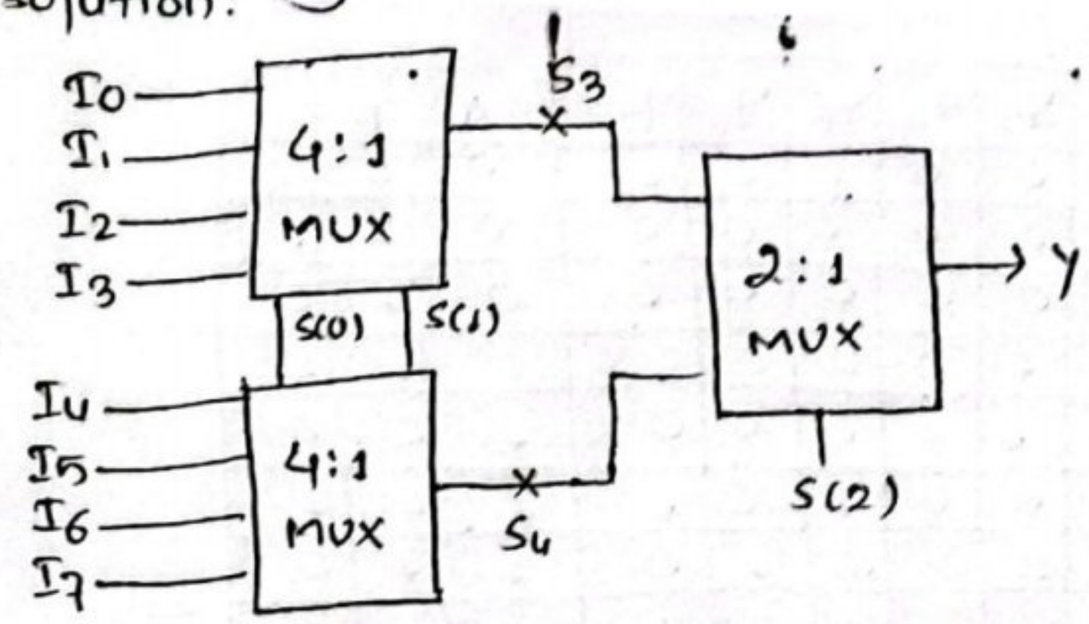

```
library ieee;
use ieee.std_logic_1164.all;
entity sag_8encoder is
port (Y: in std_logic_vector(7 downto 0);
      A,B,C: out std_logic);
end sag_8encoder;

architecture 8encoder_sag of sag_8encoder is
 component sag_or
 port(a,b,c,d: in std_logic;
         x: out std_logic);
 end component;

begin
 x1: sag_or portmap ( Y(4), Y(5), Y(6), Y(7), A);
 x2: sag_or portmap (Y(2), Y(3), Y(6), Y(7), B);
 x3: sag_or portmap (Y(1), Y(3), Y(5), Y(7));
end 8encoder_sag;
```

CamScanner

Available on notes.arpanadhikari7.com.np

2017 spring write down the VHDL code to simulate 8x1 MUX.

solution:



Truth Table

S(0)	S(1)	S(2)	Y
0	0	0	I(0)
0	0	1	I(1)
0	1	0	I(2)
0	1	1	I(3)
1	0	0	I(4)
1	0	1	I(5)
1	1	0	I(6)
1	1	1	I(7)

Code:

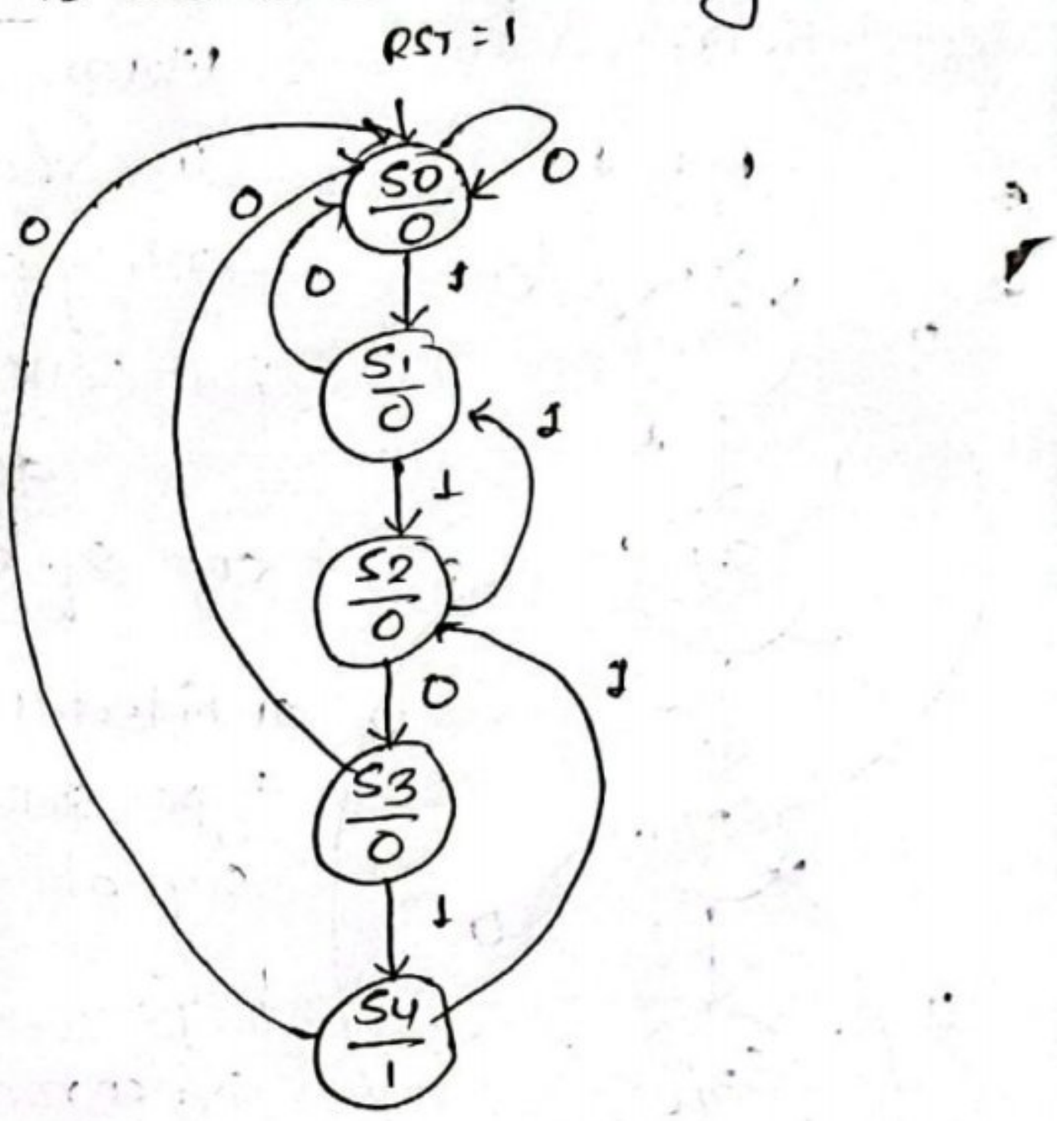
```
library ieee;
use ieee.std_logic_1164.all;
entity sag_8x1mux is
port( I: in std_logic_vector (7 down to 0);
      s: in std_logic_vector (2 down to 0);
      y: out std_logic);
end sag_8x1mux;

architecture 8x1mux_sag of sag_8x1 mux is
 component sag_4x1mux
 port (a,b,c,d: in std_logic;
       s0, s1 : in std_logic;
       z: out std_logic);
  end component;

component sag_2x1 mux
port (a,b: in std_logic;
      s: in std_logic;
      x: out std_logic);
end component;

signal s3, s4 :std_logic;
begin
x1: sag_4x1mux portmap ( I(0), I(1), I(2), I(3), s(0), s(1), s3);
x2: sag_4x1mux portmap ( I(4), I(5), I(6), I(7), s(0), s(1), s4);
x3: sag_2x1mux portmap ( s3, s4, s(2), Y);
end 8x1mux_sag;
```

CamScanner

Available on notes.arpanadhikari7.com.np

2019 spring
2017 fall
2016 fall

write down the VHDL program for which output will be '1' when sequence 1101 is detected considering the overlapping condition.

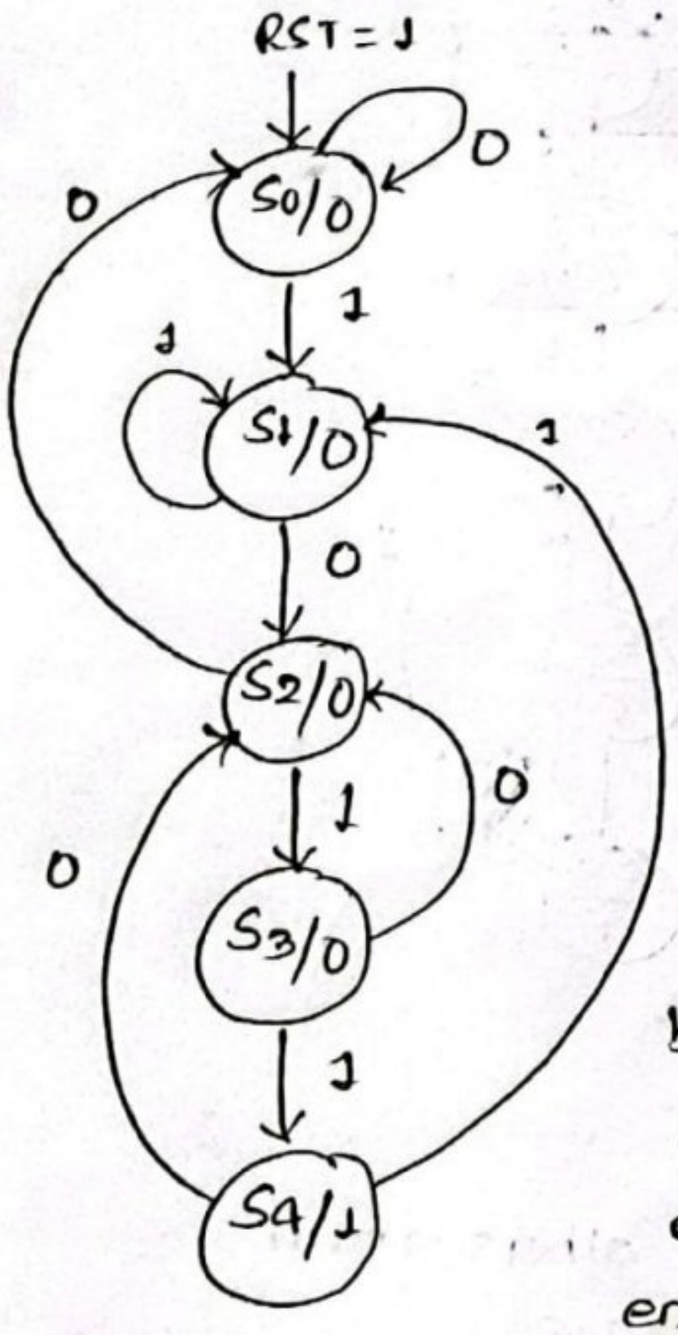



```
Library ieee;
use ieee.std_logic_1164.all;
entity seq_dect is
port (clk, rst, din : in std_logic;
      dout : out std_logic);
end seq_dect;

architecture dect_seq of seq_dect is
type state_type is (s0, s1, s2, s3, s4);
signal cs, ns : state_type;
begin
synchronous_process : process (clk, rst)
begin
  if (rst = '1') then cs <= s0;
  else
    cs <= ns;
  end if;
  end process;

c1: process ( cs, din)
begin
   case cs is

when s0 =>
   if(din = '1') then
      ns <= s1 ;
   else
     ns <= s0;
   endif;

when s1 =>
   if (din = '1') then
      ns <= s2;
   else
     ns <= s0;
   endif;

when s2 =>
   if (din = '0') then
     ns <= s3;
   else
     ns <= s1;
   endif;

when s3 =>
   if (din = '1') then
     ns <= s4;
   else
     ns <= s0;
   endif;

when s4 =>
   if (din = '1') then
     ns <= s2;
   else
     ns <= s0;
   endif;

when others => null
 end case;
 end process;

c2 : process ( cs)
begin
   if ( cs = s4 ) then
     dout <= '1';
   else
     dout <= '0';
   end if;
 end process;
end dect_seq;
```

SEQUENCE DETECTOR

notes.arpanadhikari7.com.np
CamScanner
CamScanner
Available on notes.arpanadhikari7.com.np

2019 Fall write down VHDL program for which output will be 1 when the sequence is 1011 is detected considering overlapping condition.




```
library ieee;
use ieee.std_logic_1164.all;
entity seq_dect is
port (clk, rst, din : in std_logic;
      dout: out std_logic);
end sq_dect;

architecture behaviour seq_dect is
 type state_type is (s0, s1, s2, s3, s4);
 signal cs, ns : state_type;
 begin
  synchronous_process : process (clk, rst)
begin
 if (rst = '1') then cs <= s0;
 else cs <= ns;
end if;
end process;

c1: process (cs, din)
begin
case cs is
when s0 => if (din = '1') then
      ns <= s1;
  else
      ns <= s0;
   end if;
when s1 => if (din = '0') then
      ns <= s2;
  else
      ns <= s1;
  end if;
when s2 => if (din = '1') then
      ns <= s3;
  else
      ns <= s0;
  end if;
when s3 => if (din = '1') then
      ns <= s4;
  else
      ns <= s2;
  end if;
when others => null
end case;
end process;

c2: process (cs)
begin
  if (cs = s4) then dout <= '1';
  else
      dout <= '0';
  end if;
end process;
end dect_seq;
```

notes.arpanadhikari7.com.np

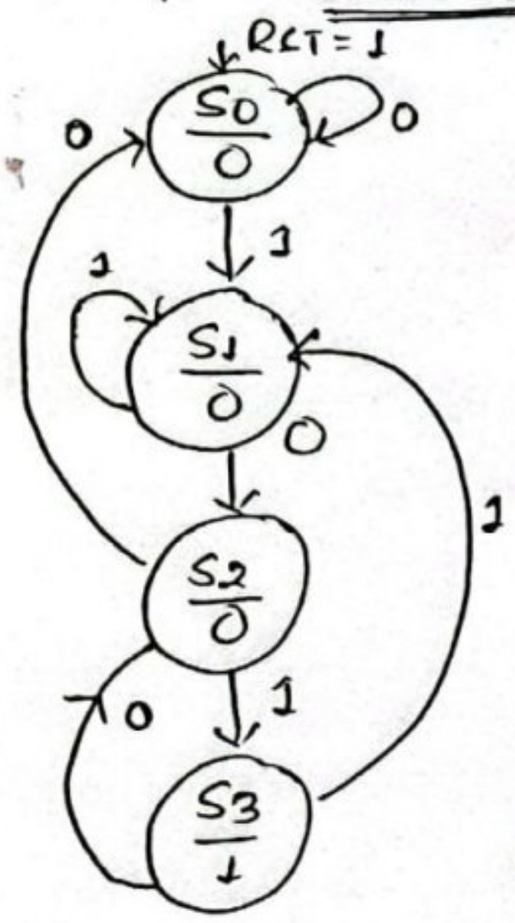
CamScanner

Available on notes.arpanadhikari7.com.np

2016spring: Write a VHDL program for which output will be 1 when the sequence '101' is detected.

Soln:

State Diagram:



VHDL program

```
library ieee;
use ieee.std_logic_1164.all;
entity seq_dect is
port ( clk, rst, din: in std_logic;
       dout : out std_logic);
end seq_dect;

architecture dect_seq of seq_dect is
type state_type is (s0, s1, s2, s3);
signal cs, ns : state_type;
begin
synchronous_process: process (clk, rst)
begin
if (rst = '1') then cs <= s0;
else cs <= ns;
end if;
end process;

c1: process (cs, din)
begin
case cs is

when s0 =>
  if (din = '1') then
  ns <= s1;
else
   ns <= s0;
end if;

when s1 =>
 if (din = '0') then
  ns <= s2;
else
  ns <= s1;
endif;

when s2 =>
 if (din = '1') then
  ns <= s3;
else
  ns <= s0;
end if;

when s3 =>
 if (din = '1') then
  ns <= s1;
else
  ns <= s2;
endif;

when others => null
end case;
end process;

c2: process ( cs)
begin
if ( cs = s3) then
  dout <= '1';
else
 dout <= '0';
endif;
end process;
end dect_seq;
```

notes.arpanadhikari7.com.np

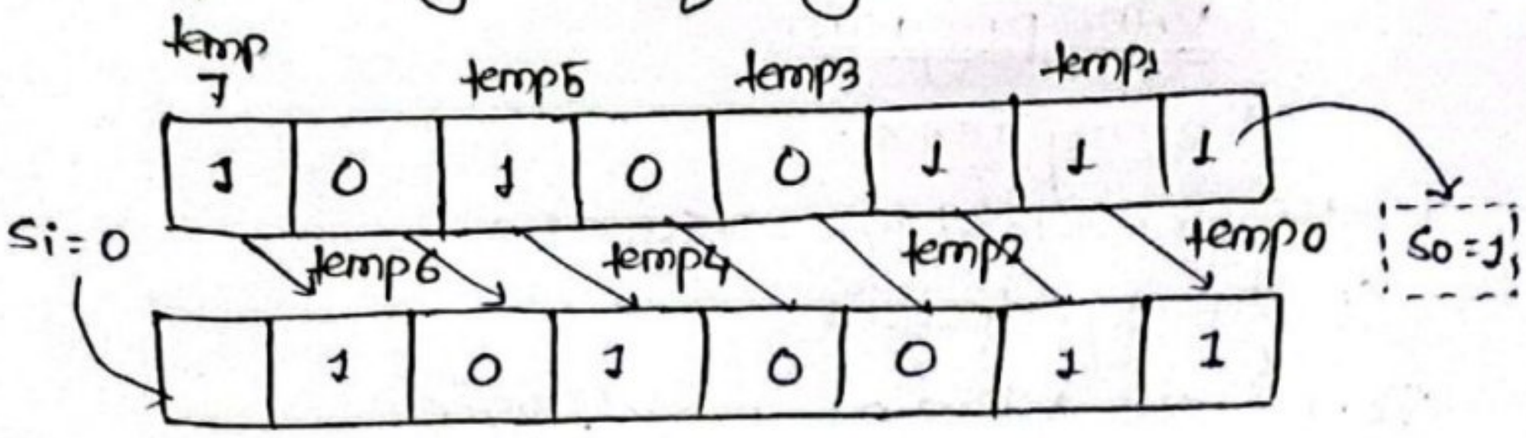
CamScanner

CamScanner

Available on notes.arpanadhikari7.com.np

2017 spring: VHDL code to simulate shift register.

soln: Right shift register (8-bit)




```
library ieee;
use ieee.std_logic_1164.all;
entity sag_sright is
port (clk, Si : in std_logic;
        So : out std_logic;
    shift_out : out std_logic_vector (7 down to 0));
end sag_sright;
architecture sright_sag of sag_sright is
signal temp : std_logic_vector (7 down to 0) := "10100111";
begin
  process (clk)
begin
if (clk = '1') then
  temp(7) <= Si;
  temp(6) <= temp(7);
  temp(5) <= temp(6);
  temp(4) <= temp(5);
  temp(3) <= temp(4);
  temp(2) <= temp(3);
  temp(1) <= temp(2);
  temp(0) <= temp(1);
  So <= temp(1);
end if;
end process;
 shift_out <= temp;
end srighd_sag;
```

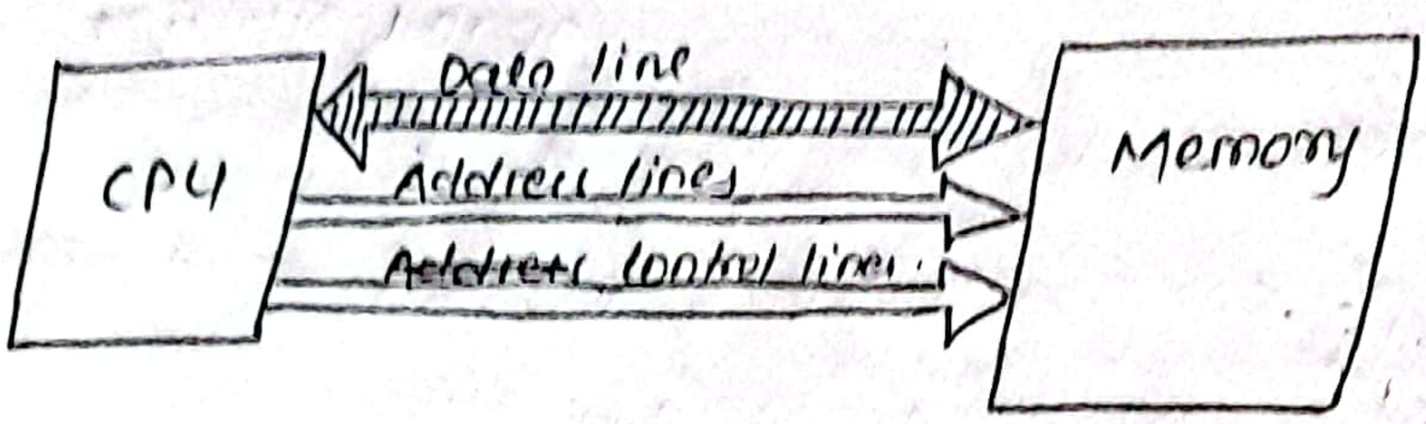
notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Chapter-3 MEMORY

- memory is a storaging device.
- It stores instructions as well as data.
- memory is connected with the processor through the following lines
  - → Address lines (unidirectional)
  - → Data lines (bidirectional)
  - → control lines (unidirectional)



- memory at the basic level can be classified as:
  - i. Processor memory (Register array)
  - ii. Internal on-chip memory.
  - iii. Primary memory (static or dynamic)
  - iv. cache memory
  - v. secondary memory (Hard Disk, CDROM) → (power switch off हुँदा erase) → volatile memory / Non volatile memory. (power switch off हुँदा नि केही हुँदैन ।)

Specification of memory:

1. The storage capacity: The number of bits / bytes or words it can store.
2. The memory access time (read access and write access)
3. The power consumption and voltage levels: The lesser is the power consumption the more is packing density.
4. size: size is directly related to the power consumption and data storage capacity.

Note:
- Bit → smallest unit (0 or 1)
- Nibble → 4 bits
- Byte → 8 bits

1 KB → 1024 Bytes (2^{10})
1 MB → 1024 KB (2^{20})
1 GB → 1024 MB (2^{30})
1 TB → 1024 GB (2^{40})

notes.arpanadhikari7.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

There are two important specifications for the memory as far as Real Time Embedded systems are:

3.1.
1) write ability
2) storage performance/permanance.

1) write ability: It is the manner and speed that a particular memory can be written. The memory write ability refers to the process of putting the bits in specific location of the memory and the ease and speed with which the process can be completed. On the basis of writing ability, memory can be

a) High End: The memory with high end write ability is easiest one to write data into. These are flip-flop (synchronous storage device.) based memory that processor can write directly into. Example are: RAMs. These memory are used in computation process in embedded system.

b) middle Range: The memories in this range are little bit difficult to write and are slower than high end ones. The processor can still write in them, but with slower speed. These memories are not accessed frequently and used for storing the data for a longer period of time. eg: FLASH, EEPROM.

c) Lower Range: A special programmer device is required to write data into this type of memory. It is slower memory. eg: UV EPROM in which data is written by using voltage higher than that of normal operation.

d) low end: In this low end memory device, data is written only during manufacturing. The memory device is manufactured with data into it. The data writing process starts with design of chip. Once manufactured, the data cannot be rewritten.
eg: masked program ROM.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2) storage Performance
It is the ability to hold the stored bit. The ability is temporary or permanent. The range of storage performance are:

a) High End:
The range of memory essentially never loses bits.
eg: mask-programmed ROM, OTPROM (one time PROM)

b) middle Range
This range of memory holds for a bits over certain period of times as days, months, or years after memory's power source turned off. eg: NVRAM, a battery backed RAM (SD card) or flash memory.

c) Lower Range
This range memory holds bits as long as power supplied to memory eg: SRAM.

d) Low End
It begins to lose bits almost immediately after written ie. refreshing circuit is needed to hold data correctly eg. DRAM.

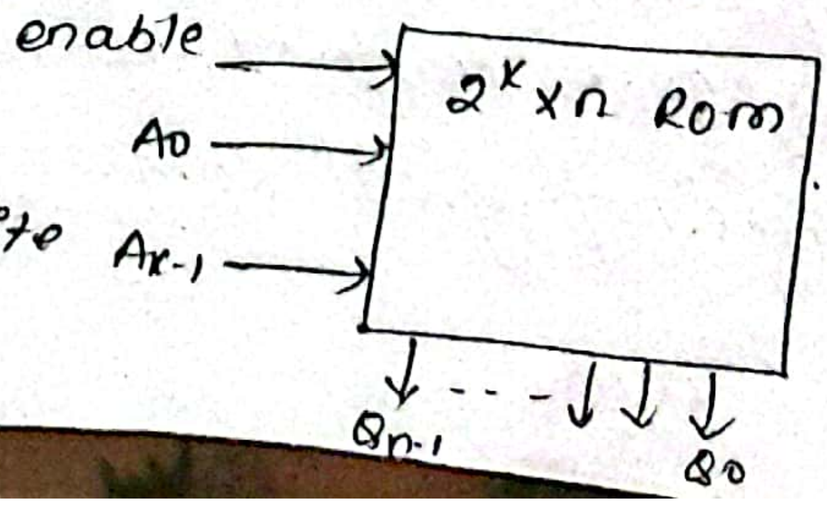
notes.arpanadhikari7.com.np

3.2.
COMMON MEMORY TYPES

1. ROM (Read Only Memory)
2. RAM (Random Access Memory)

1. ROM
→ Read only Memory
→ Nonvolatile memory
→ It can be read from but not write into

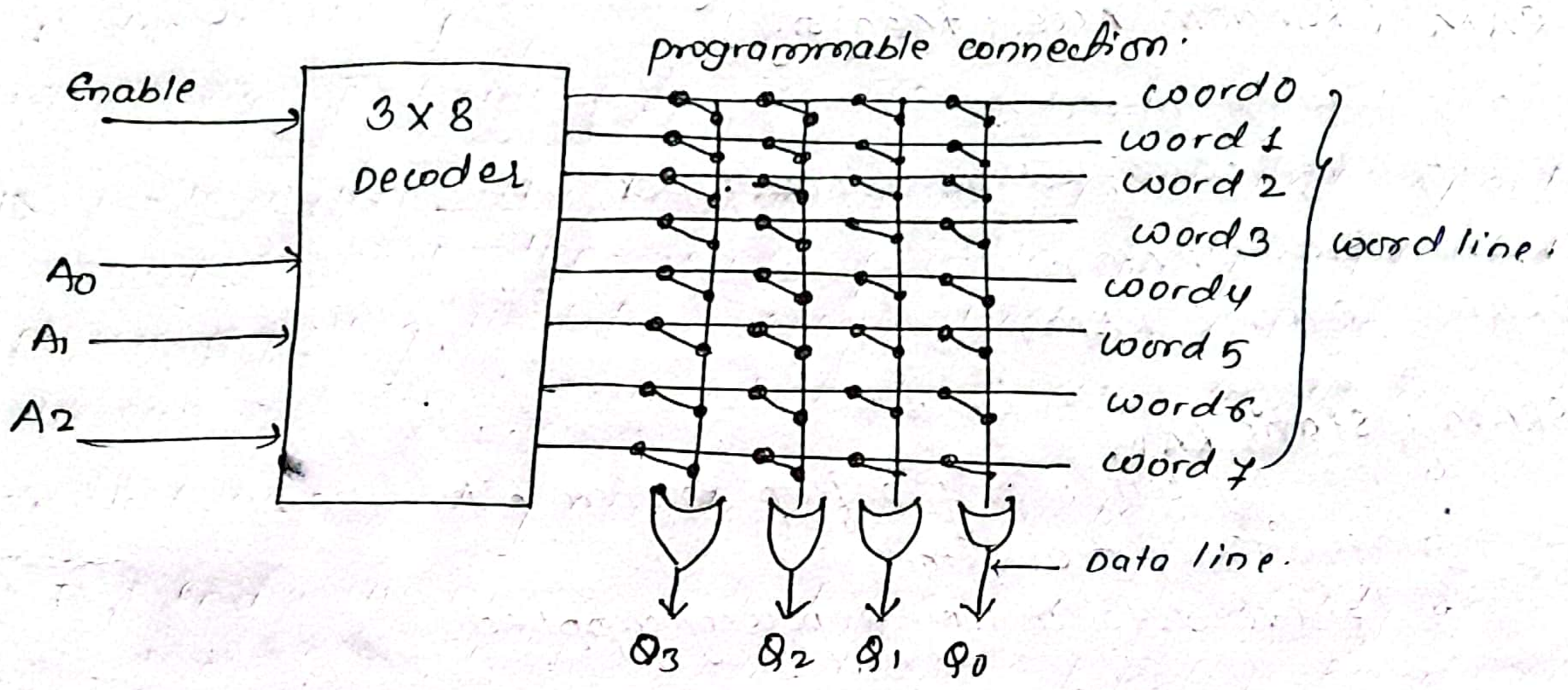
External view



CamScanner
Available on notes.arpanadhikari7.com.np

Internal view

8×4 ROM



1) mask-programmed ROM:

→ The connection is made during manufacturing (fabrication).
→ They are a set of mask
→ It can be written only once.
→ But it stores data forever, thus it has highest storage performance / permanence.

2) OTP ROM (one time Programmable ROM).

→ The connections "programmed" after manufacture by users.
→ The user can modify programs connection at only once.

3) EPROM (Erasable PROM)

→ This is known as erasable programmable read only memory. The programmable component is a MOS transistor.

4) EEPROM

→ Electrically erasable PROM.
→ It is typically erased by using voltage higher than normal level voltage.
→ Better writeability
→ lower storage permanence / performance.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

5) Flash memory → can be serially erased and modified.
→ It is an extension of EEPROM.
→ can be erased with the help of electricity in blocks.

2. RAM (Random Access Memory)
→ Typically volatile memory.
→ RAM is further classified into too types.
a) SRAM (static RAM)
b) DRAM (Dynamic RAM)

a) SRAM: Static RAM
- memory cell uses flip-flop to store bit
- requires 6 transistors
- Holds data as long as power supplied

b) DRAM: Dynamic RAM
- memory cell uses MOS transistor and capacitor to store bit
- more compact than SRAM
- Refreshment circuit required.
- Typical refresh rate 15.625 microsec.
- slower to access than SRAM.

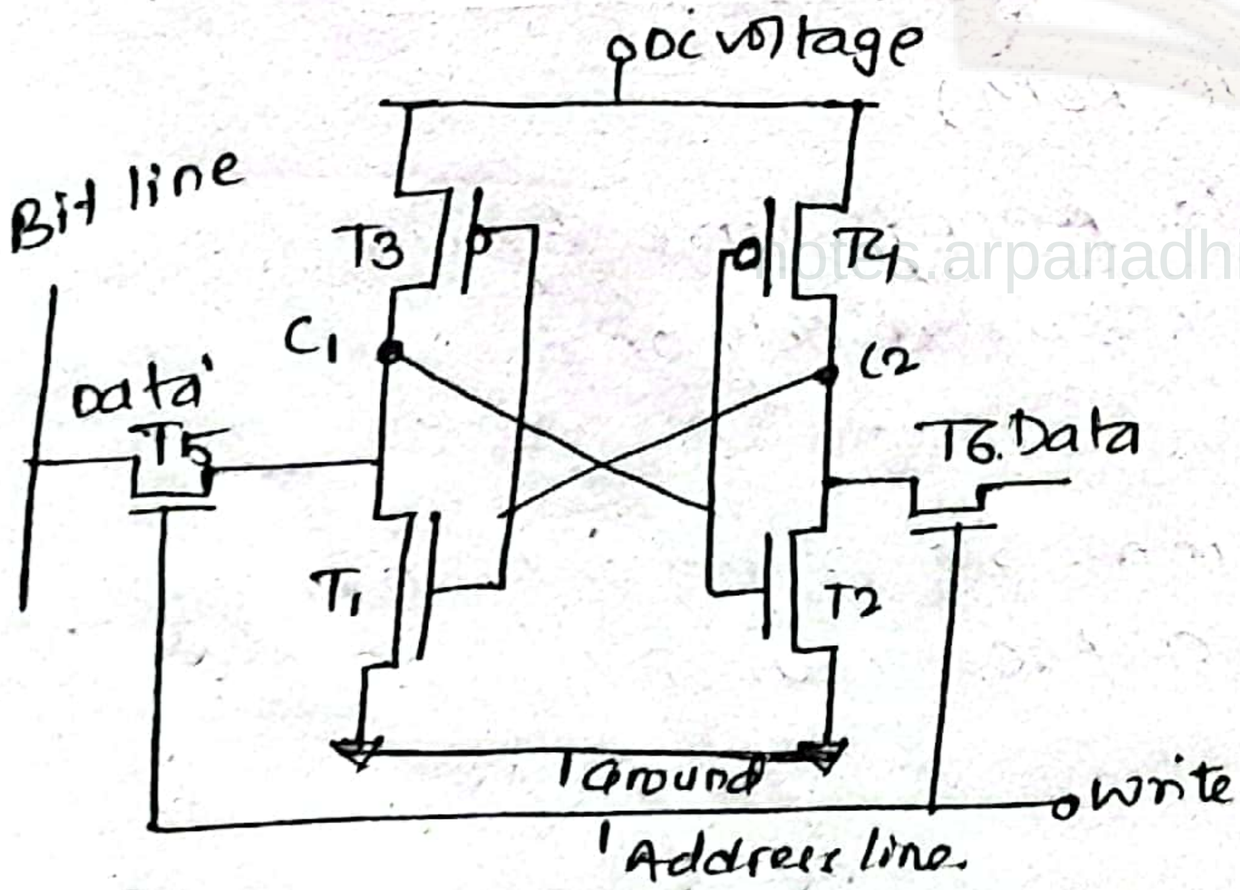
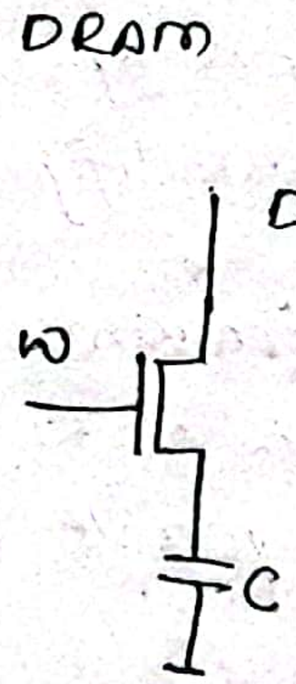


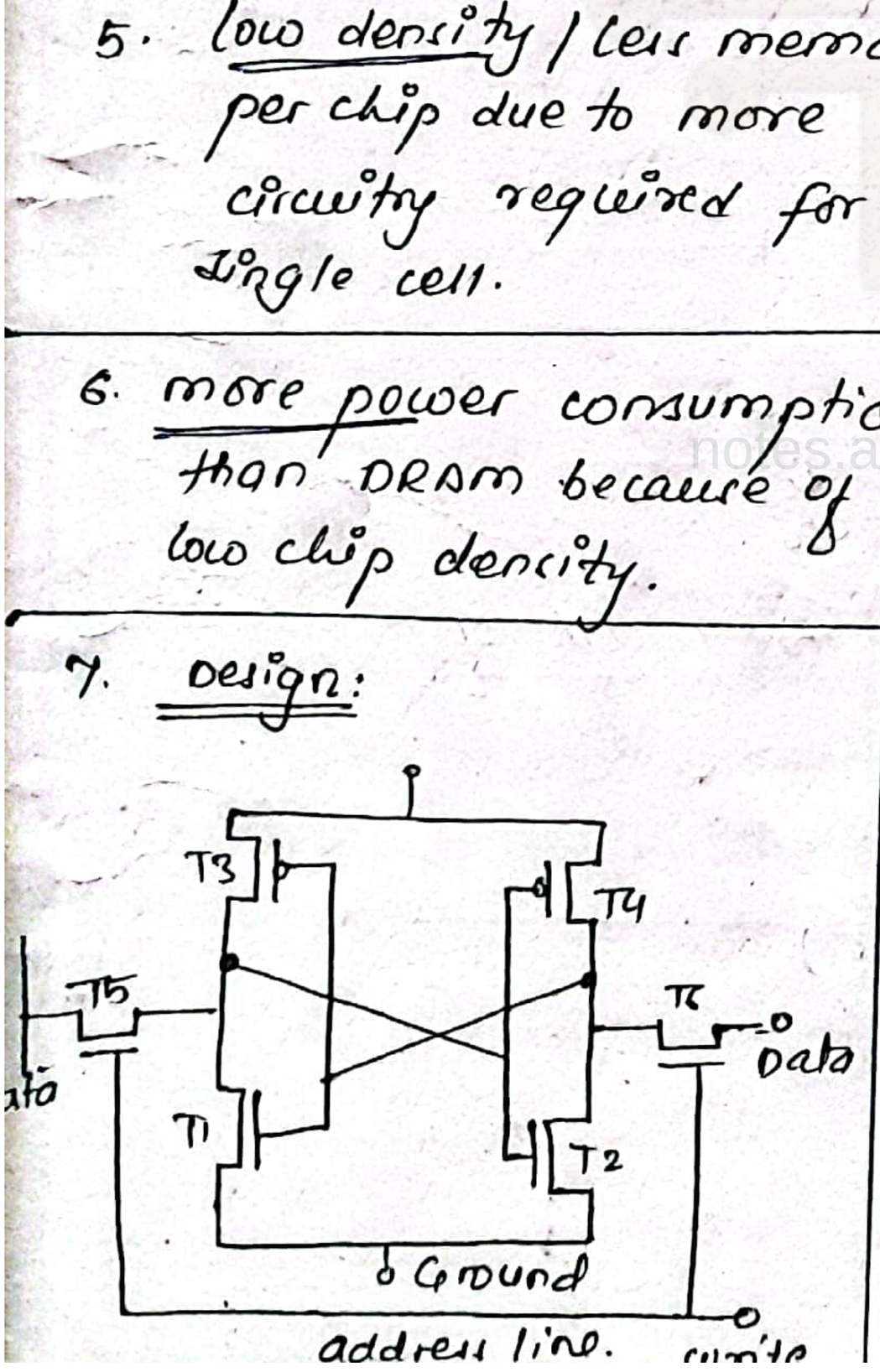
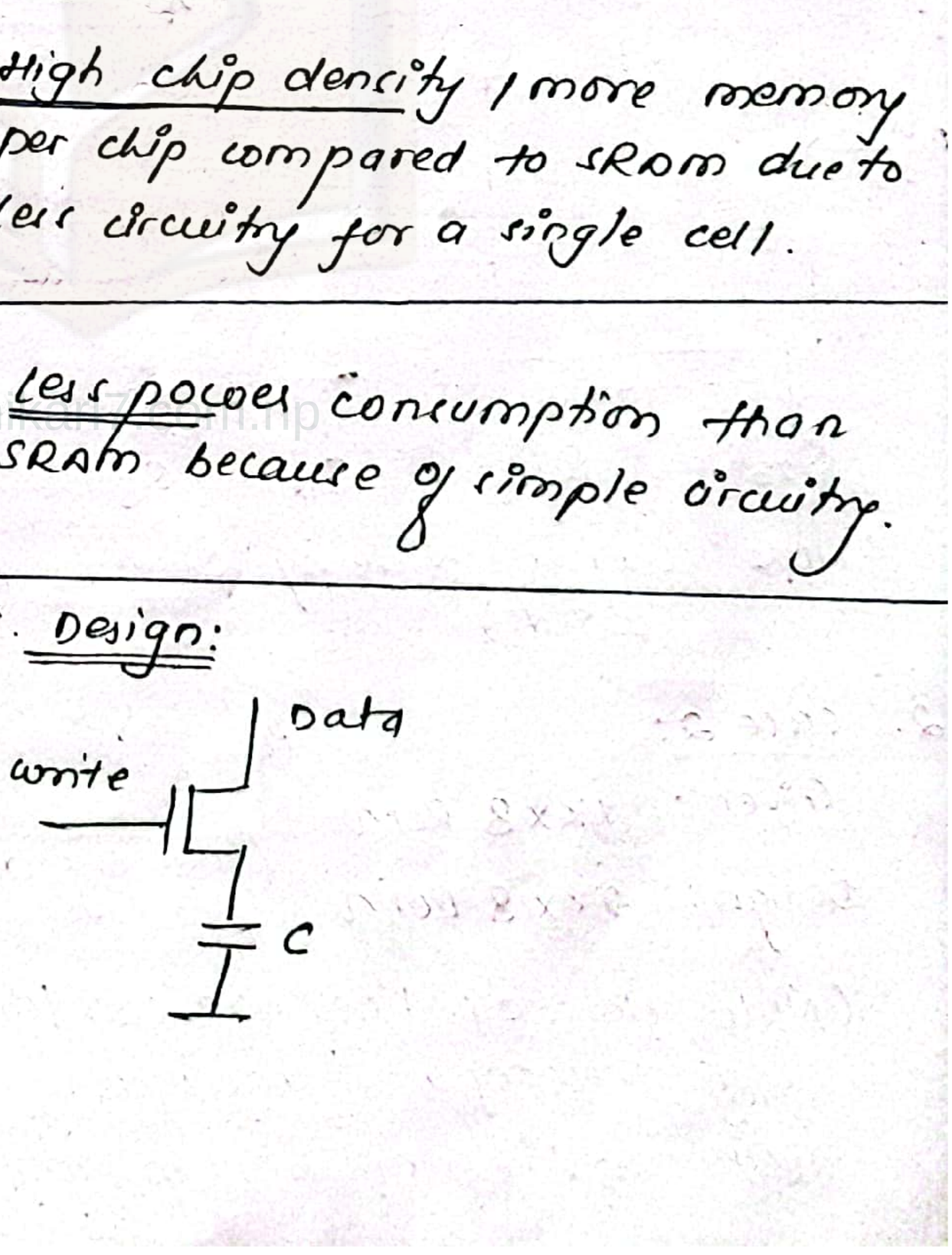
fig: SRAM



notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

SRAM	DRAM
1. Stores bits in memory cells composed to flip flops.	1. memory cells are composed of capacitors and transistor.
2. Each cell which can store a single bit requires 6 transistor.	2. Each cell requires a capacitor (which stores bit as charge) and a transistor.
3. Doesnot need to be refreshed periodically as flip-flop retain the data.	3. Needs to be refreshed every few milliseconds to retain data because the charge of the capacitor leaks.
4. access time <u>faster</u> than DRAM.	4. <u>slower</u> access time compared to SRAM as it cannot be read while being refreshed.
5. <u>low density</u> / less memory per chip due to more circuitry required for a single cell.	5. <u>High chip density</u> / more memory per chip compared to SRAM due to less circuitry for a single cell.
6. <u>more power</u> consumption than DRAM because of low chip density.	6. <u>less power</u> consumption than SRAM because of simple circuitry.
7. <u>Design:</u> 	7. <u>Design:</u> 

CamScanner

Available on notes.arpanadhikari7.com.np

COMPOSING MEMORY

- memory size needed often differs from size of readily available memories.
- when available memory is larger, simply ignore unneeded high-order address bits and higher data lines.
- when available memory is smaller, compose several smaller memories into one larger memory.

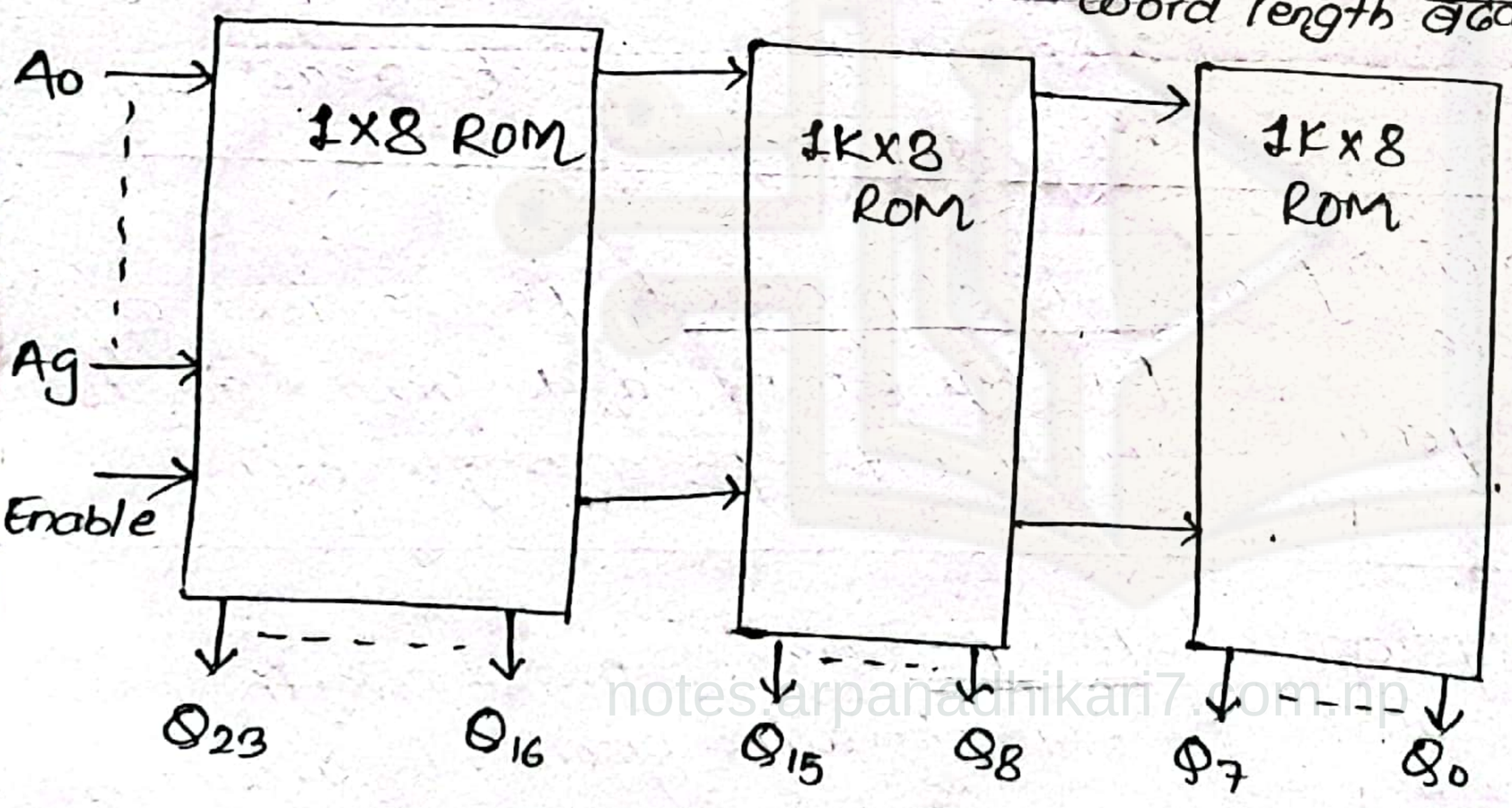
2018 SP
- ✱ connect side-by-side to increase width of words
- ✱ connect top to bottom to increase number of words.

Case 1:

1. 1K x 24 ROM, composite the memory.

We know,

$1K = 2^{10}$ (A0 - A9) (यहाँ memory constant छ, coord length बढेको छ।)



(Note: connect side by side to increase ~~size~~ width of coords)

2. Case 2:

Given: 1K x 8 ROM

Design: 2K x 8 ROM (memory बढेको छ यहाँ)

(Note: Here o/p lines are constant)
memory is increased.

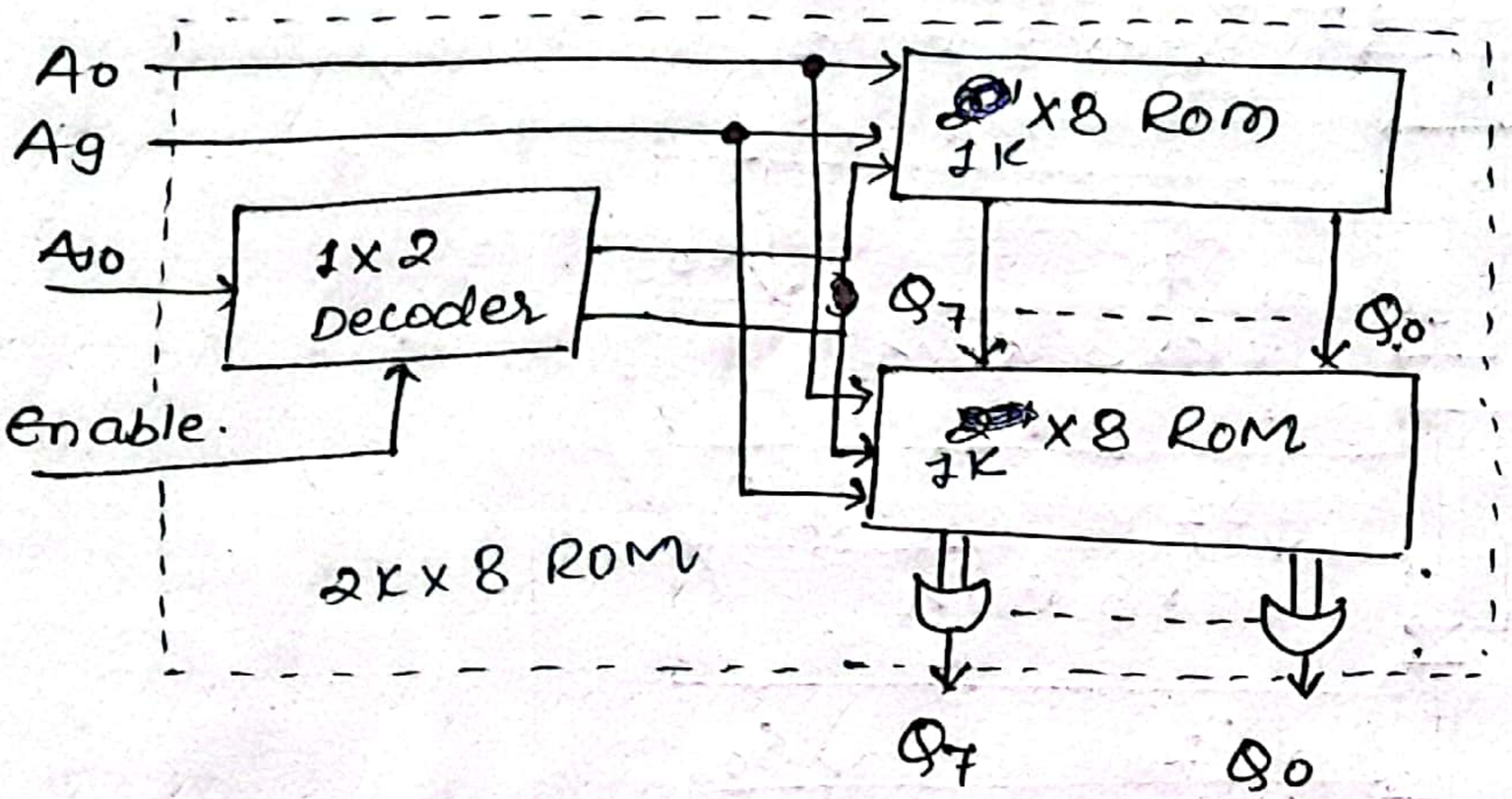
यस्तो case मा parallel setup गर्ने।

Available on notes.arpanadhikari7.com.np

Here,

$1K = 2^{10} = (A_0 - A_9)$

$2K = 2^1 \times 2^{10} = 2^{11}$ $(A_0 - A_{10})$



where,
$m = 11$
$n = 8$.

Case 3: 2K x 16 ROM

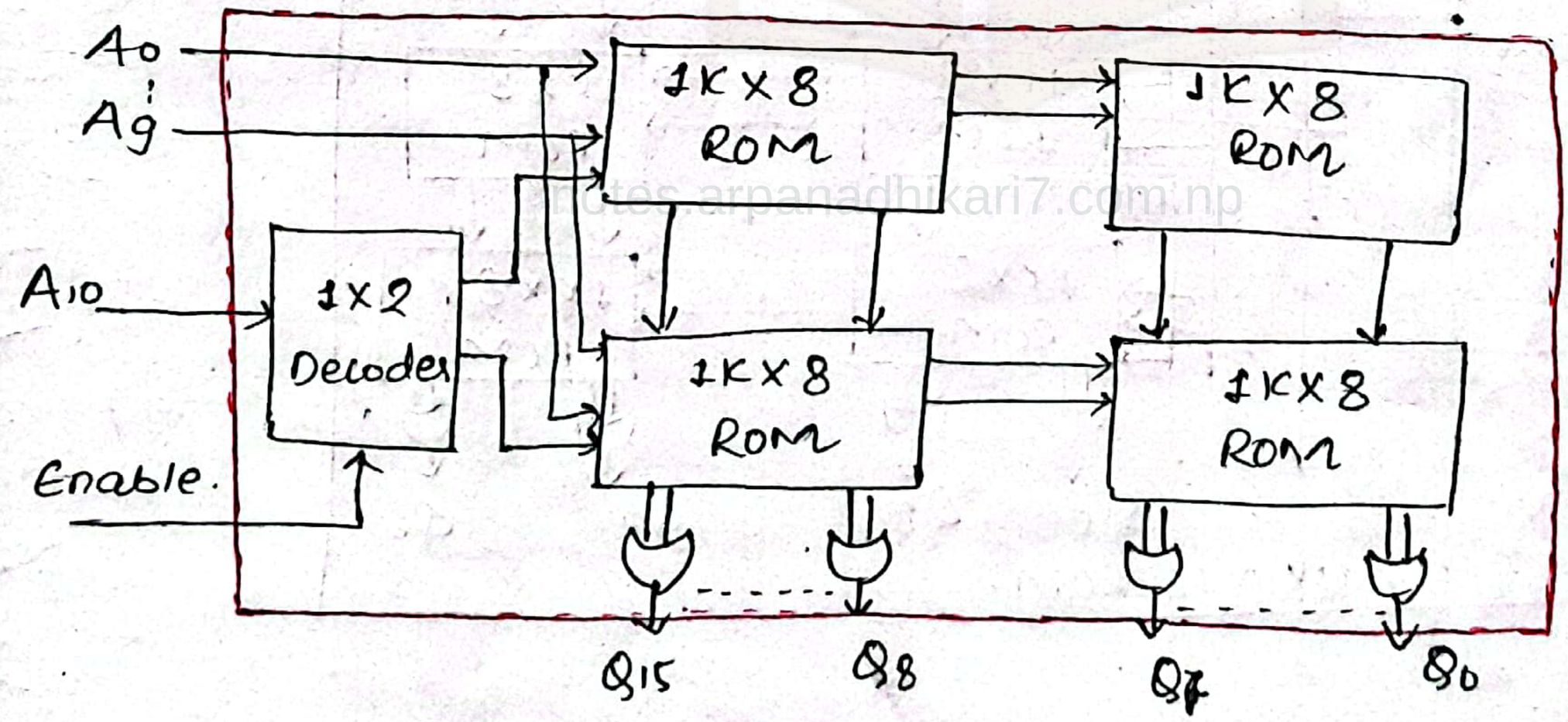
Hint:

- 2K लाई 1K, 1K बनाउने
- 1st को 2K ले memory मनाउँछ
- 2K means parallel मा हुन्छ

[समग्रमा : 2 वटा parallel मा
2 वटा series मा]

- 16 लाई 8, 8 को जोड्ने भन्ने

Soln $2K = 2^1 \times 2^{10} = 2^{11}$ $(A_0 - A_{10})$.



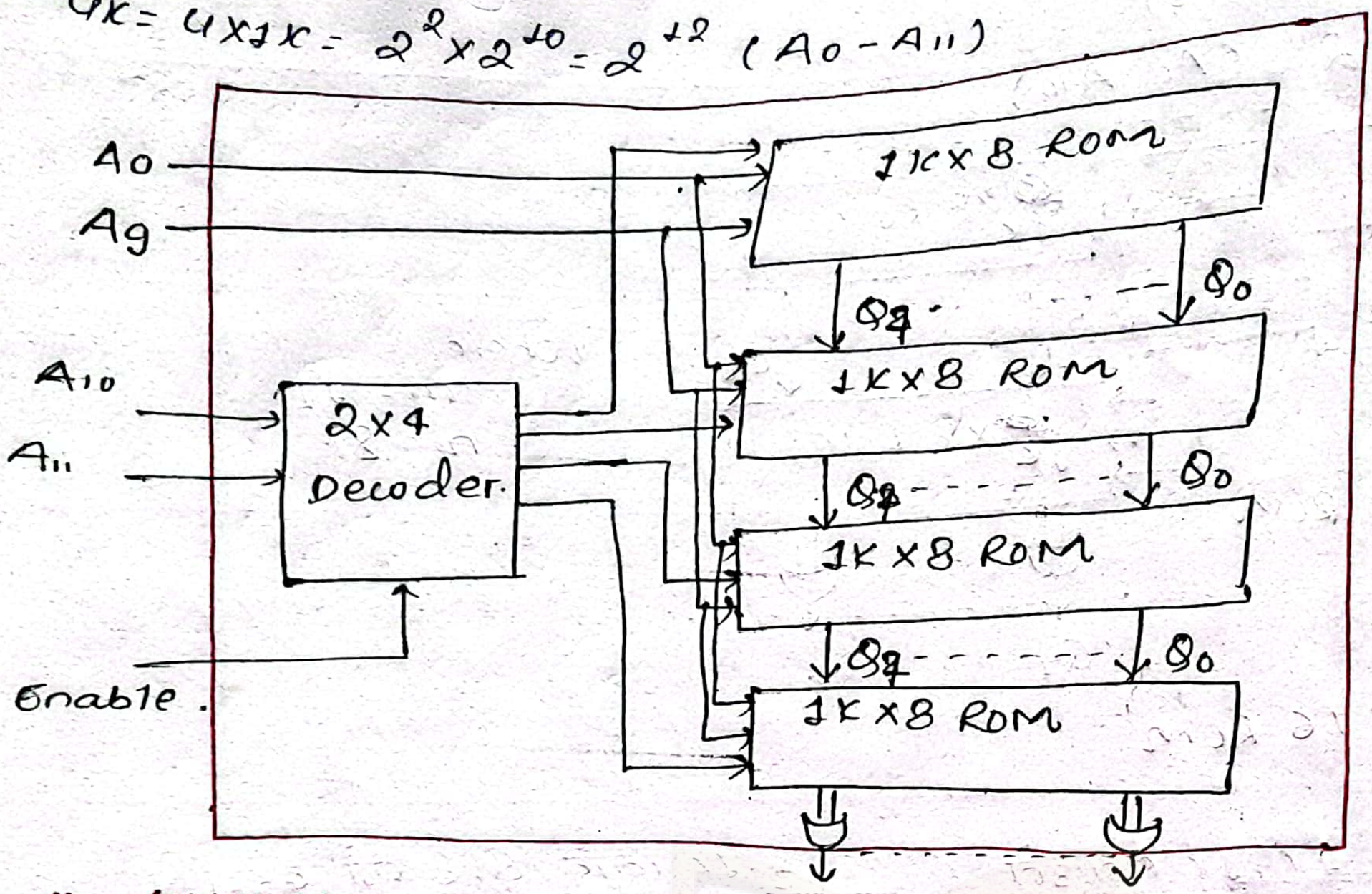
notes.arpanadhikari7.com.np

CamScanner

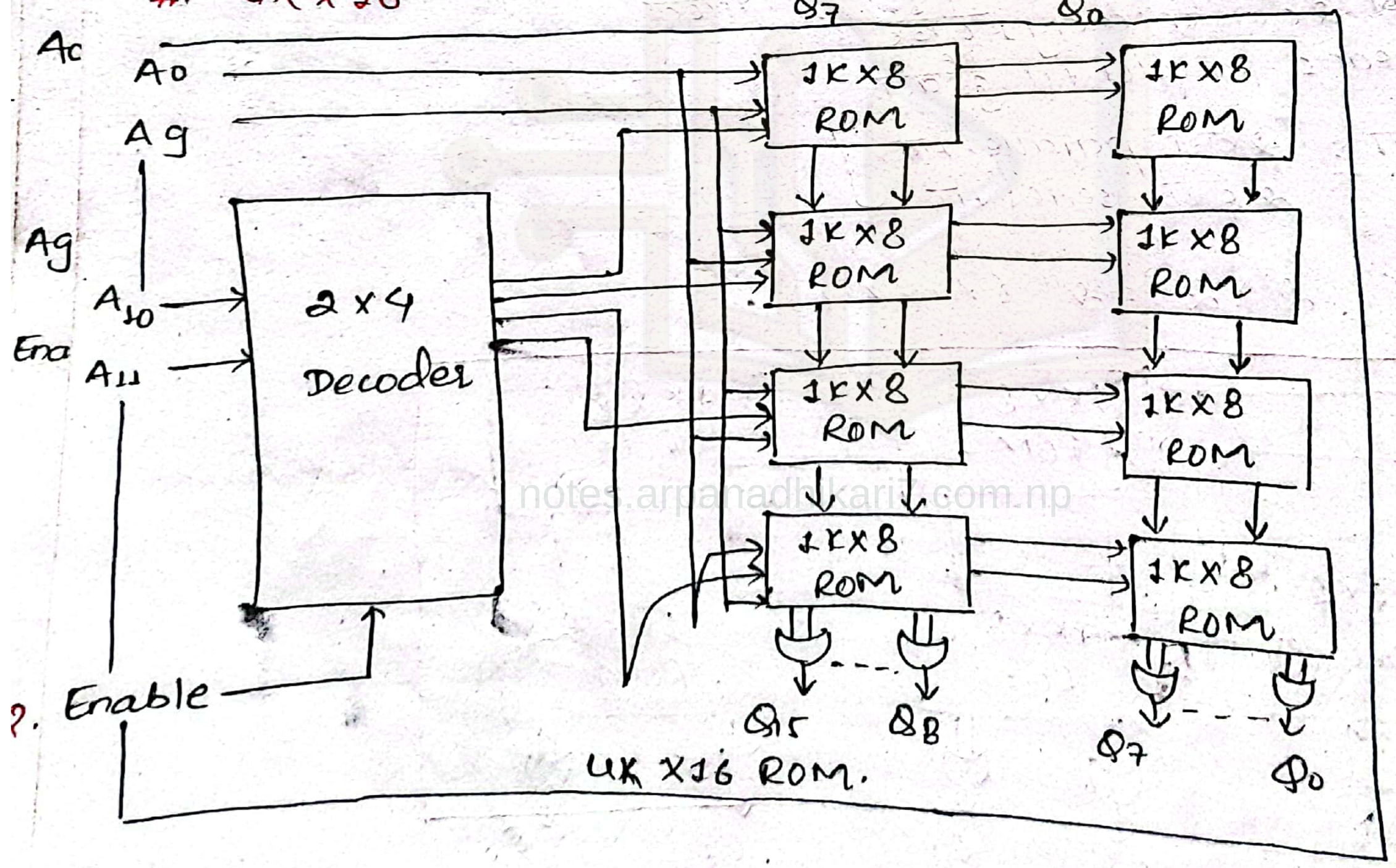
Available on notes.arpanadhikari7.com.np

4K x 8 ROM.

4K = 4 x 1K = $2^2 \times 2^{10} = 2^{12}$ (A0 - A11)



4K x 16



4K x 16 ROM.

notes.arpanadhikari.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

3.4. memory Hierarchy.

→ The memory hierarchy separates computer storage into a hierarchy based on response time.

→ on the basis of performance, capacity, cost, the memories are placed in the hierarchial fashion known as memory hierarchy.

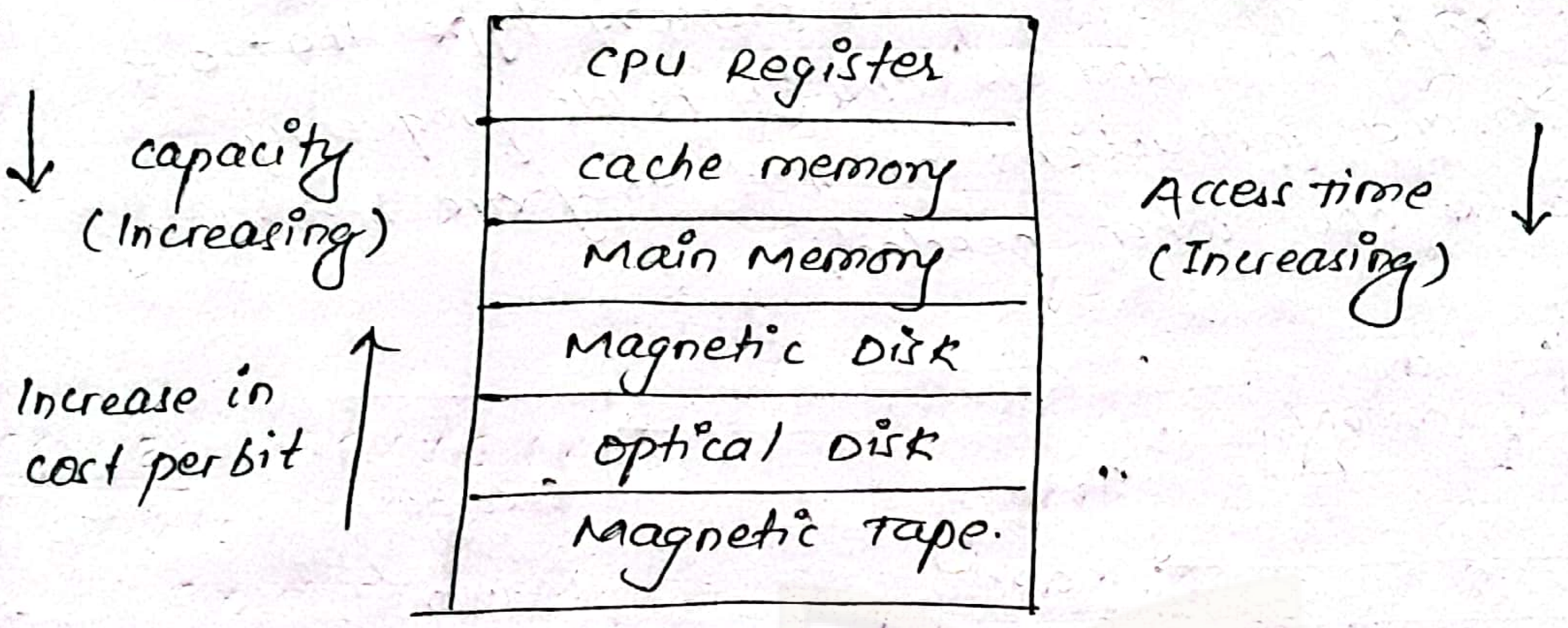


fig: memory Hierarchy.

Main Memory: - large
- Inexpensive
- slow memory stores entire program and data.

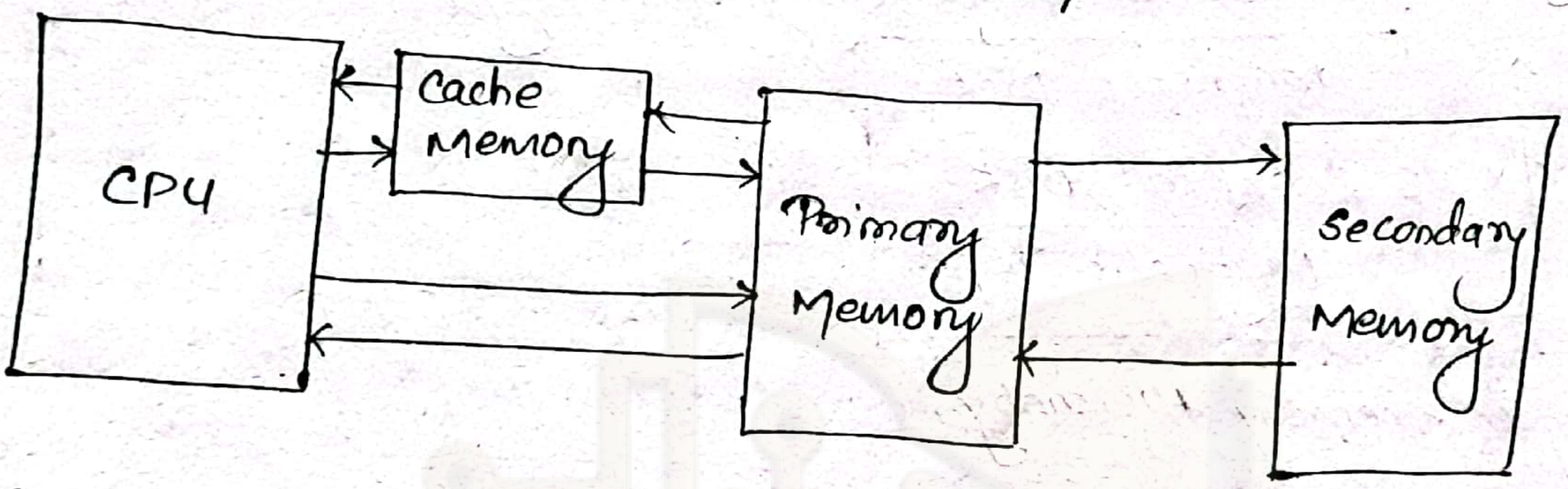
Cache Memory: - small
- expensive
- fast memory stores copy of likely accessed parts of larger memory
- can be multiple levels of cache.

CamScanner

Available on notes.arpanadhikari7.com.np

Cache Mapping

Cache memory is a special very high speed memory. It is used to speed up and synchronizing with high -speed CPU. Cache memory is costlier than main memory or disk memory but economical that CPU registers. Cache memory is an extremely fast memory type that acts as a buffer betn RAM and the CPU. It holds frequently requested data and instructions so that they are immediately available to the CPU when needed.



↳ Levels of memory:
- Level 1 or Register
- Level 2 or cache memory
- Level 3 or main memory
- Level 4 or secondary memory.

Cache performance:

↳ When the processor need to read or write a location in main memory, it first checks for a corresponding entry in the cache.

↳ If the processor finds the memory address location is in the cache, a cache hit has occured and data is read from cache.

↳ If the processor doesnot find the memory location in the cache, a cache miss has occured. For a cache miss, the

notes.arpanadhikari7.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

cache allocates a new entry and copies in data from main memory, then the request is fulfilled from the contents of the cache.

↳ The performance of cache memory is frequently measured in terms of a quantity called as Hit Ratio.

$$\text{Hit Ratio} = \frac{\text{hit}}{(\text{hit} + \text{miss})} = \frac{\text{no of hits}}{\text{Total access}}.$$

↳ we can improve cache performance using higher cache block size, higher associativity, reduce miss rate, reduce miss penalty and reduce the time to hit in the cache.

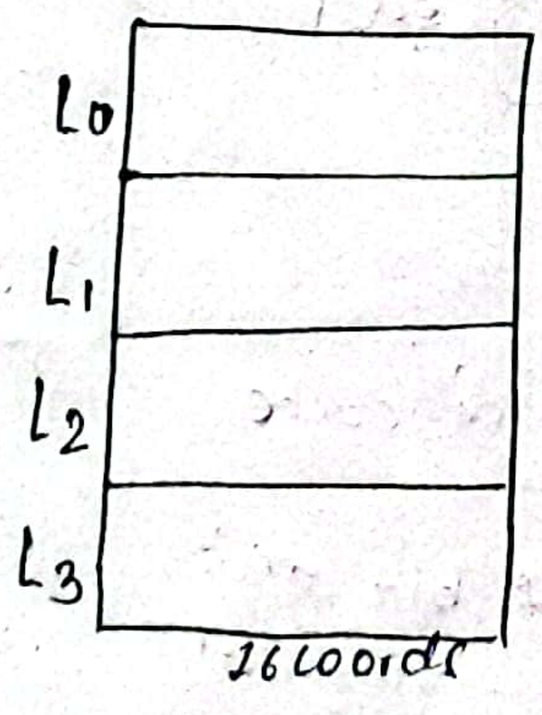
Cache Mapping

There are three different types of mapping used for the purpose of cache memory which are as follows:

[main memory को Block, cache memory को कुन line मा जाएर बस्छ, cache mapping]

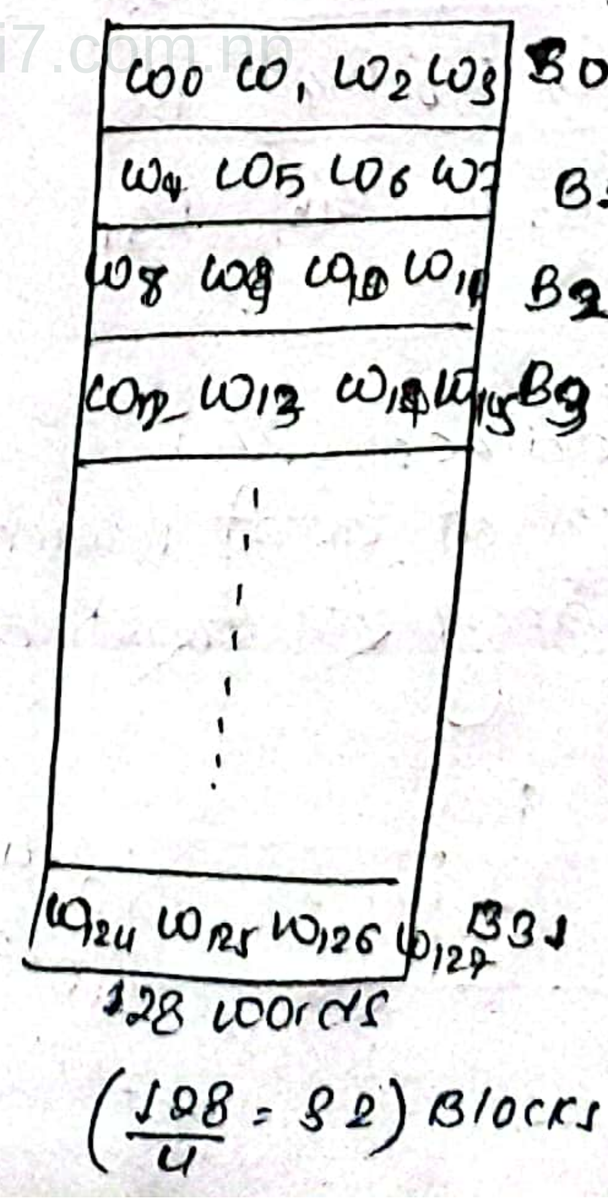
i. Direct mapping
ii. Associative mapping
iii. set-Associative mapping.

1. Direct Mapping



$\frac{16}{4} = 4$ lines

L = line (4) (0-3)
B = Block (32) (0-31)
w = word सुरु Byte भर्छ हो।
ie. 1 word = 1 byte



word = addressable unit of data.

$\left(\frac{128}{4} = 32\right)$ Blocks

notes.arpanadhikari7.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

↳ Remember;

Line size = Block size

↳ एउटा Block मा 4 words हुन्छ

↳ एउटा Line मा 4 Blocks हुन्छ

formula for Direct mapping;

K mod n

↓ ↓

Block No line no.

eg: Block no 0 and,
no. of line = 4

∴ 0 mod 4 = 0 (पहिले Block 0 लाई जनाउँछ।) (B_0) पहिलो line no. 0 मा आउँछ।
1 mod 4 = 1 (ie. B_1) ← Block no. 1 पहिले line no. 1 मा आउँछ
2 mod 4 = 2 (ie. B_2) ← Block no. 2 पहिले line no. 2 (l_2) मा आउँछ।
3 mod 4 = 3 (ie. B_3)) ← Block no. 3 पहिले line no. 3 (l_3) मा आउँछ।
4 mod 4 = 0 (ie. B_4) ← Block no. 4 पहिले line no. 0 मा आउँछ।

Remender
हो।

L_0	B_0 B_4 B_8 B_{12}	-- B_{28}
L_1	B_1 B_5 B_9 B_{13}	-- B_{29}
L_2	B_2 B_6 B_{10} B_{14}	-- B_{30}
L_3	B_3 B_7 B_{11} B_{15}	-- B_{31}

यसरी mapping हुन्छ।

→ एउटा time मा कुनै एउटा Block मात्र mapping हुन्छ।

#. Physical address (PA)

↳ CPU ले कहिले reference गर्छ त अथवा CPU ले cache memory सँग कहिले deal गर्छ त?

0-127 = 2^7 = 7 bit लामो physical address चाहिन्छ जुन 2 part मा divide हुन्छ।

PA

5	2

2^5=32 Block size — Block no. — Block offset (size) — 2^2=4 (0-3)

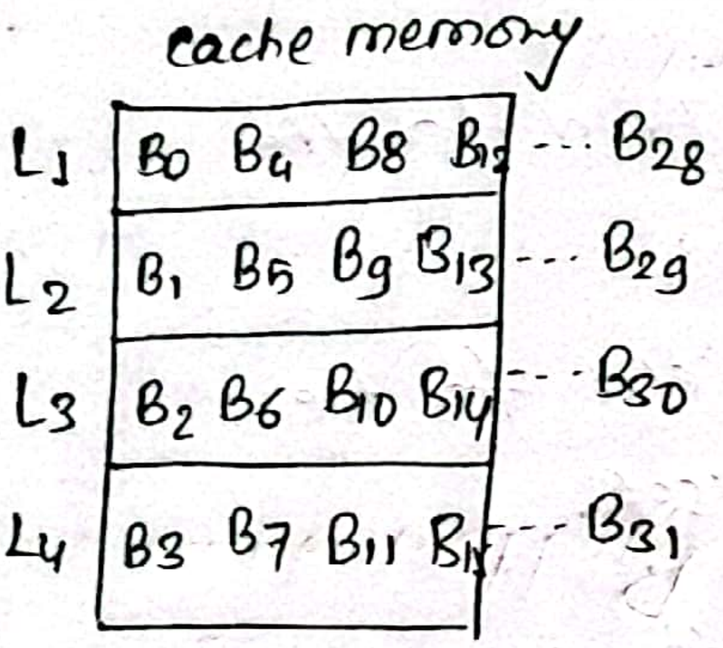
Available on notes.arpanadhikari7.com.np

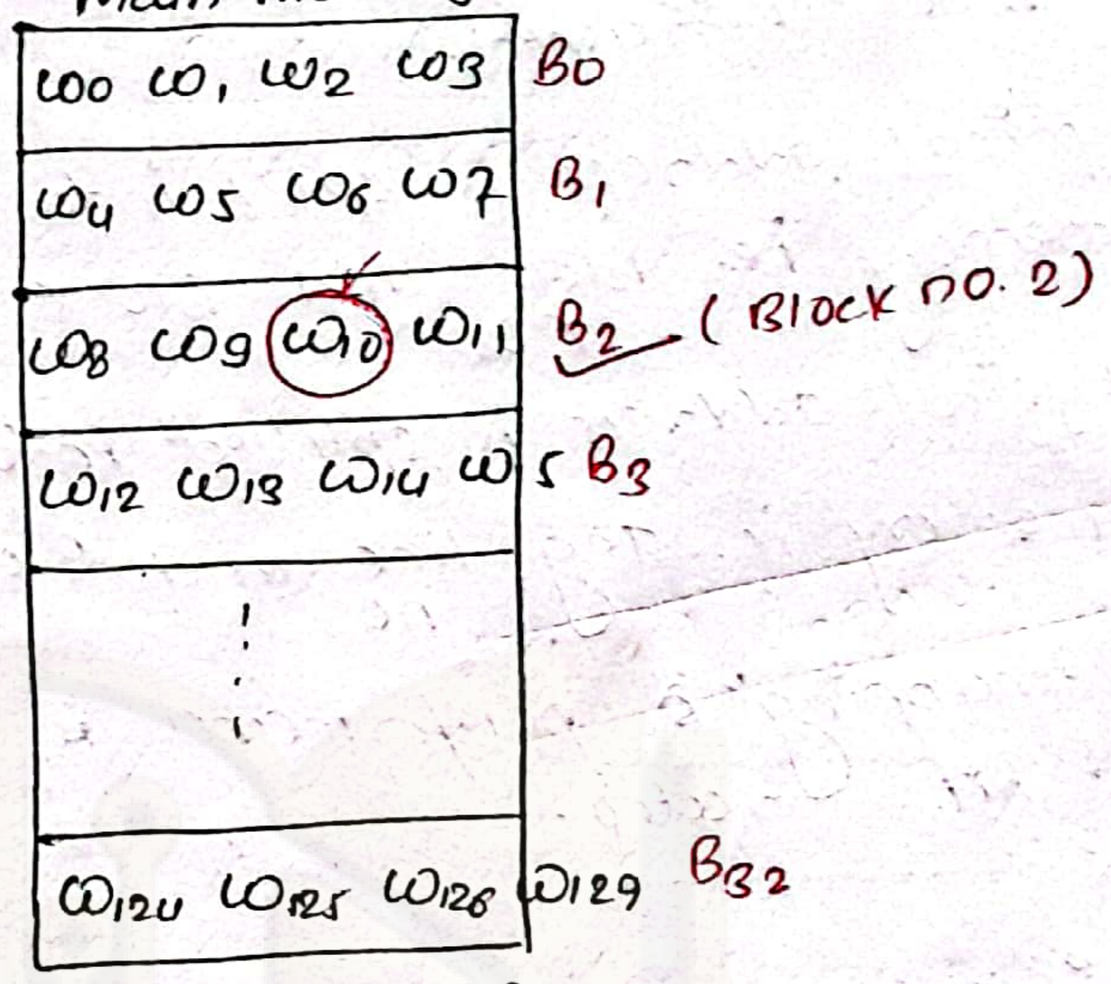
eg: यदि हामी सँग एउटा address छ (7 bit को) जुन CPU ले generate गरेको छ।

ie. 0001010
   └──┘└┘

Here,
Block no = 00010 = 2 (ie. Block no. 2)
Block off = 10 = 2 (2nd word) (0, 1, 2, 3)

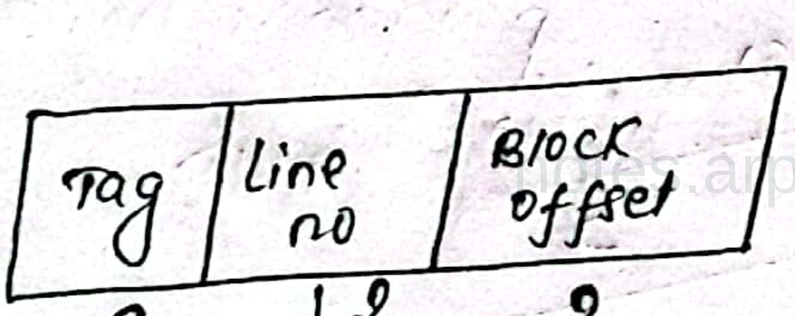
ie.





यसरी main memory मा CPU ले word खोज्छ।

अब,
cache memory मा यादि कसरी काम हुन्छ?
Soln physical address change हुन्छ।



[यहाँ, Block no change भएर tag र line no मा divide हुन्छ।]

↳ एउटा Block मा कति वटा word छ ($2^2=4$)
यो चार वटा line छ त्यसैले 2, ($2^2=4$)

→ अस्तो कुन line मा कुन यादि Block आउँछ त्यो कुरा tag ले भन्छ।

(eg: line no 00 मा B0 आउँछ की B4, B8, B12 कुन आउँछ त्यो कुरा tag ले गर्छ।)

ie,
tag bit = 001
line no = 00 (for L0)
Block offset = 00, 01, 10

∴ 1st combination = 0010000 = 32th word
2nd combination = 0010001 = 33th word
3rd combination = 0010010 = 34th word
4th combination = 0010011 = 35th word

Available on notes.arpanadhikari7.com.np

- The simplest technique, known as direct mapping, maps each block of main memory into only one possible cache line or,
- In direct mapping, assign each memory block to a specific line in the cache. If a line is previously taken up by a memory block when a new block needs to be loaded, the old block is trashed.
- The physical address is split into two field for the main memory ie. Block no and Block offset.
- The physical address is splitted into 3 field for the cache memory ie. Tag, line no and Block offset.
- Direct mapping's performance is directly proportional to the hit rate.

Advantages:

1) mapping technique is simple and inexpensive to implement
2) fast performance

Disadvantages:

1) fixed cache location for any given block
2) The hit ratio will be low.

(यदि CPU ले Line no 1 मा Bo खोज्यो भने भेराउँदैन, त्यतिबेला miss भयो, hit भएन।)

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2) Associative Mapping (fully):

note:

↳ Line = Block size = size.

↳ main difference, यहाँ जुन Block हुन्छ उनसुकै line मा पनि map हुन सक्छ। तर direct mapping मा Block हुन line अनुसार fix थिस

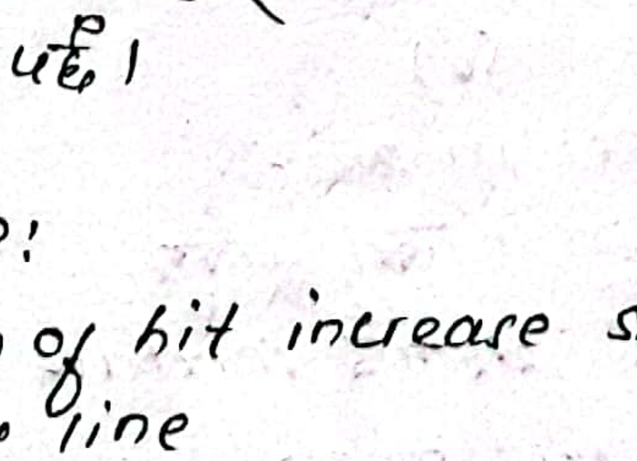
B_0 B_1 ---B_{31}	L_0
B_0 B_1 ---B_{31}	L_1
B_0 B_2 ---B_{31}	L_2
B_0 B_3 --B_{31}	L_3

↳ यही भएर hit ratio increase हुन्छ।

↳ main memory र cache memory को Block बनाउने अगाडी भएँ,

↳ यस्मा Block हुन जुन line मा पनि जान सक्ने भएर line no हुँदैन

∴ P(A) for cache memory



Block no = tag हुन्छ.
fully associative

↳ tag हेर्दै हुनको ~~discriminator~~ कारण एउटै line मा B_0 देखी B_{31} सम्मको जुनसुकै Block हुन सक्छ।

[यस्को मतलब B_0 देखी B_{31} सम्म सबै एउटै time मा हुन्छ अन्ने होइन। एउटा time मा केवल एउटा Block]

↳ tag हेर्दै जाँदा भएकाले comparision धेरै गर्नु पर्छ।

Advantage:

→ No of bit increase size, any block can come at any line

Disadvantage:

tag bit = 5

so, No of comparision increases.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

3. SET ASSOCIATIVE MAPPING.

- combination of direct mapping and fully associative mapping.
- It eliminates the drawback of direct mapping.
- also known as "K-way set associative mapping".

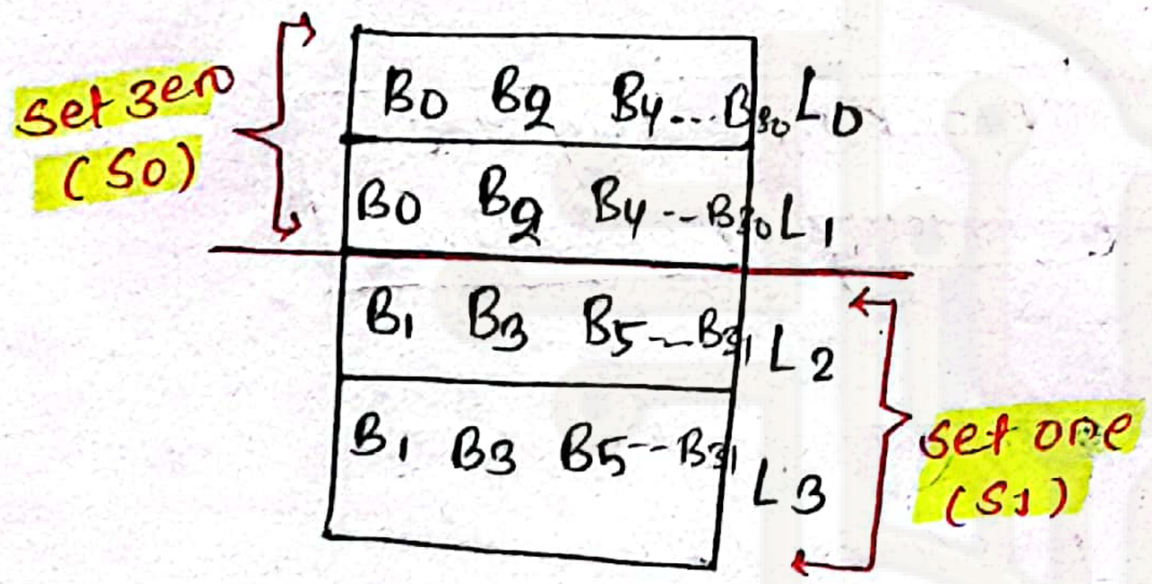
eg: 2-way set associative
4-way set associative

where,

$$\text{no of set} = \frac{\text{No. of Lines}}{K}$$

for, 2-way set associative, $K = 2$

$$\therefore \text{no. of set} = \frac{4}{2} = 2 \quad \begin{pmatrix} \text{set no zero - S0} \\ \text{set no one - S1} \end{pmatrix}$$



formula: K mod n (no of set)

Block no

~~Direct mapping~~

(eg: 0 mod 2 = 0 (set no zero, S0)
यस्को अर्थ B0 block S0 set मा आउँछ,
set S0, मा पुग्न सुरुको line मा आउन सक्छ.
ie. L0 or L1, त्यही भएर, यस्लाई mixture of associative and direct mapping भनिन्छ।)

notes.arpanadhikari7.com.np

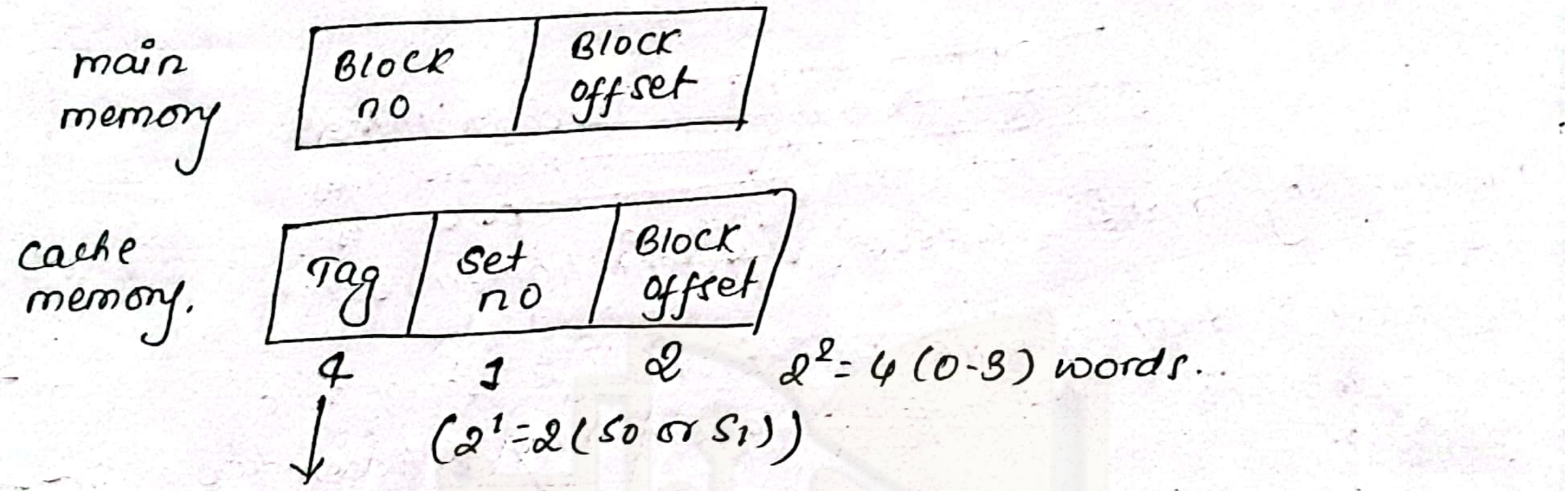
CamScanner

Available on notes.arpanadhikari7.com.np

→ It does this by saying that instead of having exactly one line that a block can map to in the cache, we will group a few lines together creating set.

→ Then a block in memory can map to any one of the lines of a specific set.

→ set associative cache mapping combines the best of direct and associative cache mapping techniques.



Advantages:

→ combines advantages of direct and associative mapping techniques.

notes.arpanadhikari7.com.np

Disadvantages:

→ Increase the size of the set doesnot always improve the hit ratio.

CamScanner

Available on notes.arpanadhikari7.com.np

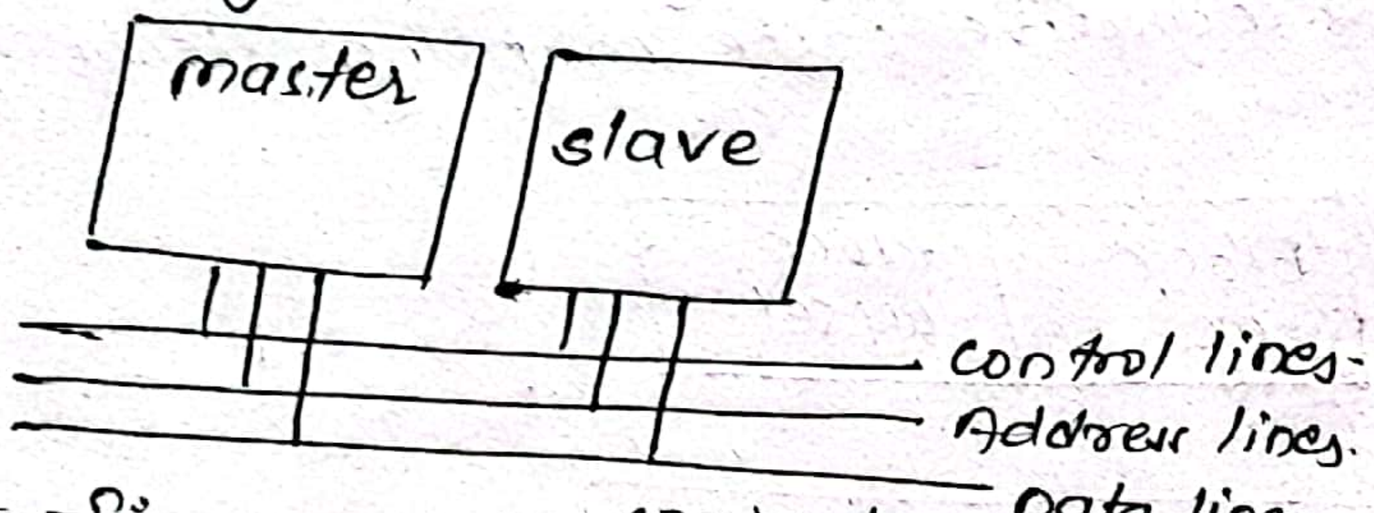
chapter: 04

Interfacing (processor र peripheral को बिचमा intermediate को काम गर्ने)

① Interfacing is a way to communicate and transfer information in either way without ending into deadlocks.

Interfacing is a way of effective communication in real time.
Interfacing involves 3 major points:

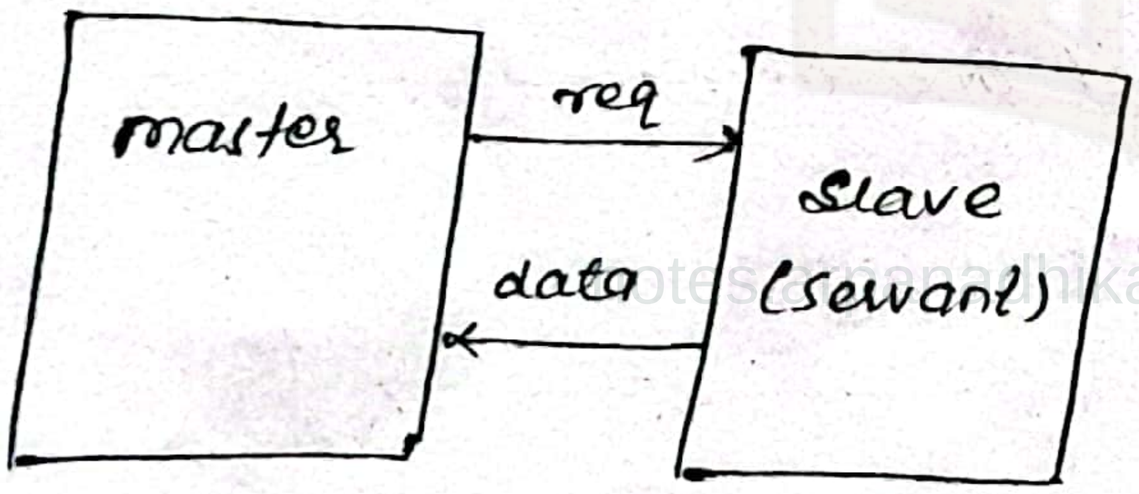
i) Addressing
ii) Arbitration
iii) protocol.



↳ Addressing माथाडि master ले specified different line मा data पठाउँछ

↳ protocol is a set of rule. Here protocol is set of rule describing how to transfer data, especially between two devices.

↳ Arbitration: process of generating the logic to select the bus connection over different connected peripherals.



(eg: interfacing between master and slave)

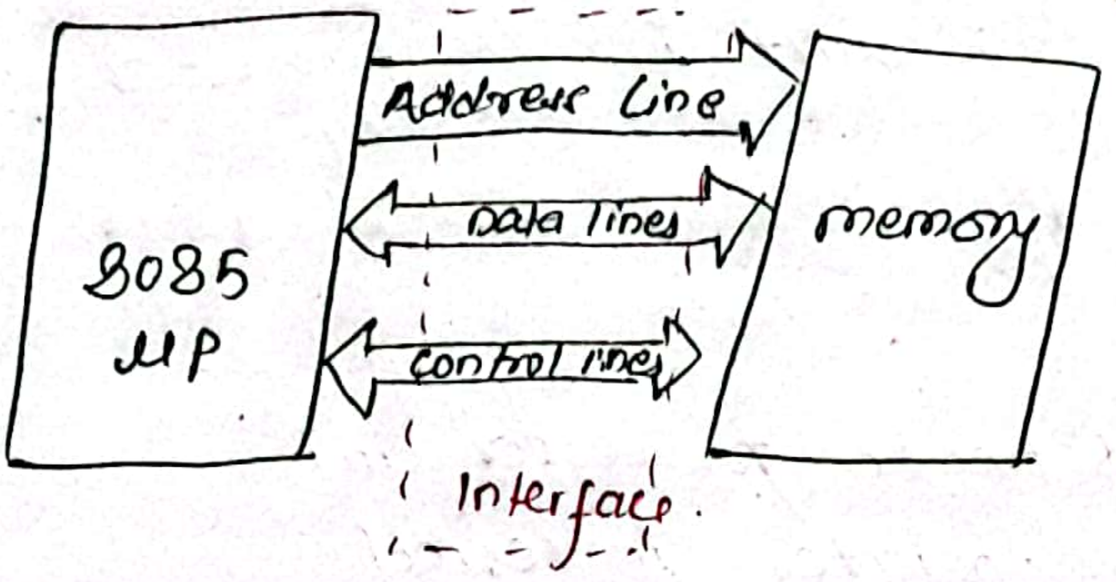
1. master asserts req. to receive data
2. servant puts data on bus within time taken.
3. master receives data and deasserts req.
4. servant ready for next request.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Processor and memory Interfacing:



Interface किन use गरिन्छ?

→ CPU र peripheral को बिचमा speed increase गर्न

→ peripheral बाट data serial form मा आउँछ तर interface ले यसलाई parallel मा convert गरिदिन्छ, speed fast हुन्छ।

→ Interface मा register हुन्छन्।

→ signal conversion गर्न (electromechanical to electrical signal)

4.2. Microprocessor Interfacing.

a) I/O Addressing.

A microprocessor communicates with other devices using some of its pins.

i) Port-Based I/O (Parallel I/O):

→ processor has one or more N-bit ports.

→ processor's software reads and writes a port just like a register.

ii) Bus-Based I/O:

→ processor has address, data and control ports that form a single bus.

→ communication protocol is built into the processor.

→ A single instruction carries out the read or write protocol on the bus.

notes.arpanadhikari7.com.np

2014 Fall
4(a)

Port-Based I/O	Bus Based I/O
1. The i/o peripherals are connected to the general i/o port of processor.	1. In Bus Based I/O, the i/o peripherals are connected via address bus. The address bus then put data into processor.
2. processor has one or more N bit ports.	2. processor has address, data and control ports that form a single bus.
3. Processor's software reads and write ports just like a registe	3. A single instruction carries the read or write protocol on the bu

CamScanner

Available on notes.arpanadhikari7.com.np

4. Communication protocol is not built into the processor.

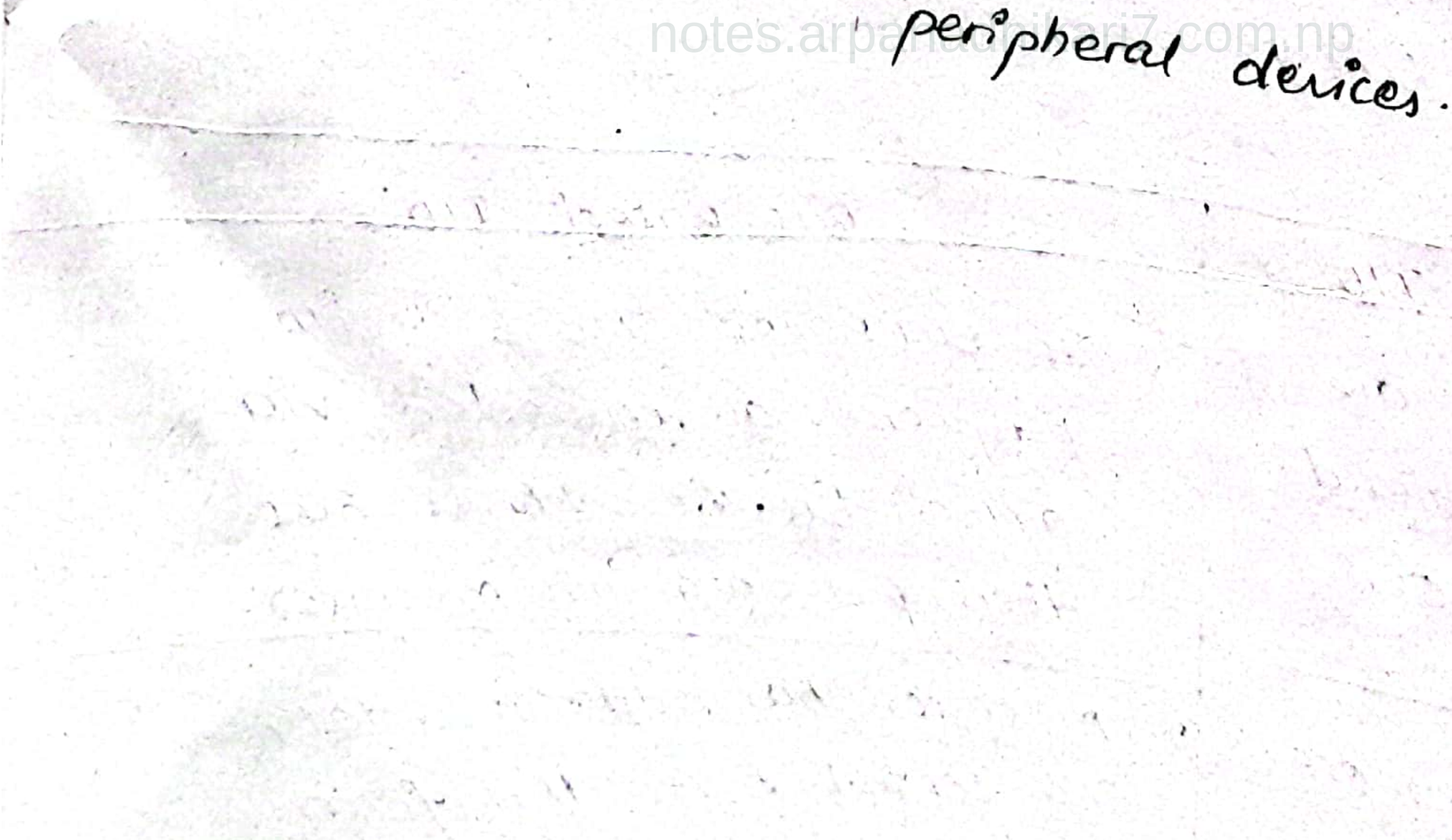
4. communication protocol is built into the processor.

(2) Q. Why is interface required?

ans:

1) for speed synchronization between processor and peripherals since peripherals operates in serial communication mode whereas processor operate in parallel mode.

2) To manage the data format since the peripheral operates in bit or byte format.



notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Types of Bus-Based I/O:

1. memory-mapped I/O
2. standard I/O (I/O mapped I/O)

memory mapped I/O

1. memory mapped I/O uses the some address space for both memory and I/O devices.
2. only one set of read and write instruction lines.
3. No separate instruction like IN, OUT and MOV. are used.
4. Device address
   - 16 bit for 8085
   - 20 bit for 8086
5. $\overline{MEMRD}$, $\overline{MEMWR}$ control signals are used.
6. Data transfer in betn any register and I/O device.

I/O mapped I/O.

1. I/O mapped I/O uses two separate address spaces for memory and I/O devices.
2. I/O read and I/O write lines for I/O transfer. memory read and memory write lines for memory transfer.
3. IN and OUT instruction for I/O transfer and MOV instruction deals with memory transfer.
4. Device address
   - 8 bit for 8085
   - 16 bit for 8086.
5. $\overline{IORD}$, $\overline{IOWR}$ control signals are used.
6. Data transfer in betn accumulator and I/O device.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Interrupts:

An interrupt is a signal to the processor emitted by hardware or software indicating an event that needs immediate attention.

Whenever an interrupt occurs, the controller completes the execution of the current instruction and starts the execution of an interrupt service Routine (ISR) or interrupt handler.

ISR tells the processor or controller what to do when the interrupt occurs.

Interrupts can be broadly classified as,

- → Hardware Interrupt → Maskable Interrupt
  → Non-maskable Interrupt
- → Software Interrupt → Normal Interrupt
  → Exception Interrupt

is Hardware Interrupt: (TRAP, RST 7.5, RST 6.5, RST 5.5, INTR in 8085)

A Hardware interrupt is an electronic alerting signal sent to the processor from an external device, like a disk controller or an external peripheral.

for example, when we press a key on the keyboard or move the mouse, they trigger hardware interrupts which cause the processor to read the keystroke or mouse position.

is Software Interrupt: (RST 0 to RST 7 in 8085 pin)

A software interrupt is caused either by an exceptional condition or a special instruction in the instruction set which causes an interrupt when it executed by the processor.

for example, if the processor's arithmetic logic unit runs a

Available on notes.arpanadhikari7.com.np

command to divide a number by zero, to cause a divide-by-zero exception, thus causing the computer to abandon the calculation or display an error message.

Depending upon service the interrupts also can be classified as.

1. fixed Interrupt (Non vectored Interrupt)
2. vectored Interrupt

→ fixed interrupt मा यदि interrupt occur भयो भने fixed address allocate गरेको हुन्छ, त्यहाँ जाने र त्यो address मा भएको program execute गर्ने } fixed Interrupt

→ Address of the ISR built into µp cannot be changed.

# Vectored Interrupt	Non-vectored Interrupt
1. The source that interrupts the CPU provides the branch information. This information is called the interrupt vector.	1. In non-vectored interrupt, the branch address is assigned to fixed address to the fixed address
2. It has a memory address.	2. It does not have a memory address.
3. It allows the CPU to be able to know what ISR to carry out in software.	3. When a non-vectored interrupt is received it jumps into the program counter to fixed address in hardware.
4. eg: RST 7.5, RST 6.5, RST 5.5, INTR.	4. eg: TRAP.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

maskable and Non-maskable interupts.

a. maskable:

→ ignore गर्न सक्ने processor को यस्तो interupt हो
→ programmer can set bit that causes processor to ignore interupt.
→ This is important when the processor is executing a time-critical code.
→ It has lower priority → They are vectored or non vectored.
eg: ~~TRAP~~ RST 7.5, RST 6.5 RST 5.5 INTR.

b. Non-maskable:

→ It cannot be masked off or made pending.
→ It is used for emergency purpose like power failure, smoke detector etc.
→ It has higher priority
→ They are vectored
→ Response time is low.
eg: TRAP.

notes.arpanadhikari7.com.np

CamScanner

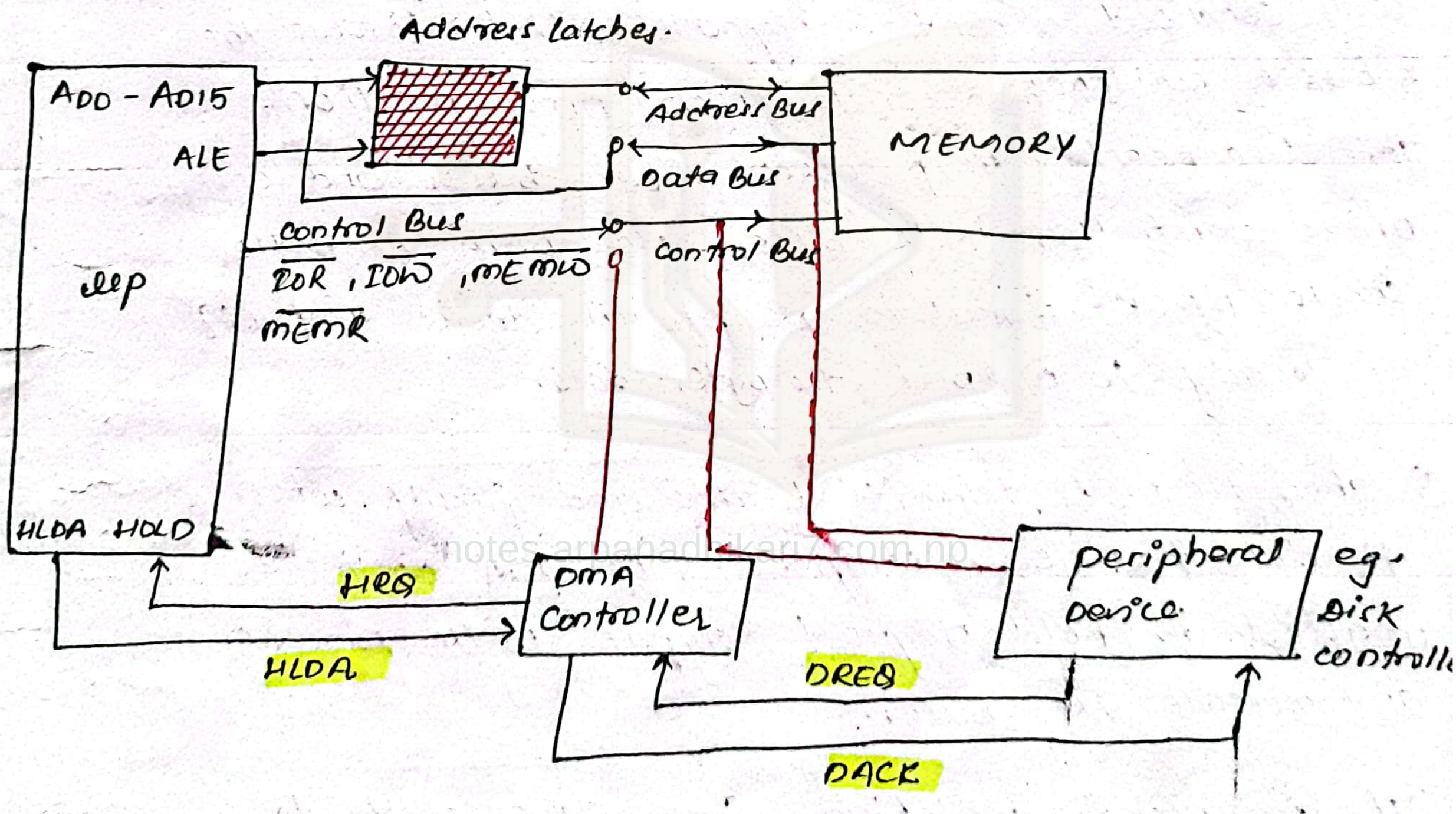
Available on notes.arpanadhikari7.com.np

DMA

Direct memory Access is a process of transferring data from one memory location to another without the direct involvement of the processor (CPU). The main benefit of using DMA is more efficient data movement in the embedded system.

DMA is IIO technique commonly used for high speed data transfer. for eg. data transfer betn system memory and a floppy disk.

In DMA, the MPU releases the control of the buses to a device called a DMA controller.



The sequence of dma transfer as shown in above block diagram can be explained below.

1. Originally microprocessor is connected to the memory as shown in fig with switches closed for address, data and control buses. when peripheral wants to transfer data

Available on notes.arpanadhikari7.com.np

using DMA transfer. It sends DMA request, DREQ signal to the DMA controller.

② If the i/p of the DMA controller is unmasked, the DMA controller will send a hold request. HRQ signal to the µp HOLD input.

③ the µp finishes the current machine cycle and floats its buses, sending out a hold-acknowledge signal, HLDA to the DMA controller.

④ When DMA controller receives HLDA signal, it will send out a control signal which throws the 8 buses switches down to their DMA position. This disconnects the processor from buses and connect DMA controller to the buses. Now DMA controller sends DACK signal to peripheral device to tell it get ready to output the bytes (data).

⑤ Then the DMA transfer begins and finally when the data transfer is complete, the DMA controller unasserts its hold request signal to the processor and release the buses.

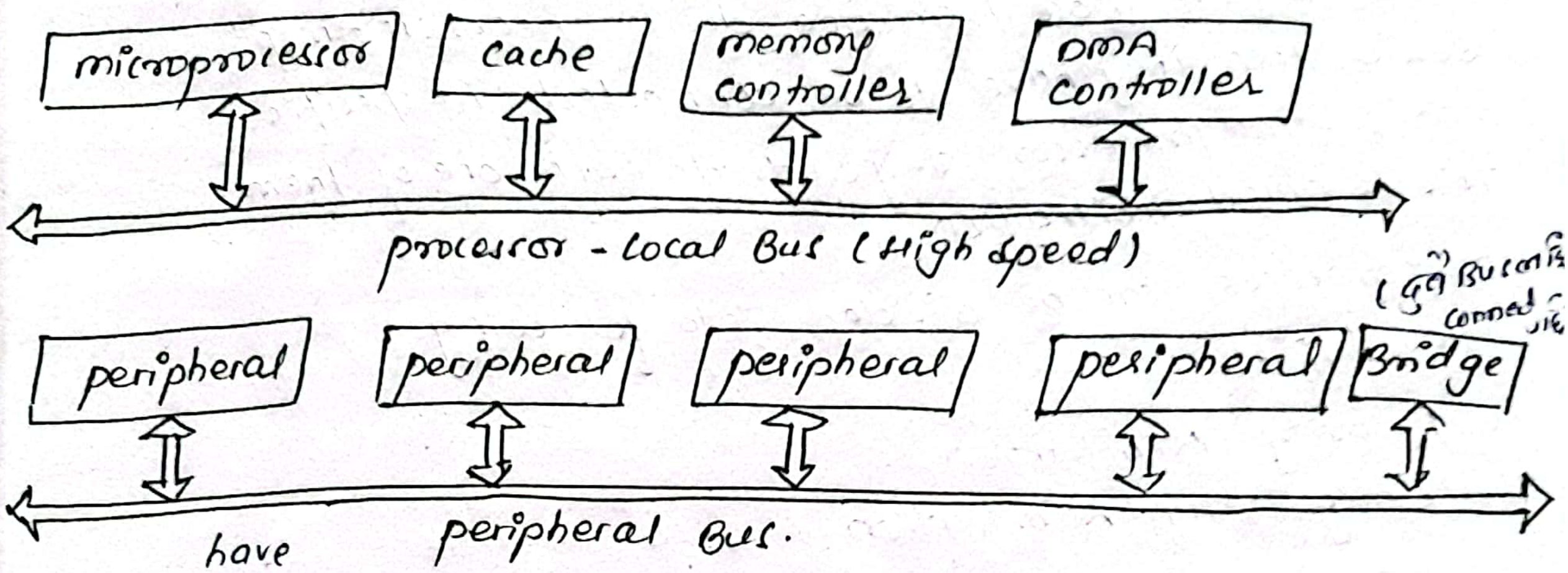
The process is switching from slave mode to master mode.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

MULTILEVEL BUS ARCHITECTURE



have peripheral Bus.

As we seen, a µp communicates with a number of devices and peripherals via various protocols. Each of these communicatio protocol requires the communication links to allocate data transfe at a particular rate.

→ microprocessor ले सबै peripheral सँग एउटै frequency मा interact गर्दछ। [यसै Bridge function's as single purpose processor that transfer data from one bus to another bus]

→ Interact with processor and memory is more frequent than Interact with processor and peripheral like UART.
(universal asynchronous receiver / transmitter)

→ सबै communication को लागी high speed single bus system बनाउन खोज्नु impractical हुनजान्छ।

→ सबै peripheral ले high speed communication खोज्दैन।

→ धेरै speed communication. भए पनि peripheral को लागी high speed दिनु waste of power and speed हो।

→ एउटै Bus मा धेरै processor को load पर्दा आउने पार्ने slow हुनजान्छ यहि भएर multi-level Bus architecture चाहिन्छ।

High speed processor को लागी - local Bus
Low speed को लागी - peripheral Bus.

Characteristics of the peripheral bus:

→ low power consumption → generally slower than the
→ portability local Bus.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

ARBITRATION (Interrupt Handling process)

↳ when there are more than one device need interrupt service, then they have to be connected in specific manner. the processor responds to each one of them. This is called arbitration.

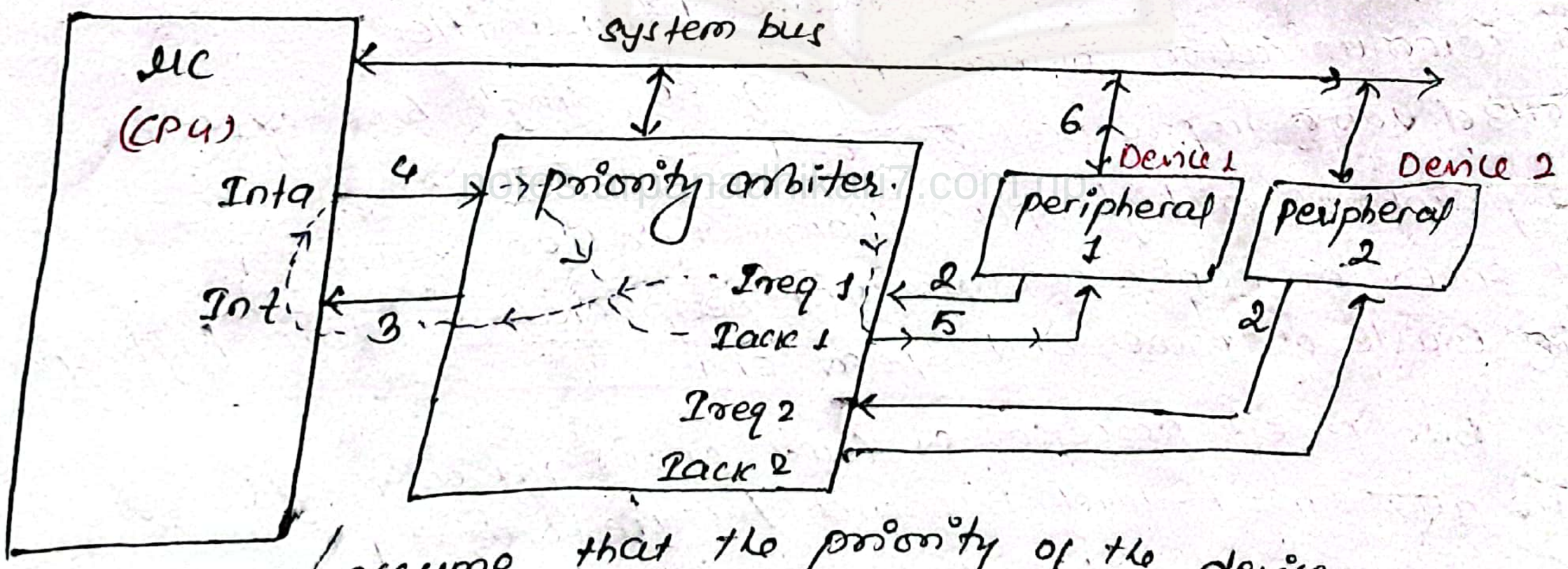
↳ The processor can be divided into following two types:

is priority Arbitration
ii's Daisy chain Arbitration.

as Priority Arbitration:

peripheral devices are connected with the processor and a priority arbiter is attached with processor selects the connection of peripheral with bus on the fixed priority basis or round robin approach.

If arbiter selects the PDs in fixed priority basis then such arbiter is called fixed priority arbiter.



let us assume that the priority of the devices are Device 1 > Device 2 ---- then priority arbiter works as:

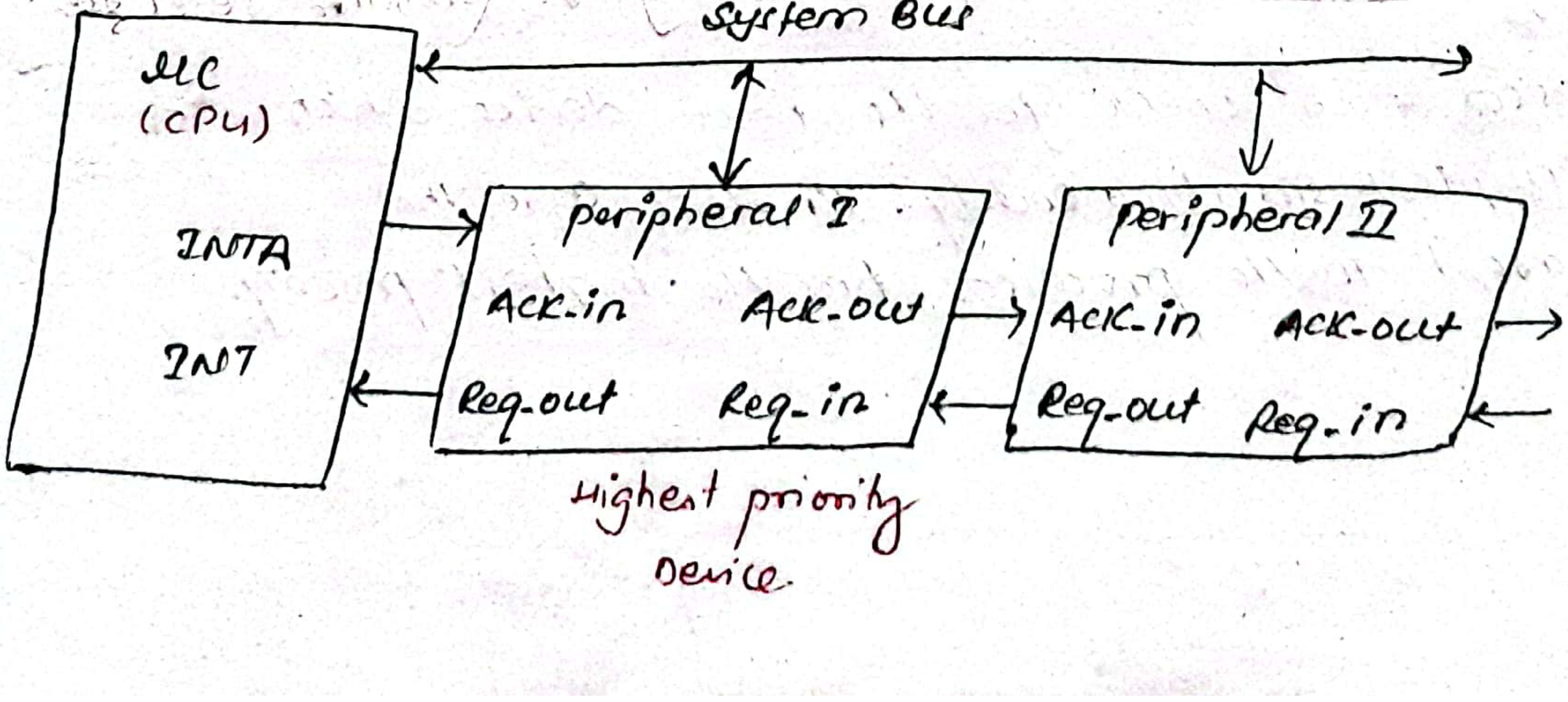
→ processor is executing its program.

Available on notes.arpanadhikari7.com.np

→ ~~processor~~
→ peripheral 1 needs servicing. so asserts Ireq 1.
peripheral 2 also needs servicing so asserts Ireq 2.
→ priority arbiter sees at least on Ireq input asserts so asserts Int.
→ Processor stops executing its program and stores its state.
→ processor asserts Inta.
→ priority arbiter asserts Iack 1 to acknowledge peripheral 1.
→ peripheral 1 puts its interrupt address vector on the system bus.
→ processor jumps to the address of ISR read from data bus. ISR executes and returns (and complete handshake with arbiter).

b. Daisy chain Arbiter:

In this case the peripherals needing service are connected in chain as shown in fig. The requests are chained and hence any device interrupting shall be transmitted to the CPU in a chain.



Available on notes.arpanadhikari7.com.np

Let us assume that the priority of the devices are Device 1 > Device 2 .. then daisy chain arbiter works as

is The processor is executing its program.

(i) Any peripheral needs services asserts Req. out. The request-out goes to the req-in of the subsequent device in the chain.

(ii) Thus the peripherals nearest to the µc assert Int..

(iii) The processor stops executing its program and stores its state

(iv) Processor asserts INTA the nearest device.

(v) The INTA passes through the chain till finds a flag which set by the device which has generated the interrupt.

(vi) The intercepting device sends the interrupt address vector to the processor for its interrupts service subroutine.

(vii) The processor jumps to the address of ISR read from data bus, ISR execute and returns.

(viii) The flag is reset. (Interrupt handling पछि reset हुन्छ)

The processor now check for the next device which has interrupted simultaneously. In this case the device nearest to the processor has the highest priority.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Chapter: 05 RTOS

→ Real time operating system.

→ A Real Time system is defined as a data processing system in which the time interval required to process and respond to input is so small that it controls the environment.

→ Real-time systems are used when there are rigid time requirements on the operation of a processor or the flow of data and real-time system. for eg: scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control system etc.

→ Basically there are 3 types of RTOS
i) Hard-real time operating system (critical task समयमै complete)
ii) soft-real time operating system (less restriction of time)
iii) firm-real time operating system (समयको त्यत्तो पालबन्दी हुँदैन, deadline समयभन्दा थोरै तल मात्रै हुन सक्छ।)

5.1. Process, Task and Thread.

5.1.1. Task:

A job is a unit of coork that is scheduled and executed by a system is knoron as task.

whether it's system task or an application task, at any time each task exists in one of a small number of states including ready, running and blocked.

As the real time embedded system runs, each task moves from one state to another, according to the logic of a simple finite state machine (FSM).

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

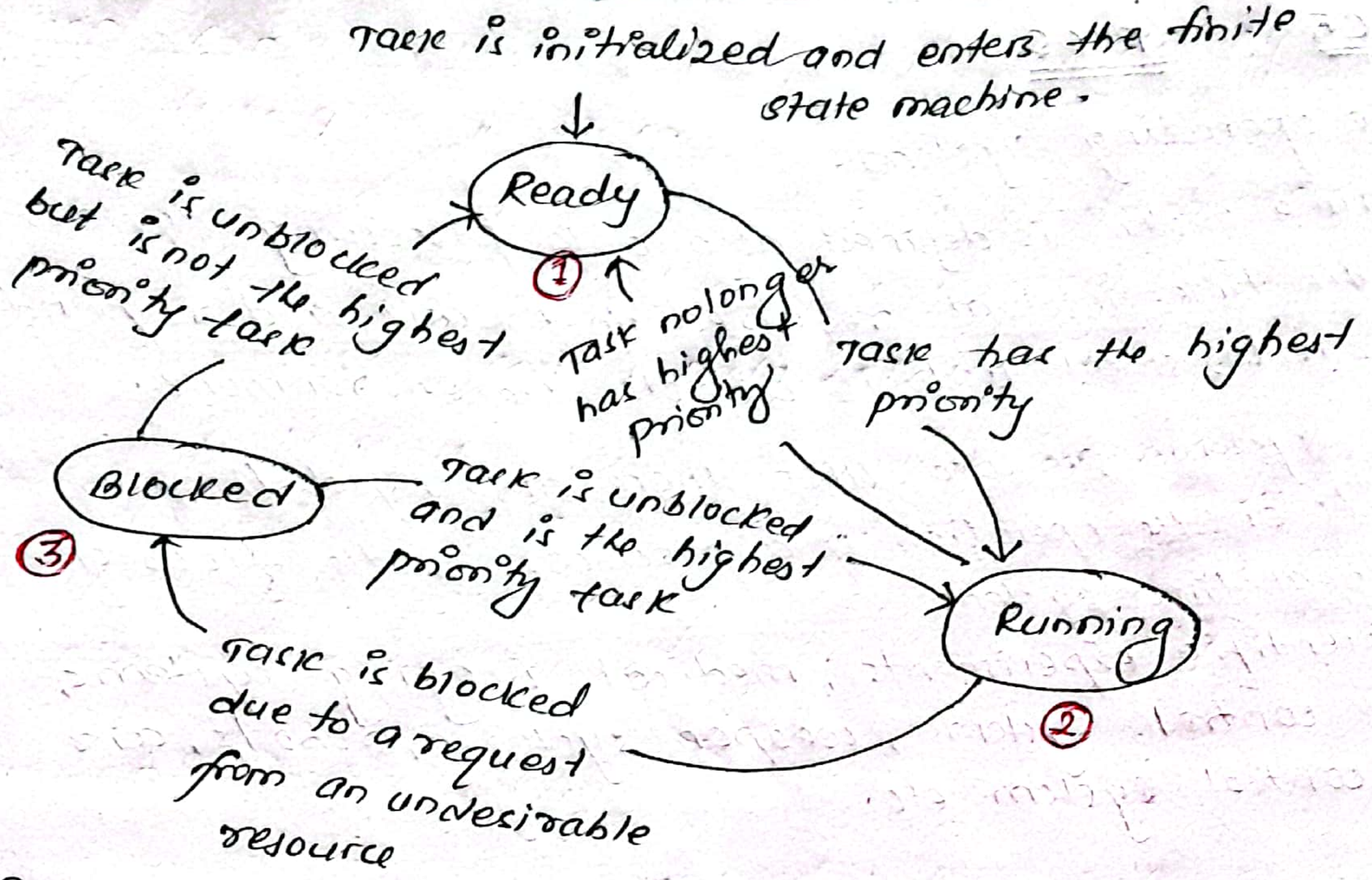


fig: A finite state machine (FSM) for task execution states.

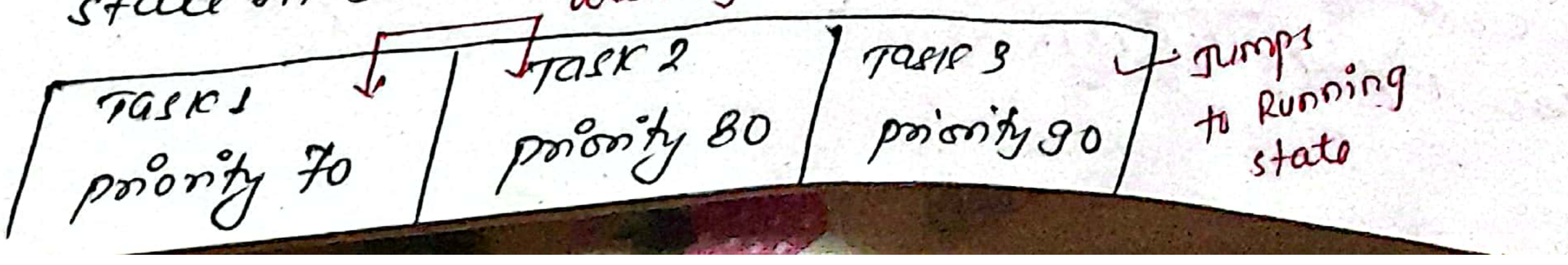
a) Ready state:

When a task is first created and made ready to run, the kernal puts it into the ready state.

→ A task is in a ready state when it is ready for execution (neither in the Blocked or suspended state)

→ But is not currently executing because of different task of equal or higher priority is already in the running state.

→ Ready state मा भएको सर्वभन्दा Highest priority task state change गरेर Running state मा जाछ।



notes.arpanadhikari7.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

b) Running state:

- When a task is actually executing it is said to be in the Running state.
- If the processor on which the RTOS is running, only ~~one~~ has a single core then there can only be one task in the running state at any given time.
- When the task moves to the running state, the processor loads its task content to the register.
- The running task can move to the blocked state in any of the following ways.
  - By making a call that requests an unavailable resources.
  - By making a call that requests to wait for an event to occur
  - By making a call to delay the task for some duration.

c) Blocking state:

- A task will go in a blocking state whenever it is waiting for an event to happen.
- The event can be completing a delay period or the availability of a resource. After the cause for the block is removed, the task is placed back in ready state.

- A task can only move to the blocked state by making a blocking call.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Imf Explain various types of task scheduling techniques in RTOS?

ans: Task is a simple subroutine set of program instructions that are loaded in memory.

where,

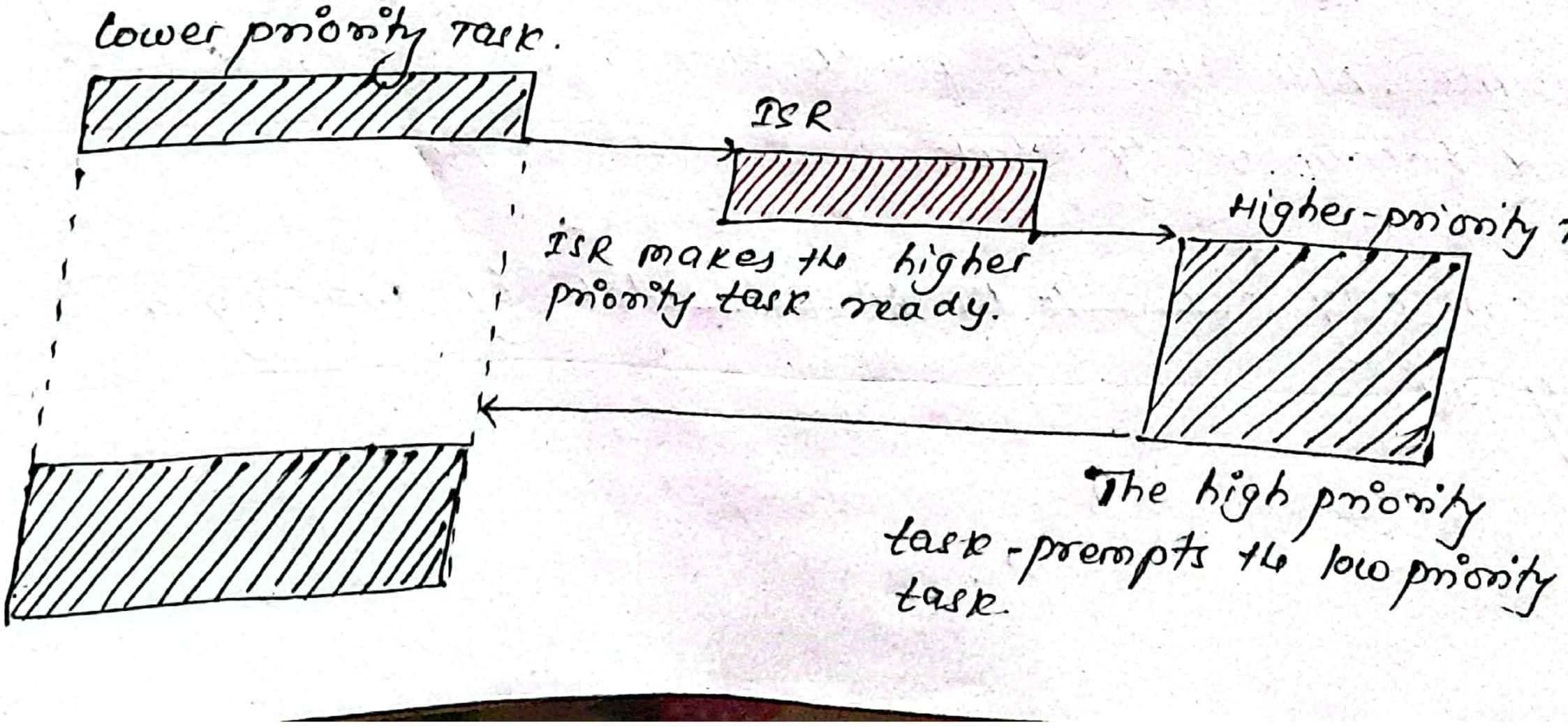
Task = code + data + state

Basically there are two types of task scheduling techniques in RTOS and they are:

i) is preemptive scheduling
ii) is Nonpreemtive scheduling.

1. Preemptive scheduling:

- preemptive scheduling is used when a processor switches from running state to ready state or from the waiting state to ready state.
- यदि processor ले एउटा lower priority task perform गरि रहेको बेला यदि ready state मा अर्को task with higher priority आयो भने processor ले गरिरहेको task लाई छोडेर higher priority task लाई स्थान दिन्छ, यस्तो किसिमको scheduling technique is preemptive scheduling हो।



Available on notes.arpanadhikari7.com.np

→ High priority task को कारणले, lower priority task बाट CPU को control खोसेर Higher priority task लाई दिने technique.

→ preemptive scheduling is flexible.

2. Non-Preemptive scheduling:

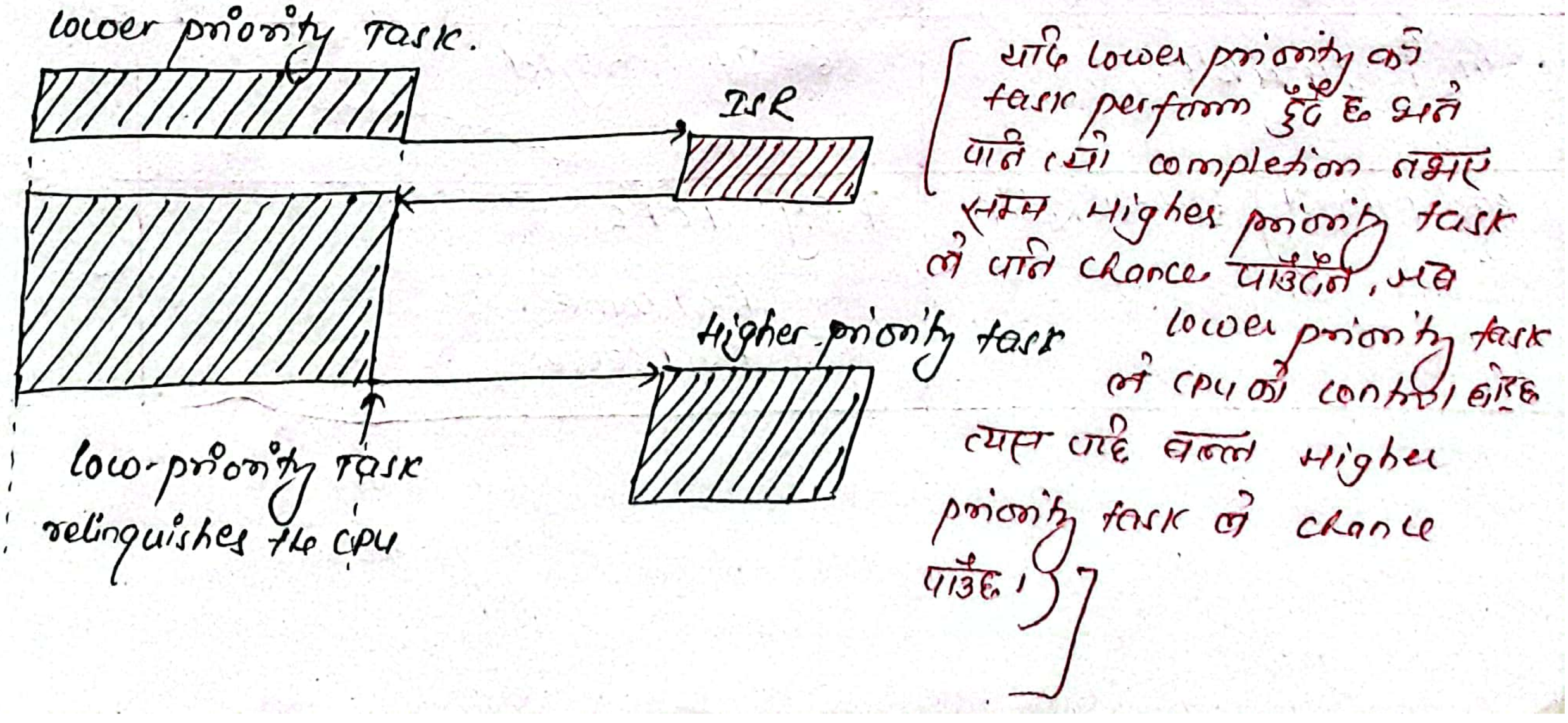
↳ Once resources are allocated to a process, the process holds it till complete its burst time or switches to waiting state.

↳ process cannot be interrupted till it terminates or switches to waiting state.

↳ If a process with long burst time is running CPU, than another process with less CPU burst time may starve.

↳ Non-preemptive scheduling is rigid.

[यदि CPU ले कुनै task perform गरिरहेको छ भने केही कुराले disturb गर्दैन सक्दैन, चाहे त्यो highest priority task लोस वा burst time एक उस्क resource allocation गरिरहेको छ भने आफ्नो task completion नभए सम्म अर्को task scheduling, processing गर्दैन]



[यदि lower priority को task perform हुँदै छ भने पनि त्यो completion नभए सम्म Higher priority task ले पनि chance पाउँदैन, यस lower priority task ले CPU को control छोड्ने त्यस पछि चल्न Higher priority task ले chance पाउँछ।]

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

S.N.	Pre-emptive scheduling.	S.N.	Non-Preemptive scheduling.
1.	The CPU allocate a process for a limited time.	1.	The CPU allocated to a process until it terminate or switches to waiting state.
2.	Process can be interrupted in between	2.	Process cannot be interrupted until it terminates or switches to waiting state.
3.	switching overhead required. [processor ले कियर्म काम छोरेर अर्को task को control मर्न सान्छ सत्नाई switching भनिन्छ, त्यस्तो समयमा state को लागेमा overhead हुन्छ।]	3.	No switching overhead required.
4.	used in time sharing system.	4.	Not suitable for time sharing system.
5.	CPU utilization is higher than non-preemptive scheduling.	5.	CPU utilization is lower than preemptive scheduling.
6.	It is flexible.	6.	It is rigid.
7.	eg: priority Round Robin	7.	eg: FCFS first come first serve.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

PROCESS

- A process is basically a program in execution. The execution of a process must progress in a sequence fashion.
- process is heavy weight task.
- process is defined as an entity which represents the basic unit of work to be implemented in the system.
- A process is an active program ie. a program that is under execution. It is more than the program code, as it includes the program counter, program stack, register, program code etc.
- process type:
  - i. clone process (child process)
  - ii. parent process (main process)
- clone process, also called a child process, is one which is created by another process, while.
- The main process is one that is responsible for creating other processes to perform multiple tasks at a time.
- various states of a process are shown below:

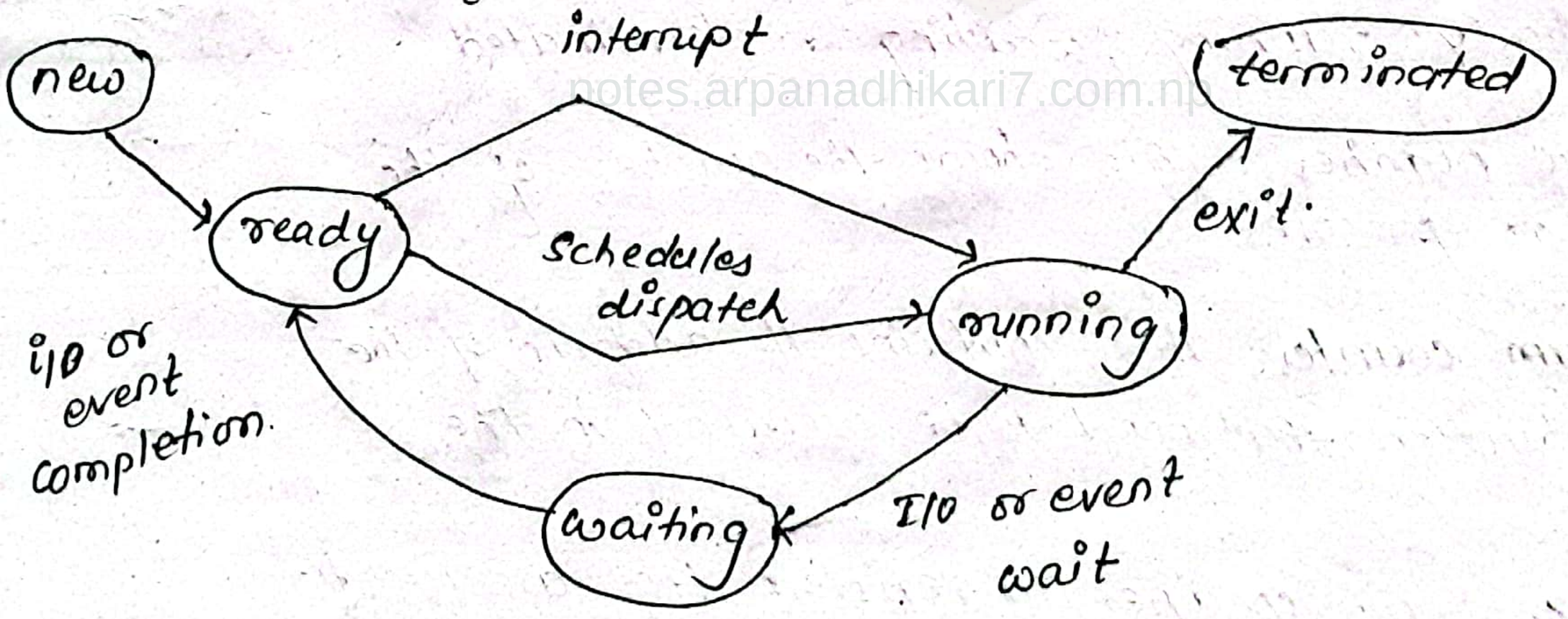


fig: process state diagram.

1) READY: The process is waiting to be assigned to a processor.

notes.arpanadhikari7.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

2) NEW: The process is being created
3) RUNNING: Instruction are being executed.
4) WAITING: The process is waiting for some event to occur.
5) TERMINATED: The process has finished. execution.

2014 fall
spring

PROCESS control BLOCK. (PCB)

1.	process state	(PS)
2.	process Number	(PN)
3.	program counter	(PC)
4.	Register	(R)
5.	memory list	(ML)
6.	List of open files.	

fig: Block diagram of PCB.

- **process state:** This specifies the process state i.e. new, ready, running, waiting or terminated.
- **process number:** This shows the number of the particular process.
- **program counter:** this contains the address of the next instruction that need to be executed in the process.
- **Register:** This specifies the registers that are used by the process. This may include accumulators, index register, stack pointer, general propose registers etc.
- **memory list:**
- **List of open files:**

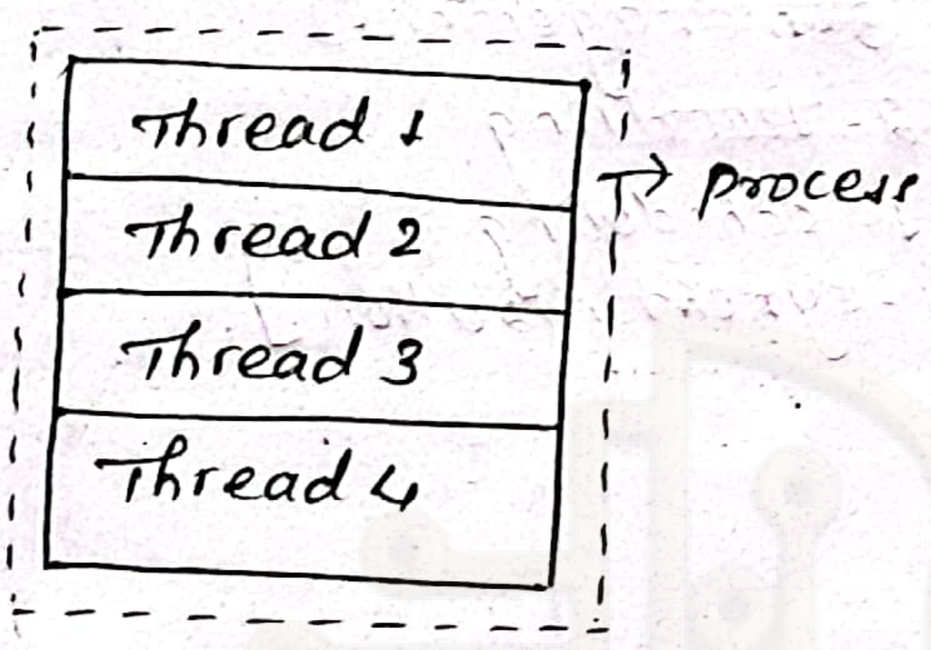
notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

THREAD

- A thread is basically a subpart of a large process. In a process, all the threads within it are interacted to each other. A typical thread contains some information like data segment, code segment etc.
- The most important feature of thread is that they share memory, data, resources, etc with their peer threads within a process to which they belong.



PROCESS	THREAD
1. An instance of a computer program that is being executed.	1. A component of a ~~program~~ process which is the smallest execution unit.
2. Heavy weighted.	2. Light weighted.
3. Switching requires interacting with the operating system.	3. Switching does not require interacting with the operating system.
4. Each has its own memory space.	4. Use the memory of the process they belong to.
5. Require more Resources.	5. Requires minimum resources.
6. Difficult to create a process.	6. Easier to create thread.
7. Inter-process communication is slow because each process has a different memory address.	7. Inter-thread communication is faster because they ~~belong to~~ share same memory address
8. In a multi-processing environment each process executes independently.	8. A thread can read, write or modify data of another thread

CamScanner
Available on notes.arpanadhikari7.com.np

5.2 The Real Time Kernel

A real time kernel is software that manages the time of microprocessor to ensure that time-critical events are processed as efficiently as possible.

The use of a kernel simplifies the design of embedded systems because it allows the system to be divided into multiple independent tasks.

Functions of RTOS Kernel:

1. Time management
2. Task management
3. Interrupt Handling
4. memory management
5. Exception Handling
6. Task synchronization
7. Task scheduling.

[3-management
2-Handling
1-scheduling
1-synchronization]

1. Time management:

- The kernel provides a part of the time to each process, when a process has finished execution the kernel starts another process, determines the state of the process, which could be one of running, waiting or ended.
- A high resolution hardware timer is programmed to interrupt the processor at fixed rate called as time interrupt. each time interrupt is called a system tick.
- Normally, the tick can vary in ms.
- eg: 1 tick = 1ms, system can run 50 days.
  1 tick = 20ms, system can run 1000 days
  1 tick = 50ms, system can run 2500 days.

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

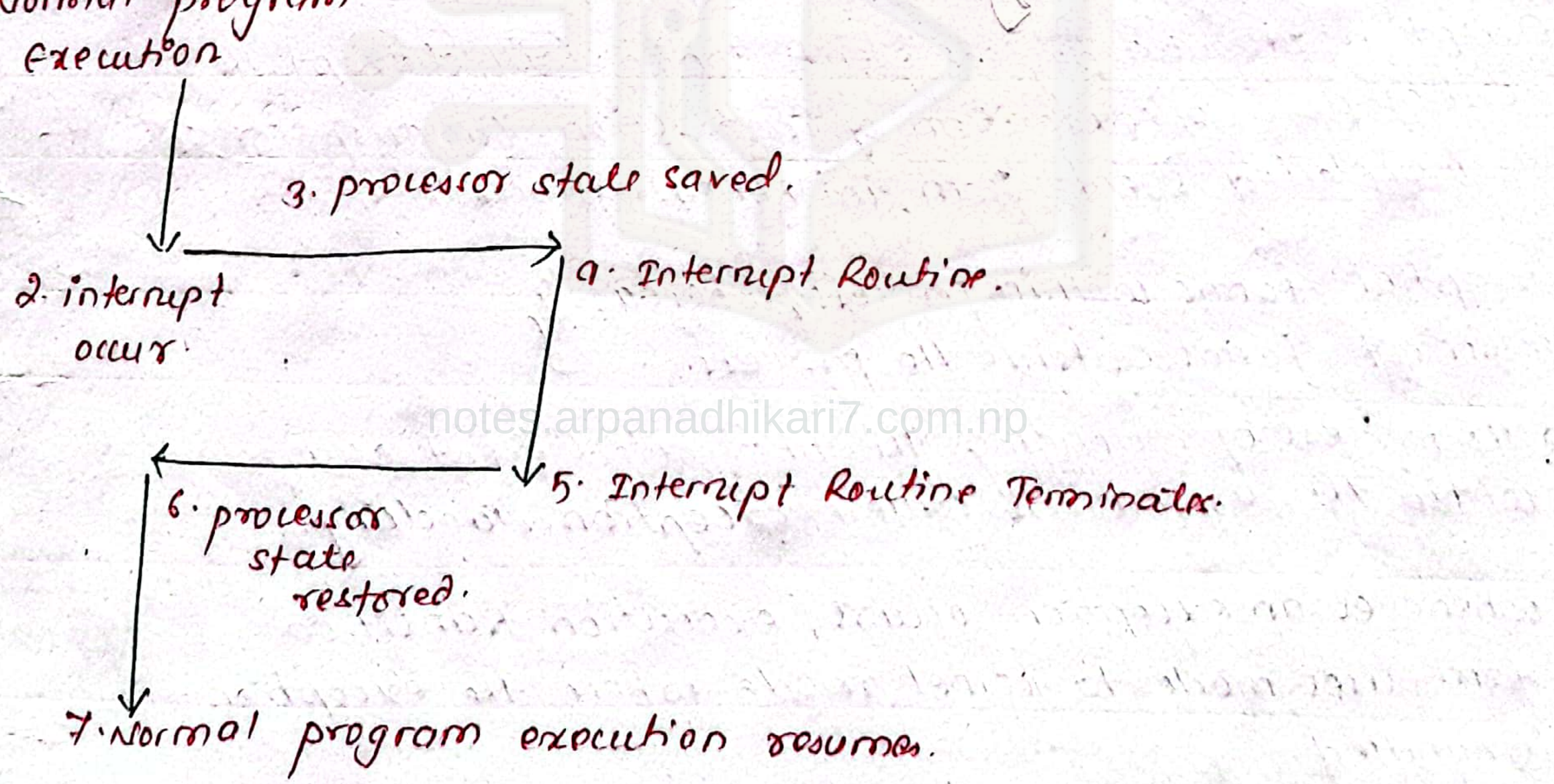
8. Task management.

The task management operation defines the following operations:

→ creation of new task with TCB (TASK control Blocks)
→ Task termination : remove the TCB
→ change priority : modify the TCB
→ state-inquiry : read the TCB.

9. Interrupt Handling.

→ An interrupt is a hardware mechanism used to inform the CPU that an asynchronous event has occured.
→ When an interrupt is recognised, the CPU saves all of its content (ie. register) and jumps to a special subroutine called an Interrupt service Routine (ISR).



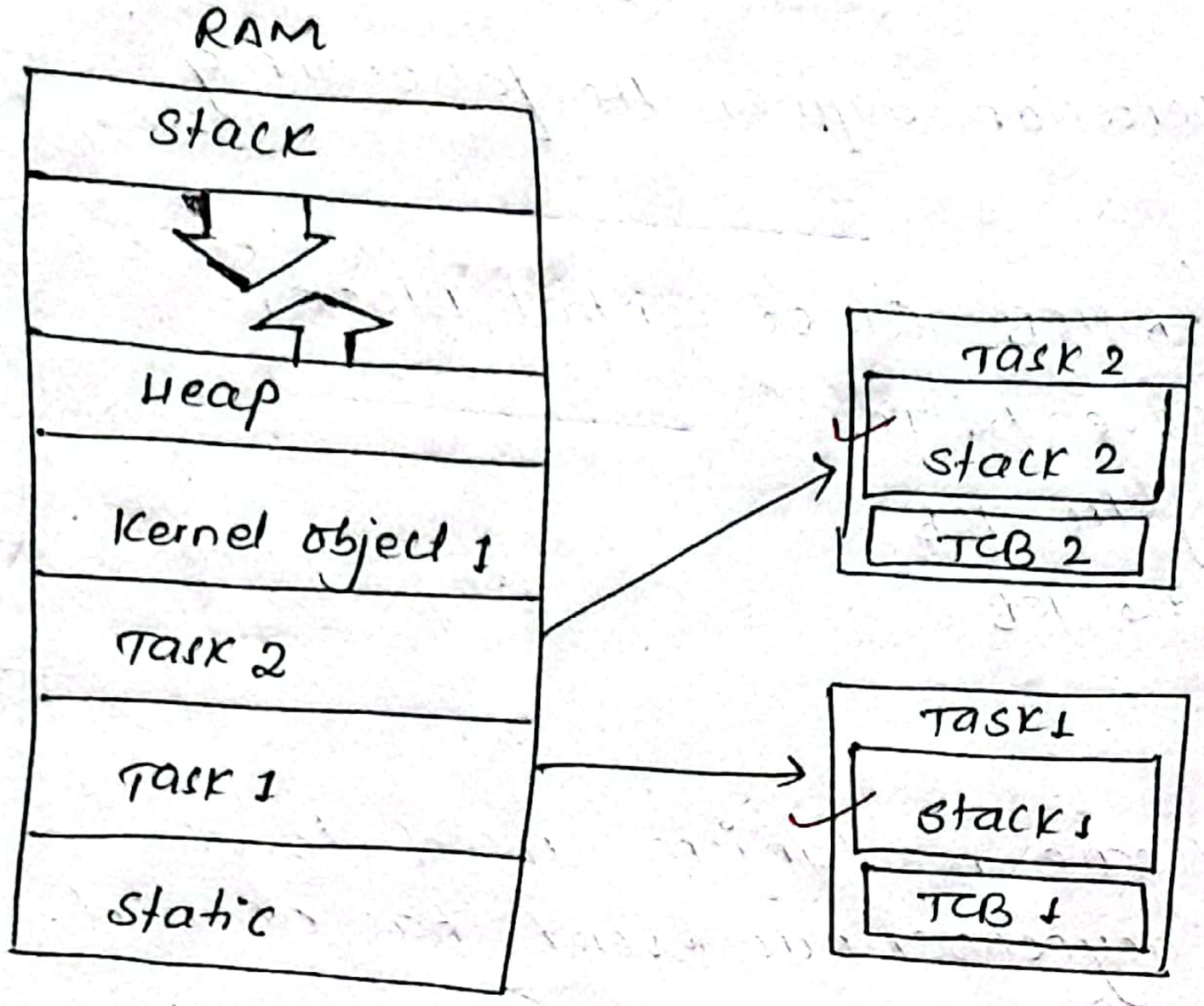
notes.arpanadhikari7.com.np

4. memory management

once the kernel creates and executes a process, it lives in memory having occupied space in it. when the process ends, the kernel removes the process from the memory. Kernel assigns the memory to process and releases it as well.

CamScanner

Available on notes.arpanadhikari7.com.np



(different task को लागी different memory allocation गरेको हुन्छ।)

→ Heap memory is used by all part of the application

→ stack memory is only used by one thread of execution.

fig: memory management in RTOS according to Task available.

5. Exception Handling.

Exceptions are unexpected events which will disrupt the normal flow of execution of instruction. An exception is an unexpected event from within the process.

{Exception forms within process, whereas interrupt forms outside the process.}

→ The process of managing the unexpected event occured within the process is called **exception Handling**.

whenever an exception occurs, execution transition from user mode to kernel mode where the exception is handled.

while entering the kernel, the context (values of all CPU registers) of the currently executing process must first be saved to memory. The kernel is now ready to handle the exception.

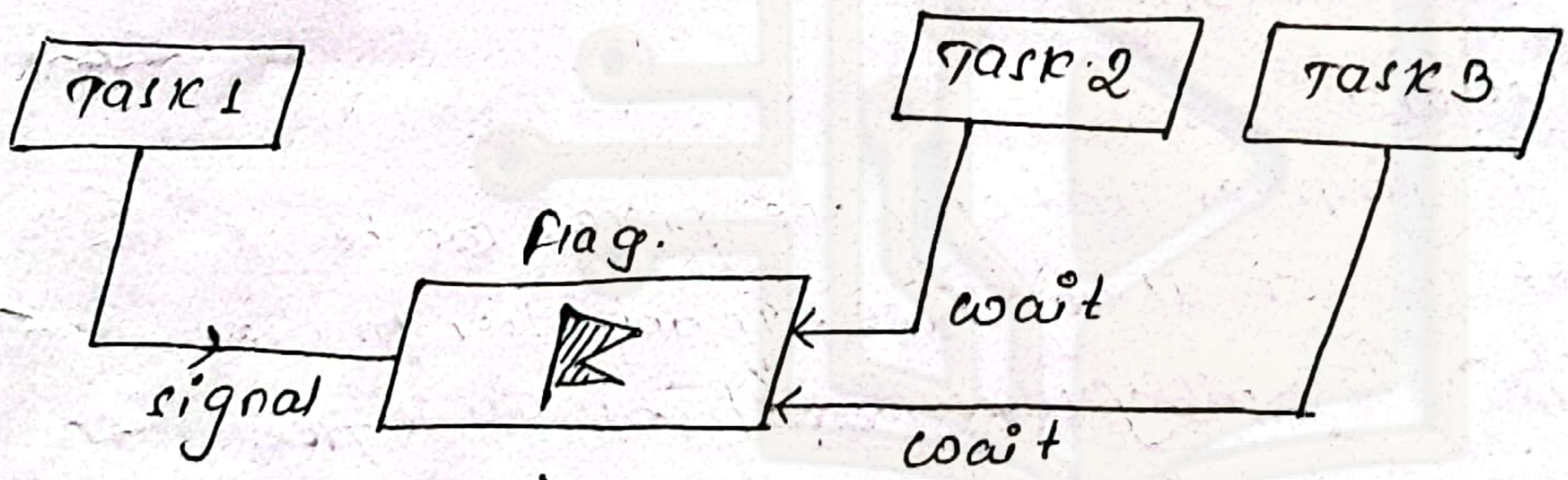
Available on notes.arpanadhikari7.com.np

first,
- determine the cause of the exception
- Handle the exception.

Kernel performs the following steps after handling exception;
→ select a process to restor and resume.
→ Restore the context of the selected process
→ Resume execution of the selected process.

6. Task synchronization

synchronization and messaging provides the necessary communication between tasks in one system to tasks in another system. The event flag is used to synchronize internal activities while message queues and mailboxes are used to send text message between systems.



यहाँ TASK 1, TASK 2 र 3 लाई synchronize गर्ने कामलाई Event flag.

↳ Task synchronization is achieved using two types of mechanism
→ Event objects
→ Semaphores.

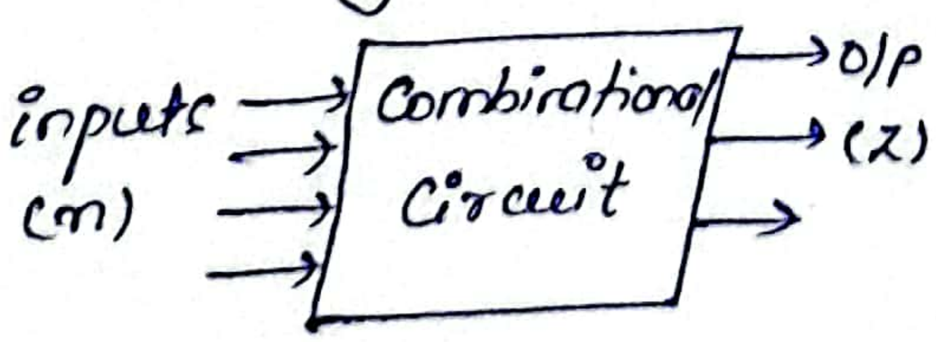
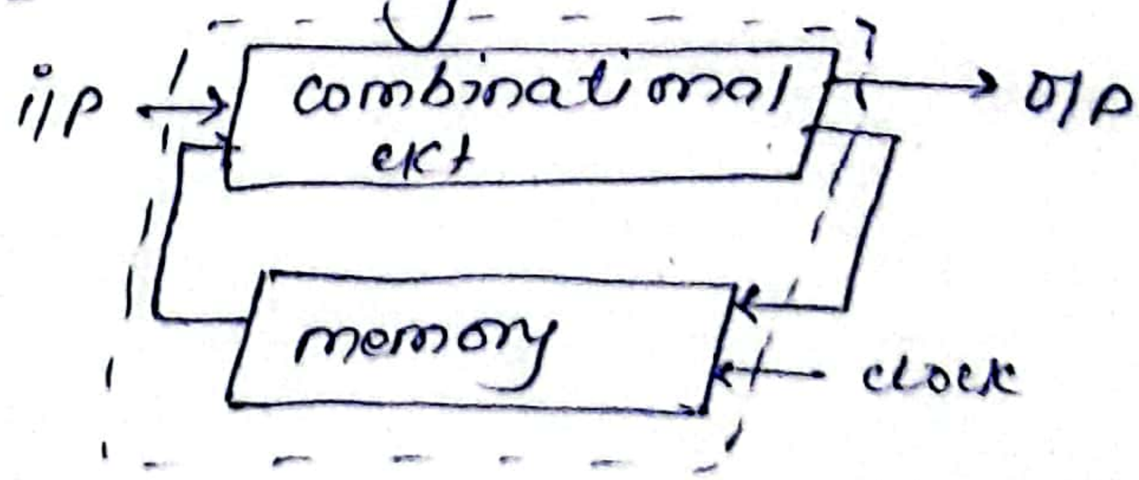
7. Task scheduling
↳ pre-emptive
↳ Non-preemptive
techniques EG Explain गर्दिने।

Available on notes.arpanadhikari7.com.np

Chapter : 02

Hardware and software design Issue

2.1. Combinational and sequential logic:

Combinational logic	sequential logic
1. combinational logic refers to circuit whose o/p is a function of the present value of the i/p only.	1. sequential logic circuit are those whose o/ps are also dependent upon past inputs and hence o/p.
2. It has no memory.	2. It has memory.
3. speed is fast.	3. speed is slow.
4. There is no feedback betn input and output.	4. There exists a feedback path betn i/p and o/p.
5. Elementary building blocks: logic gates.	5. Elementary building blocks: flip flops.
6. As combinational circuit don't have clock, they don't require triggering.	6. sequential logic ckt is clock dependent so need triggering.
7. eg: Encoder, Decoder, multiplexer, Demultiplexer.	7. eg: flip flop, counter.
8. Block Diagram: 	8. Block diagram: 

notes.arpanadhikari7.com.np

CamScanner

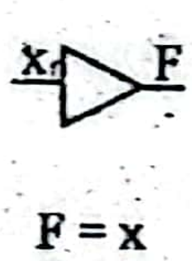
Available on notes.arpanadhikari7.com.np

- When both input "x" or "y" is logic 0 then both of the upper transistor conducts but both lower does not conducts .Thus logic 1 is appears as output "F".
- When both input "x" or "y" is logic 1 then both of the upper transistor does conducts but both lower does conducts :Thus logic 0 is appears as output "F".
- When one of the input "x" or "y" is logic 0 then at least one of the upper transistor and lower conducts .Thus logic 0 is appears as output "F".

Logic Gates:

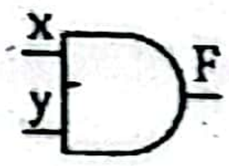
The Digital Logic Gate is the basic building block from which all digital electronic circuits and microprocessor based systems are constructed from. Basic digital logic gates perform logical operations of AND, OR and NOT on binary numbers.

Digital logic gates may have more than one input, (X, Y, Z, etc.) but generally only have one digital output, (F). Individual logic gates can be connected together to form combinational or sequential circuits, or larger logic gate functions.



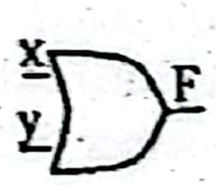
x	F
0	0
1	1

F = x
Driver



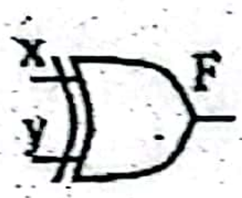
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

F = x y
AND



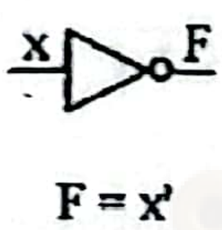
x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

F = x + y
OR



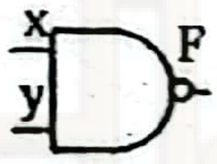
x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

F = x ⊕ y
XOR



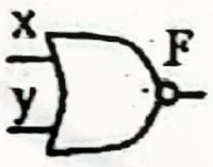
x	F
0	1
1	0

F = x'
Inverter



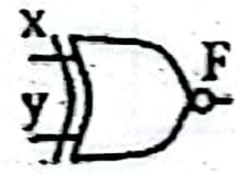
x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

F = (x y)'
NAND



x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

F = (x+y)'
NOR



x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

F = x ⊙ y
XNOR

Basic Combinational Logic Design:

2015 Spring
1 (b)

Combinational logic refers to circuits whose output is a function of the present value of the inputs only. Combinational logic circuits have no memory. The design procedure includes following steps:

① → Determine the no of input and output of system from the problem specifications.
② → Derive truth table for each output from their relationship with number of inputs.
③ → Simplify the Boolean expression using K-maps & obtain logic equations.
④ → Draw logic diagram (sharing common gates).
⑤ → Simulate circuit for design verification.
⑥ → Optimize the circuit for area and/or performance using different optimization metric.
⑦ → Re -simulate & verify optimized design.

For example: a combinational circuit that have three input say A, B and C and will give a single output X as logic 1 if the binary number consists more 1's than 0's.

CamScanner

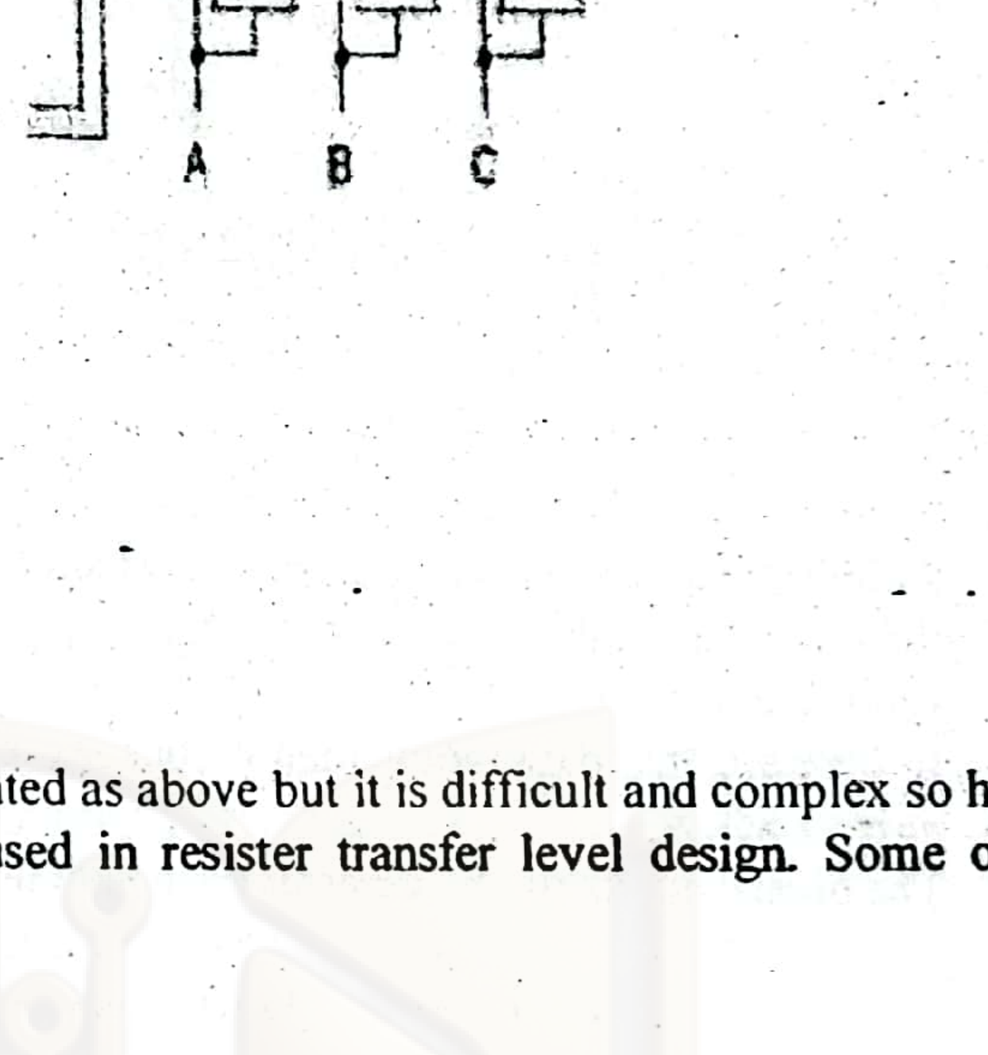
Available on notes.arpanadhikari7.com.np

	Inputs A	B	C	Output X
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

$\Rightarrow X = \sum(3,5,6,7)$

A \ BC	00	01	11	10
0	0	0	1	0
1	0	0	1	1

$X = AC + AB + BC$



RT-level Combinational Component:

All digital circuit can be designed as the procedure stated as above but it is difficult and complex so higher abstract level of combinational logic devices are used in resister transfer level design. Some of the component are:

1. Multiplexer:

A digital multiplexer is a combinational circuit that selects binary information from one of many input lines and directs it to a single output line.

Q. Why decoders and counters are respectively called combinational and sequential logic circuits? Write the design steps for both combinational and sequential logic circuit with corresponding block diagram.

A –a

Notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

1. Synchronous
2. Asynchronous

- A synchronous input value only have effect during in a clock edge.
- An asynchronous value effects the circuit independent of clock.

2015 spring

Design Procedure to Sequential Circuit Design:

The design of a clocked sequential circuit starts from a set of specifications (state table) and ends in a logic diagram or a list of Boolean functions from which the logic diagram can be obtained. The procedure can be summarized by a list of consecutive recommended steps:

1. State the word description of the circuit behavior. It may be a state diagram, a timing diagram, or other pertinent information.
2. From the given information about the circuit, obtain the state table.
3. Apply state-reduction methods if the sequential circuit can be characterized by input-output relationships independent of the number of states.
4. Assign binary values to each state if the state table obtained in step 2 or 3 contains letter symbols.
5. Determine the number of flip-flops needed and assign a letter symbol to each.
6. Choose the type of flip-flop to be used.
7. From the state table, derive the circuit excitation and output tables:
8. Using the map or any other simplification method, derive the circuit output functions and the flip-flop input functions.
9. Draw the logic diagram.

2008

Design Examples:

1. Design a combinational circuit for output y and z where y is 1 when a is 1 or b and c are 1 and z is 1 if b or c is 1 but not all i.e. a and b and c are 1.
   Truth Table:

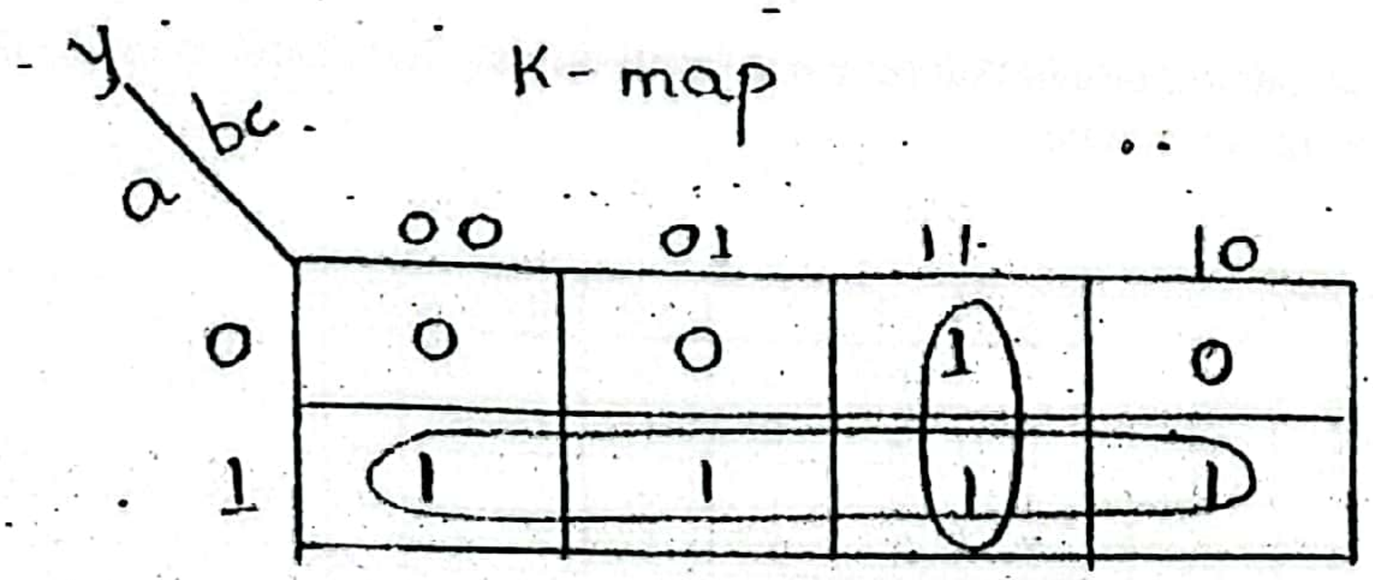
I/ps			O/ps	
a	b	c	y	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

notes.arpanadhikari7.com.np

CamScanner

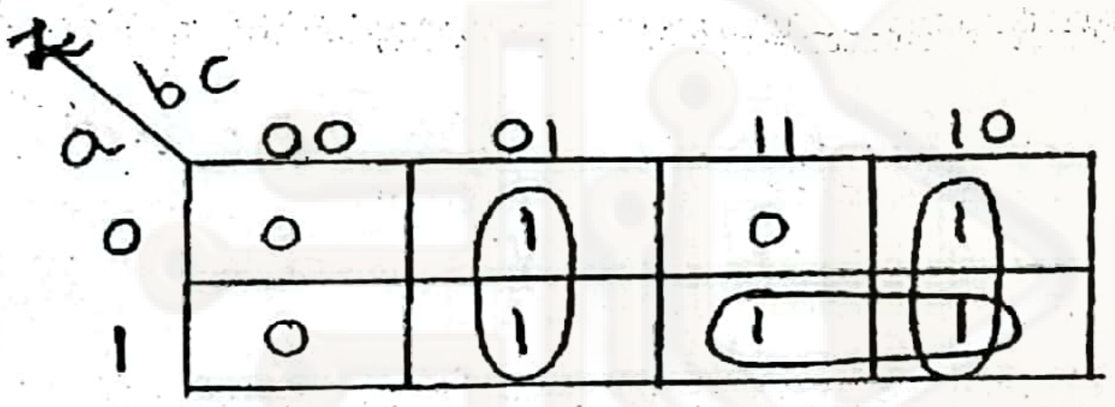
Available on notes.arpanadhikari7.com.np

K-map Reduction:



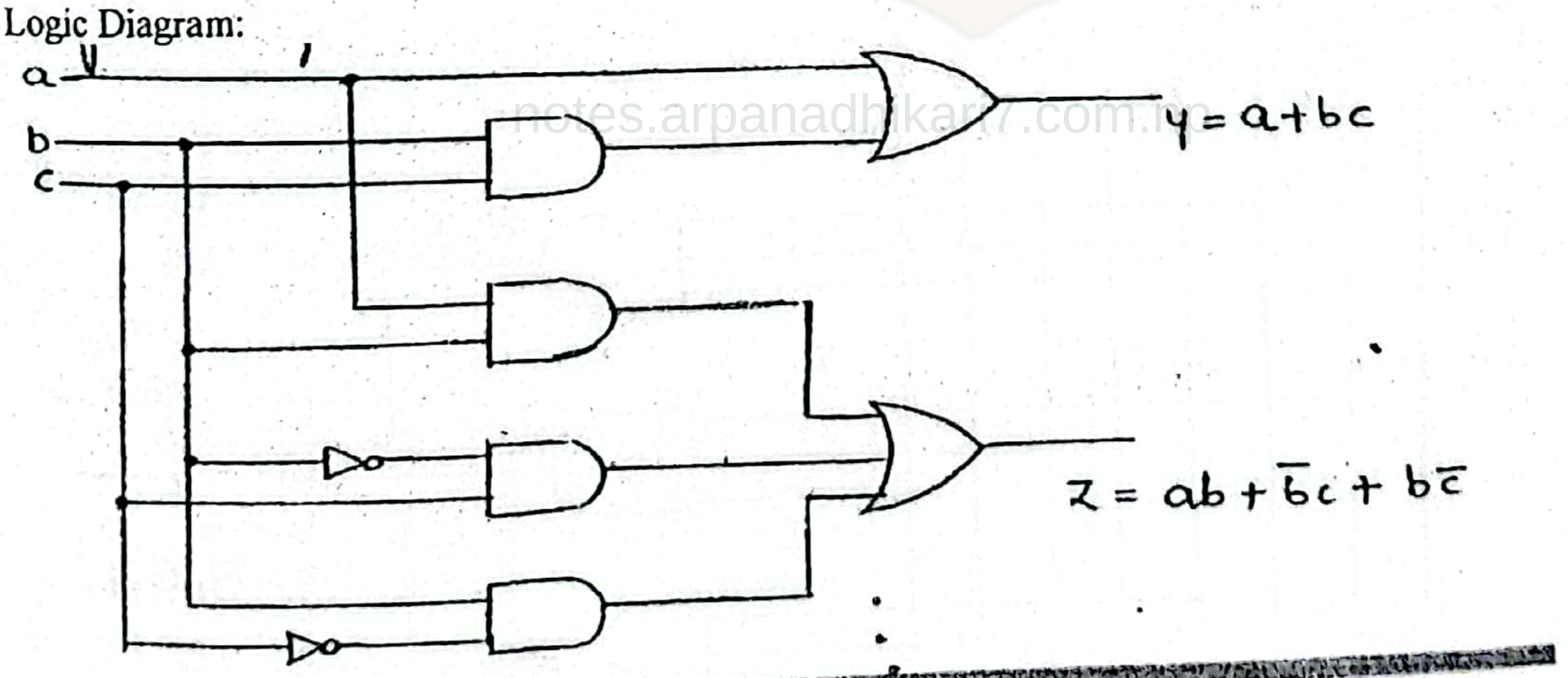
$$Y = bc + a\bar{b}\bar{c} + a\bar{b}c + abc + ab\bar{c}$$
$$= bc + a\bar{b}[\bar{c}+c] + ab[c+\bar{c}]$$
$$= bc + a\bar{b} + ab$$
$$= bc + a[\bar{b}+b]$$
$$= bc + a$$

$Y = a + bc$.



$$z = \bar{b}c + ab + b\bar{c}$$
$$z = ab + \bar{b}c + b\bar{c}$$

Logic Diagram:



notes.arpanadhikari7.com.np

Prepared By: Er.Loknath Regmi

CamScanner
Available on notes.arpanadhikari7.com.np

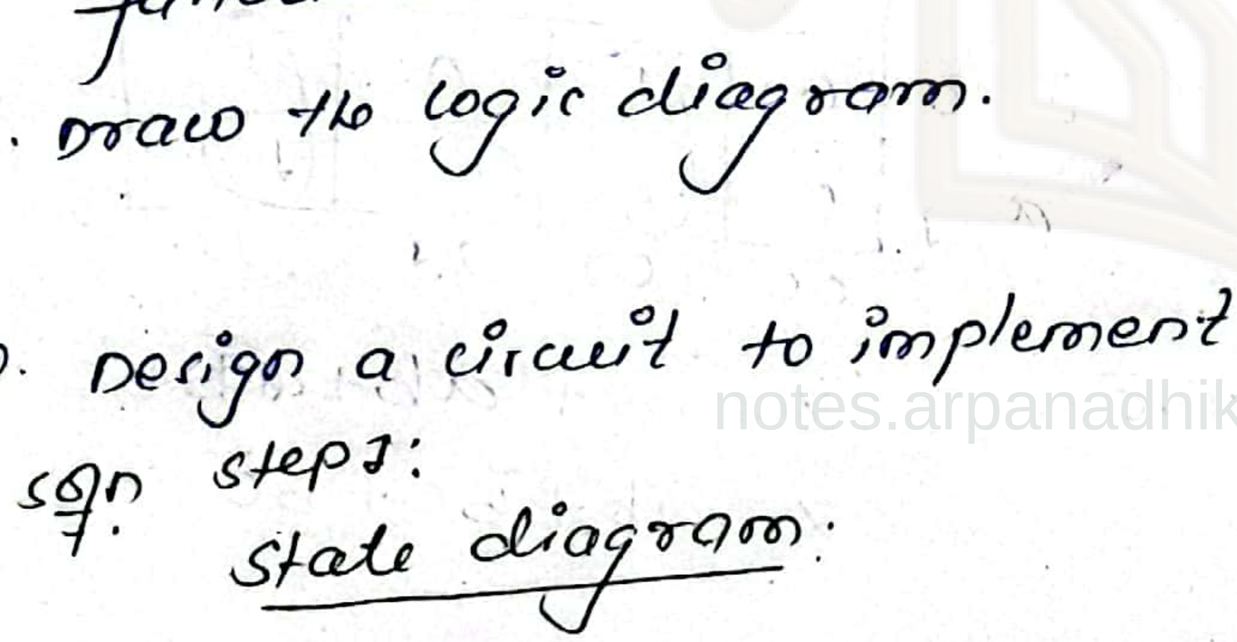
#. Design Procedure to sequence circuit Design

1. state the word description of the circuit behavior. It may be a state diagram, a timing diagram or other form.
2. from the given information about the circuit, obtain the state table.
3. Apply state reduction if possible.
4. Assign binary values to each state if the state table obtained contains letter symbols.
5. determine the number of flip-flops needed
6. choose the type of flip-flop to be used.
7. from the state table, derive the circuit excitation and output tables.
8. using the map or any other simplification method, derive the circuit output function and the flip-flop input functions.
9. draw the logic diagram.

Q. Design a circuit to implement 2 Bit gray code (self reflecting code).

Soln step 1:

State diagram:



notes.arpanadhikari7.com.np

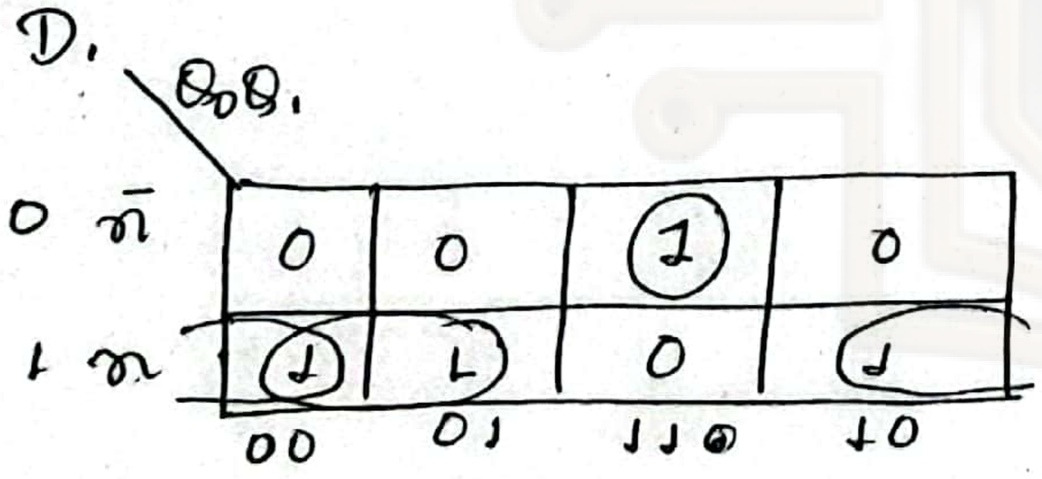
CamScanner

Available on notes.arpanadhikari7.com.np

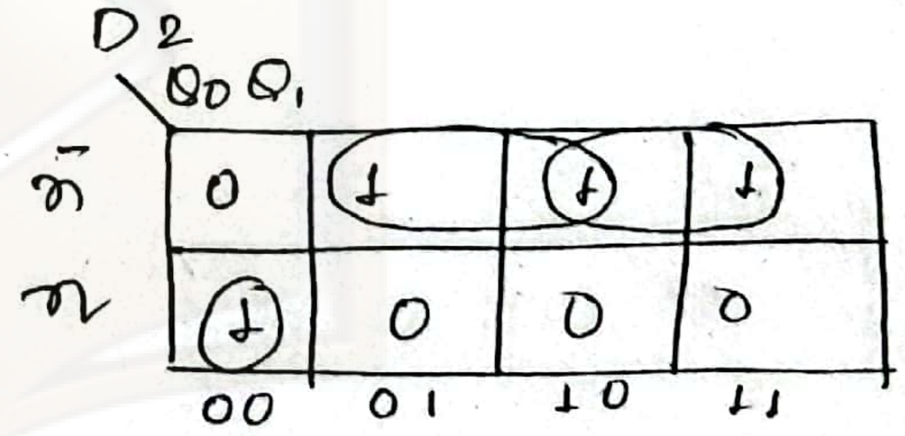
step 2:
The above state diagram can be represent in state table.

present state			Next state		D FIF (output	
Q_0	Q_1	x	Q_0'	Q_1'	D_1	D_2
0	0	0	0	0	0	0
0	0	1	0	1	0	1
0	1	0	0	1	0	1
0	1	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	0	1	0
1	0	0	1	0	1	0
1	0	1	0	0	0	0

step 3: K-map of output D_1 and D_2

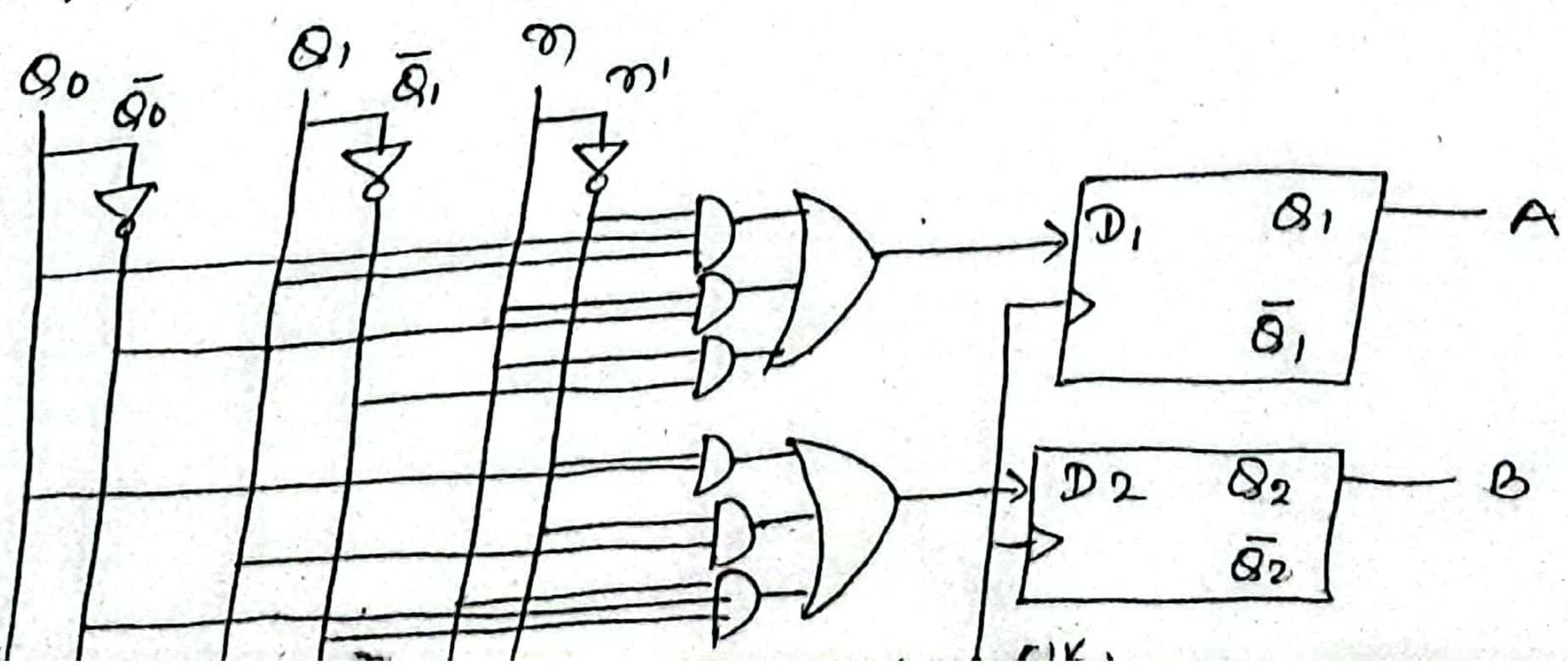


$\therefore D_1 = \bar{x}Q_0Q_1 + x\bar{Q}_0 + x\bar{Q}_1$



$\therefore D_2 = \bar{x}Q_0 + \bar{x}Q_1 + x\bar{Q}_1\bar{Q}_0$

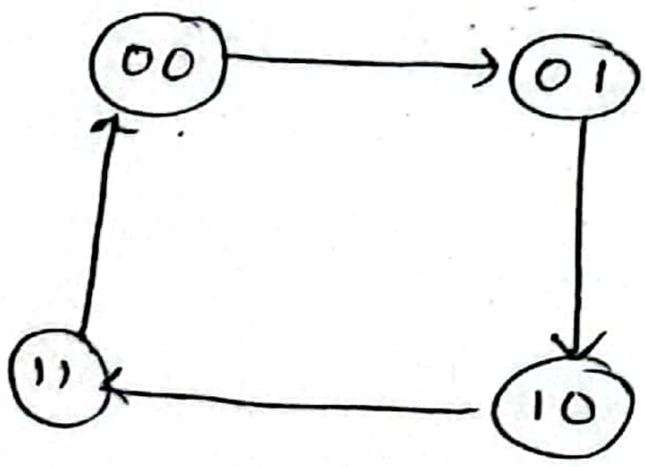
step 4: state diagram:



notes.arpanadhikari7.com.np
CamScanner
Available on notes.arpanadhikari7.com.np

2019 fall Design a seq. ckt to implement 2-bit counter with an input that controls up (increment value) or down (decrease value) count.

Seq. state Diagram for 2 bit counter:



Excitation Table:

Present value		Next value		output	
Q_0	Q_1	Q_0'	Q_1'	D_1	D_2
0	0	0	1	0	1
0	1	1	0	1	0
1	0	1	1	1	1
1	1	0	0	0	0

K-map for output D_1 and D_2

D_1

$Q_0 \backslash Q_1$	$\bar{Q}_1$	Q_1
$\bar{Q}_0$	0	(1)
Q_0	(1)	0

$$D_1 = Q_0\bar{Q}_1 + \bar{Q}_0 Q_1 = Q_0 \oplus Q_1$$

D_2

$Q_0 \backslash Q_1$	$\bar{Q}_1$	Q_1
Q_0	(1)	0
1	1	0

$$\therefore D_2 = \bar{Q}_1$$

Circuit diagram.

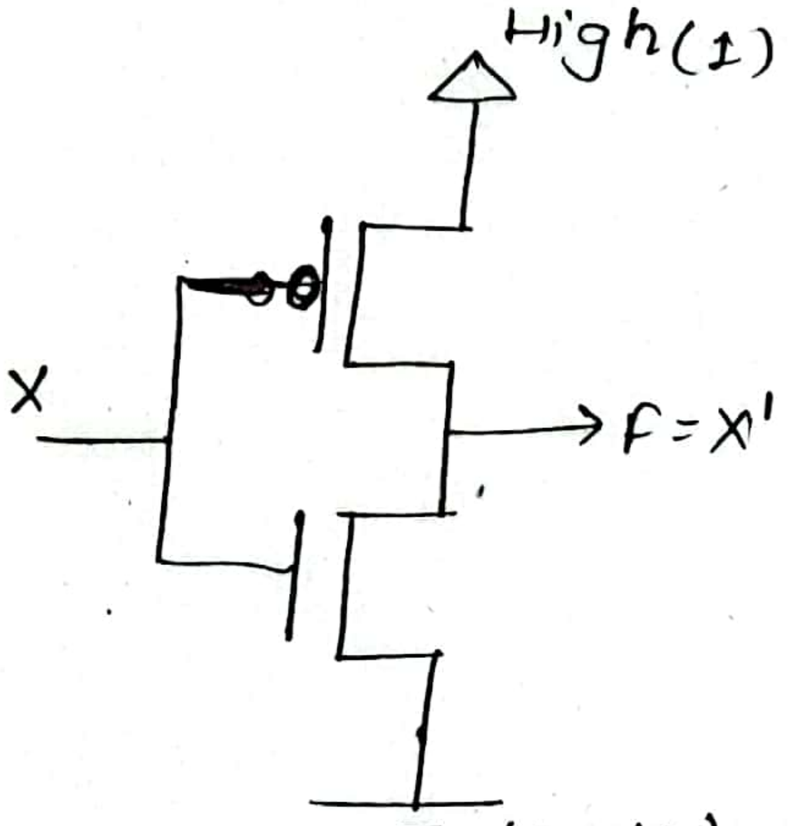
notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

Implementation of logic Gate using CMOS.

1> INVERTER (NOT):



symbol.

Truth table of NOT Gate.

X	F
0	1
1	0

[∵ F = X']

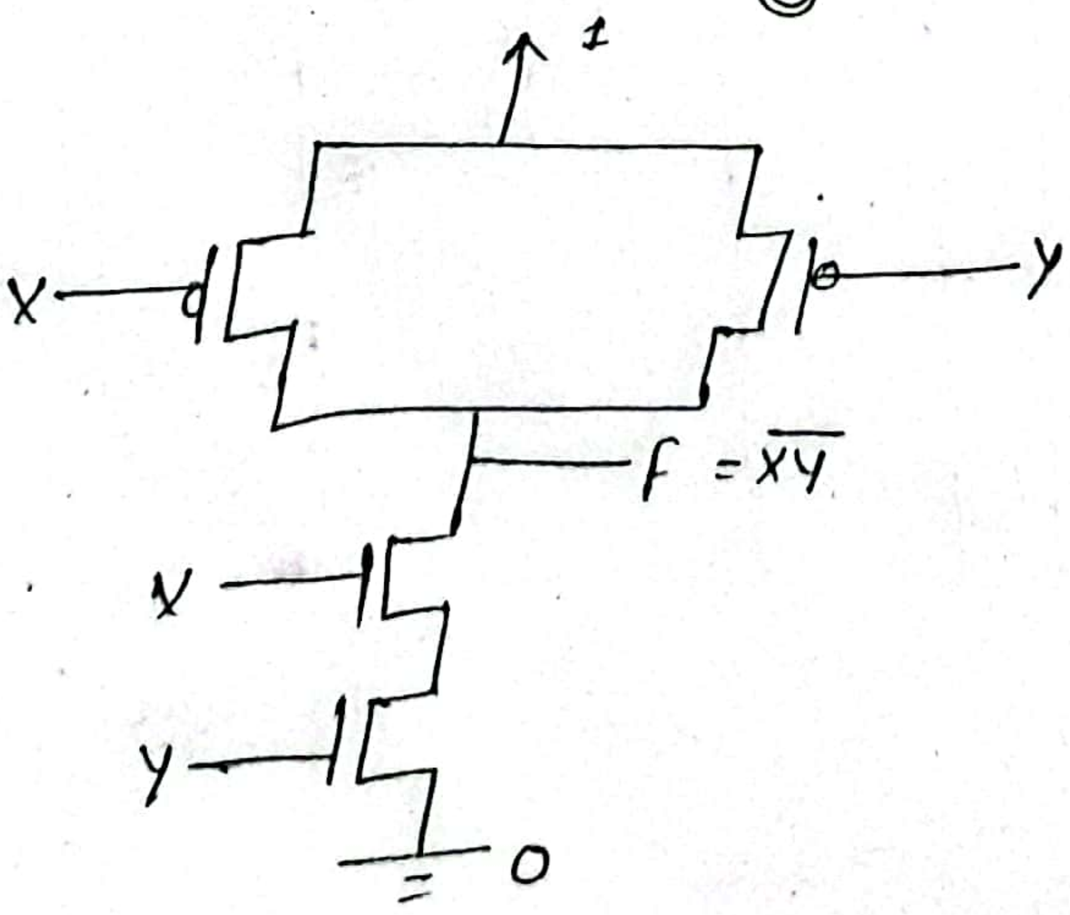
fig: NOT Gate using CMOS

Explanation:

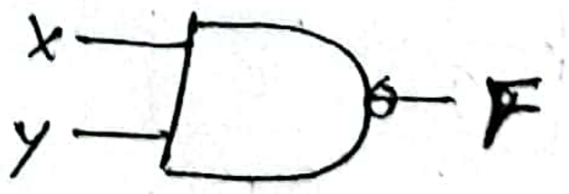
↳ When the input is Logic 0, the upper transistor conducts and lower transistor does not conducts. Thus logic 1 is appears as output "F".

↳ Similarly when the input "X" is logic 1, the upper transistor doesnot conducts and lower transistor conducts. Thus logic 0 is appears as output "F".

2> NAND Gate using CMOS.



symbol:



Truth Table

X	Y	F = $\overline{XY}$
0	0	1
0	1	1
1	0	1
1	1	0

inputs outputs

Available on notes.arpanadhikari7.com.np

- When one of the input "x" or "y" is logic 0 then one of the upper transistor and lower transistor conduct. Thus logic 1 is appears as output "F".
- When both input "x" and "y" is logic 1 then neither of the upper transistor conducts but both lower conducts.

  ie. upper 2 transistor = open circuit
  lower 2 transistor = short circuit

  Thus, ground value appear at output.

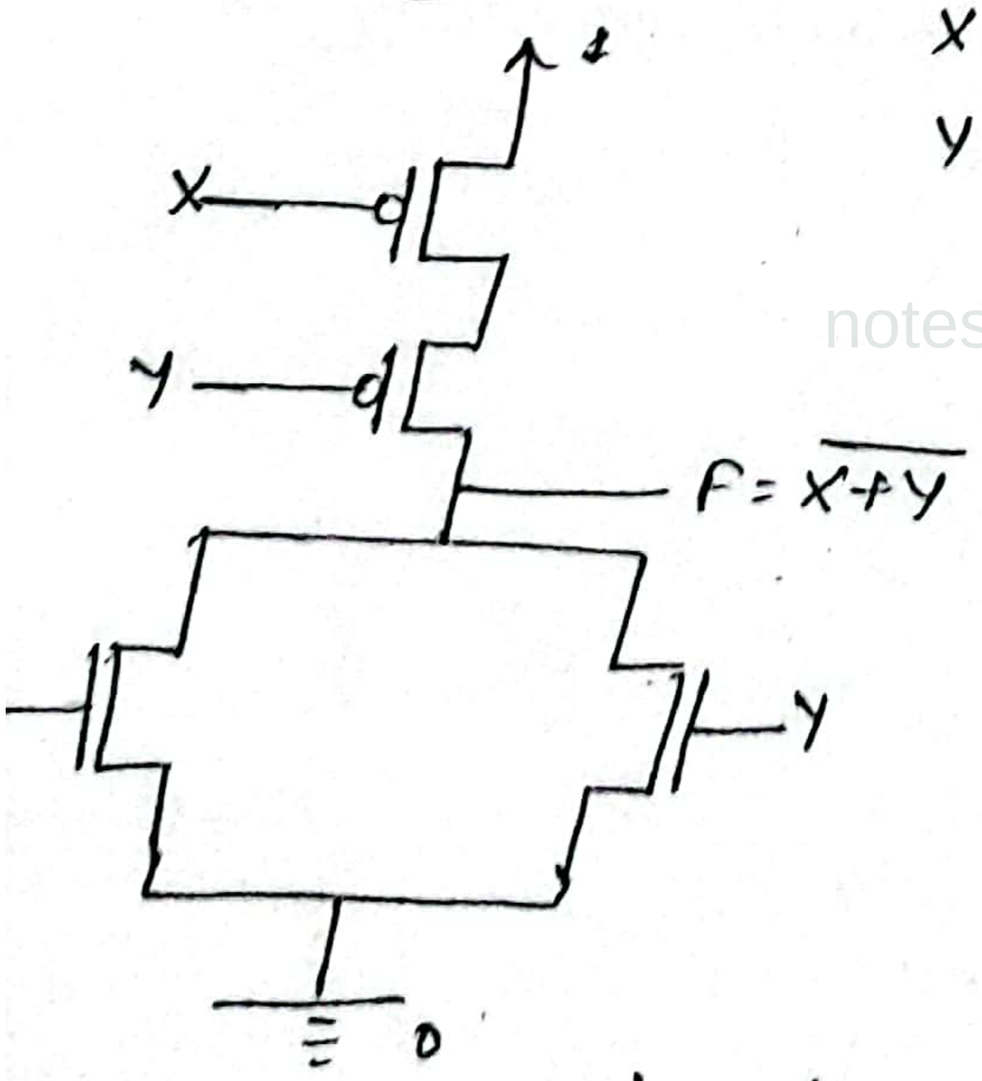
  ie. F = 0

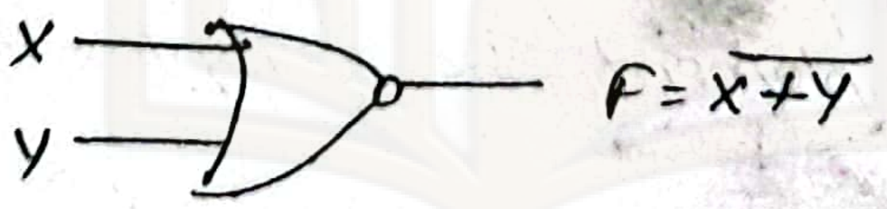
- When both input "x" and "y" is logic '0' then upper 2 transistor conducts and lower too transistor remains closed. Thus, High/upper value 1, appers at output.

  ie. F = 1.

2020 fall

3> NOR Gate :





notes.arpanadhikari7.com.np

Truth Table:

inputs		output
x	y	F = x̄+y
0	0	1
0	1	0
1	0	0
1	1	0

- When both inputs are Loco, upper too transistor conducts and upper value appears at output. ie. F = 1
- When Both inputs are high, lower too transistor conducts and

CamScanner

Available on notes.arpanadhikari7.com.np

hence ground value appears at output
ie. f= 0

6 when atleast one value is logic zero, then the output remains 0.
ie. f=0

2017 spring
2019 spring
2016 fall

Design a custom single-purpose processor to calculate GCD betn 2 integers.

soln steps to design single custom processor.

1. Draw a black box that shows the abstract view of the implementation logic.
2. Derive the algorithms to implement the functionality of system.
3. Derive the state diagram to implement the operational logic
4. Design the data path, functional unit as well as controller to implement the complex logic specified in step 2.

as black-box view

notes.arpanadhikari7.com.np

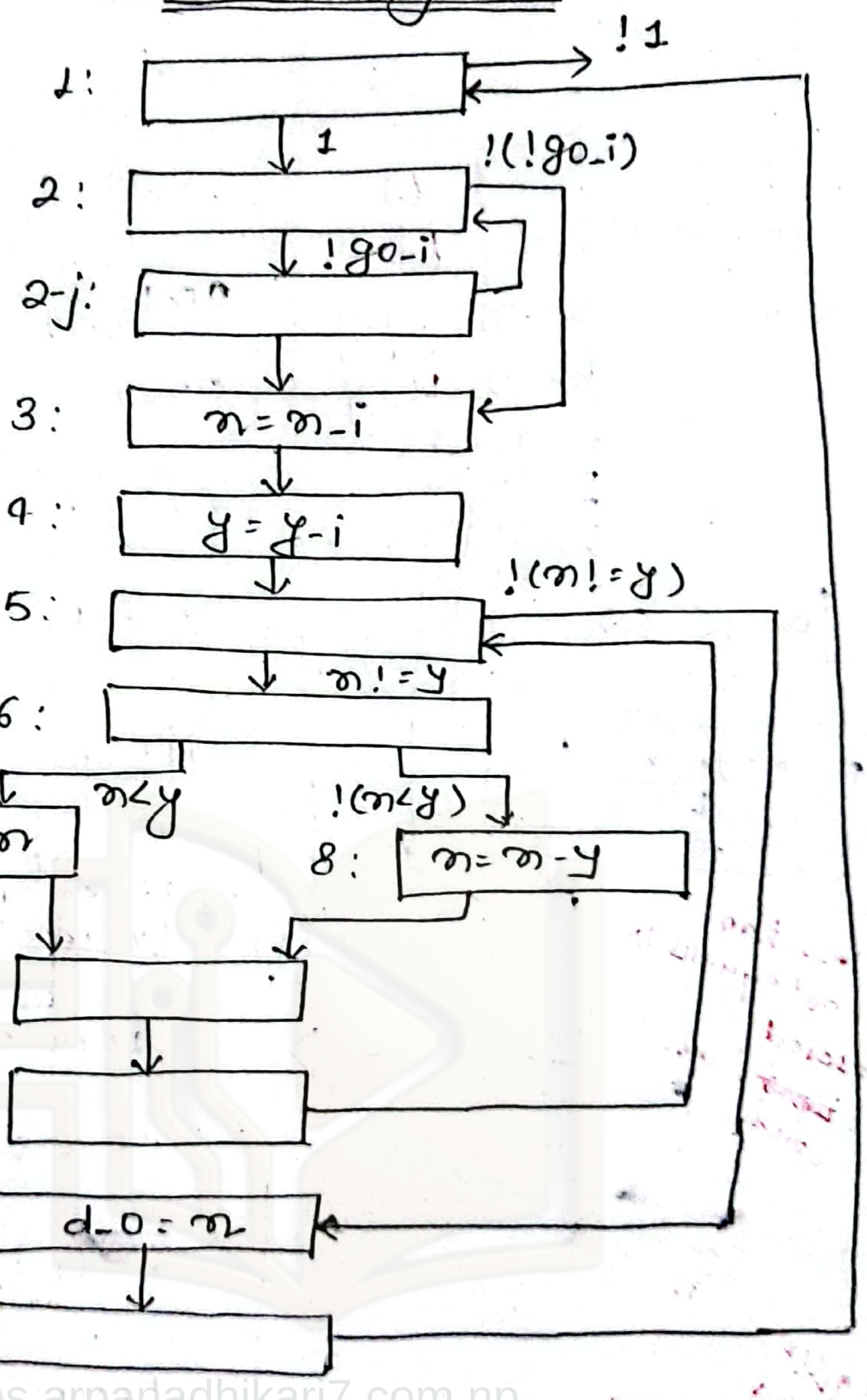
CamScanner

Available on notes.arpanadhikari7.com.np

② create algorithm

```
0: int m,y;
1: while (1) {
2: while (!go_i);
3: m=m_i;
4: y=y-i;
5: while (m!=y) {
6: if (m<y)
7: y=y-m;
   else
8: m=m-y;
   }
9: d_o=m;
}
```

③ state diagram :



part 1 को fig explain गर्दा

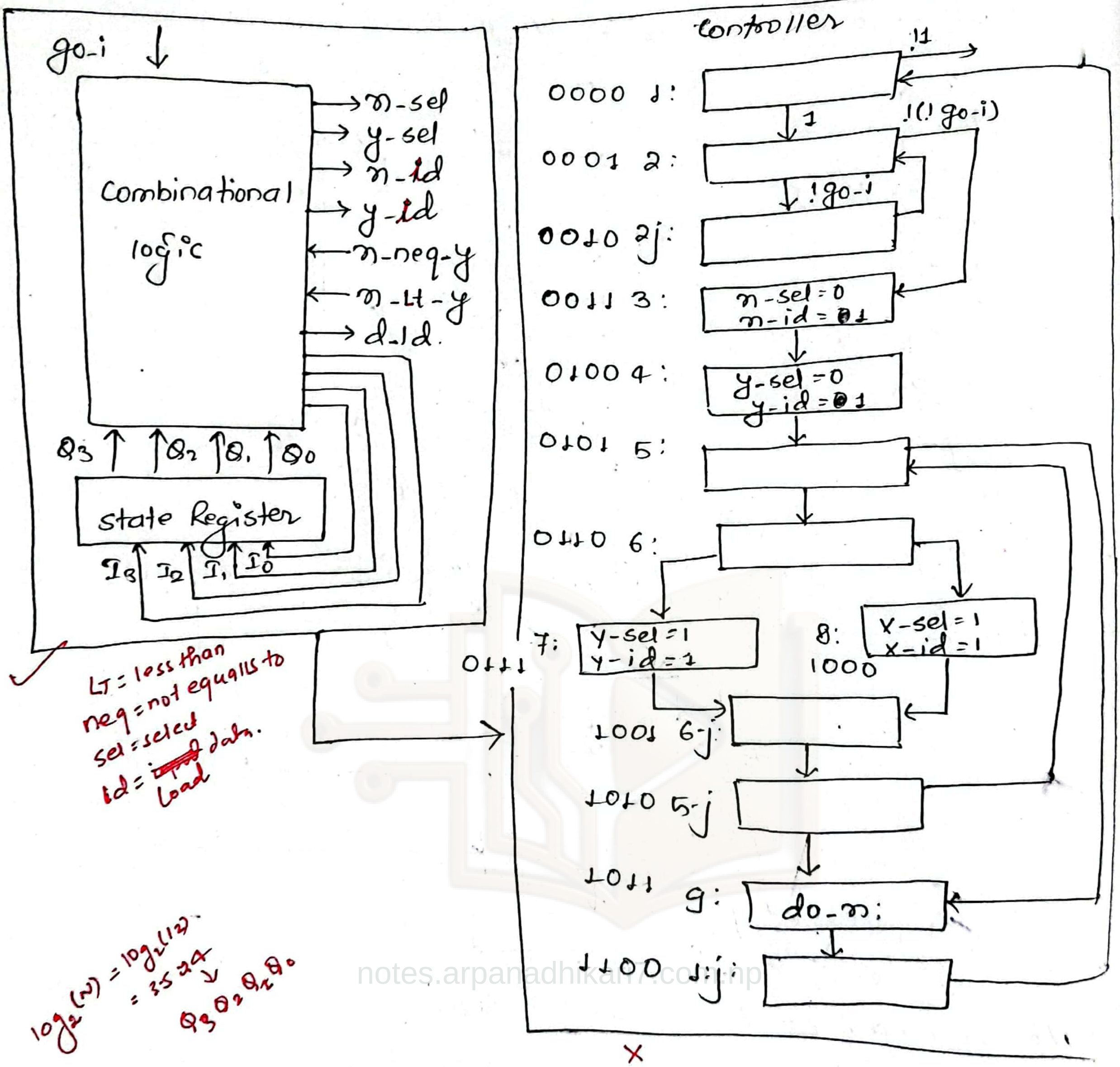
The data path can be designed as below:

1. Create a register for any declared variable.
2. Create a functional unit for each arithmetic operation.
3. connect the ports, registers and functional units
   - Based on reads and write
   - use multiplexor for multiple source.
4. Create unique identifier

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

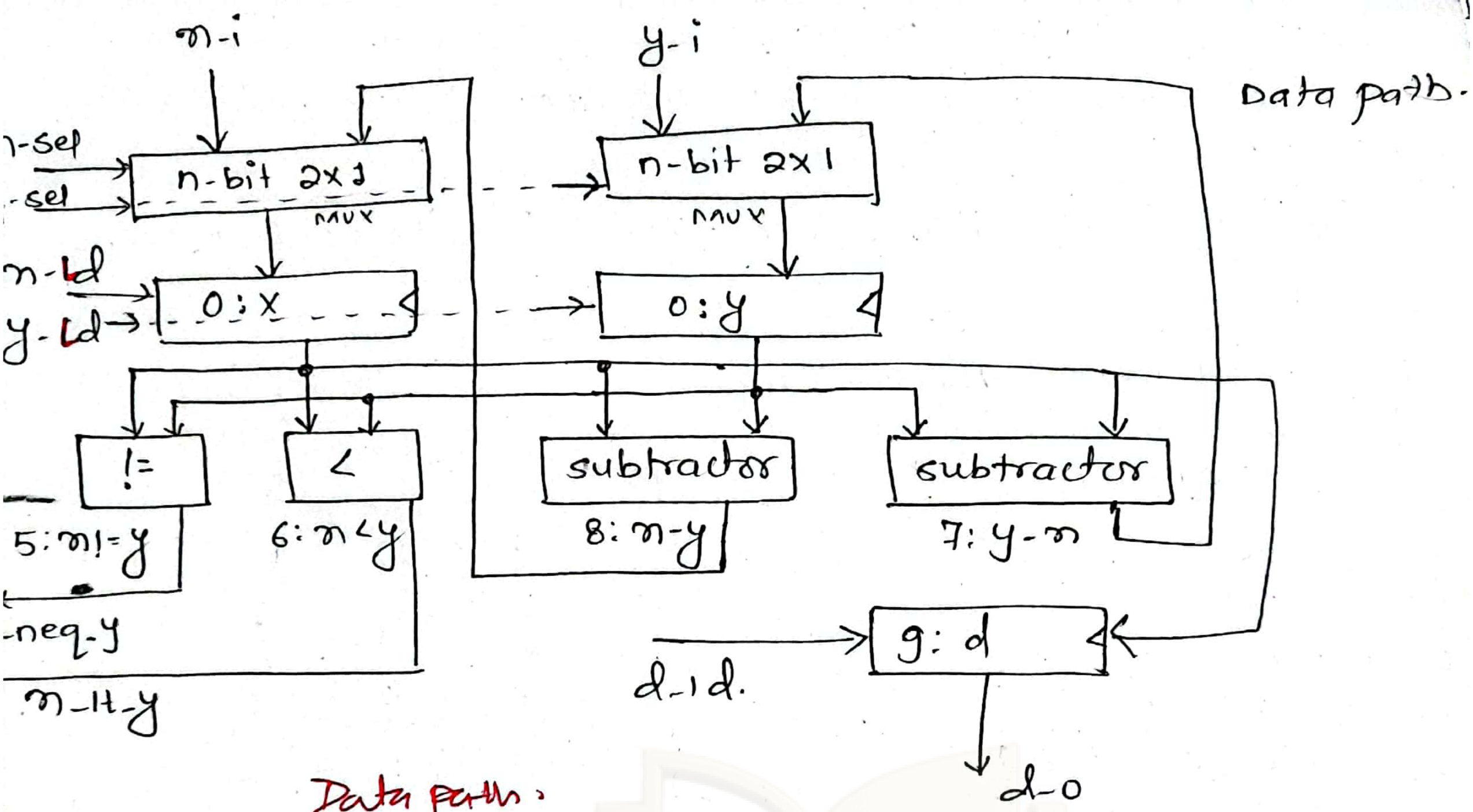


$\log_2(n) = \log_2(12)$
$= 3.584$
$\downarrow$
$Q_3 Q_2 Q_1 Q_0$

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np



Data path:

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2020 fall 2(a). Define optimization. Explain the different optimization opportunities.

ans: i) orginal program
ii) FSMD (finite state machine with datapath)
iii) Data path
iv) FSM for controller.

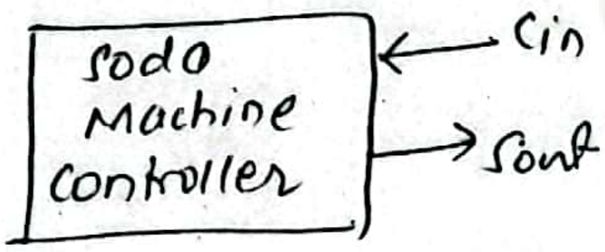
2021 fall

Target Processor: The processor that will run the program.
Development processor: The processor on which programs are written.

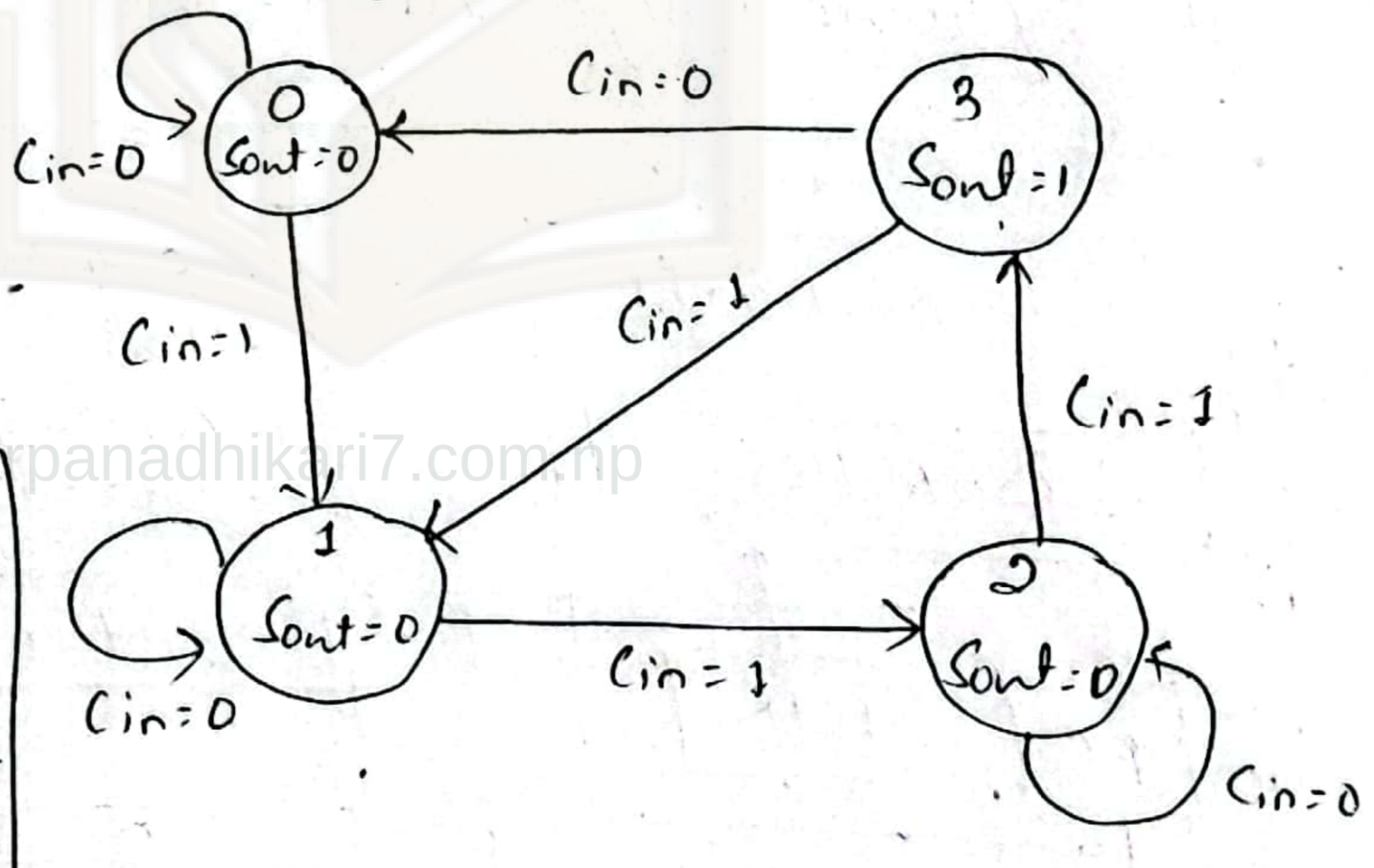
2022

Q. Design a soda controller machine given that a soda costs 75 cents and your machine accepts quarters only. Draw the black box view, come up with state diagram, and draw the final circuit using D flf.

soln Black Box view:



State Diagram



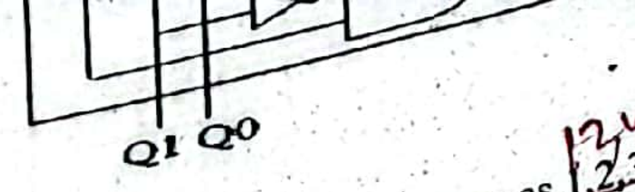
State Table

Inputs			outputs		
Q_1	Q_0	Cin	Q_1^+	Q_0^+	Sout
0	0	0	0	0	0
0	0	1	0	1	
0	1	0	0	1	0
0	1	0	1	0	
1	0	0	1	0	0
1	0	1	1	1	
1	1	0	0	0	1
1	1	1	0	1	

notes.arpanadhikari7.com.np

CamScanner

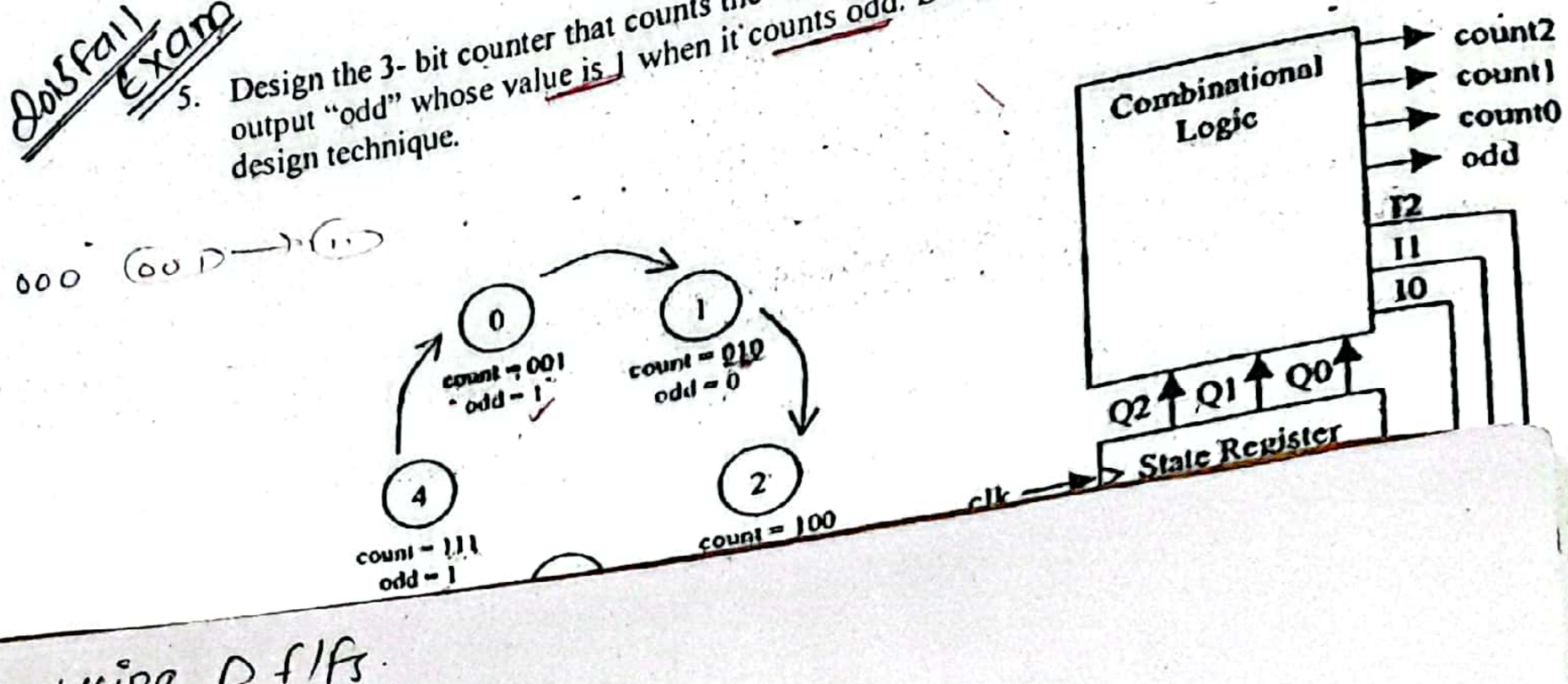
Available on notes.arpanadhikari7.com.np



This counter has

2075 Fall Exam

5. Design the 3- bit counter that counts the sequence as 1,2,3,4,5,6,7,1,2,3,...etc. . output "odd" whose value is 1 when it counts odd. Design the circuit by using sequential circuit design technique.



Excitation table using D flfs.

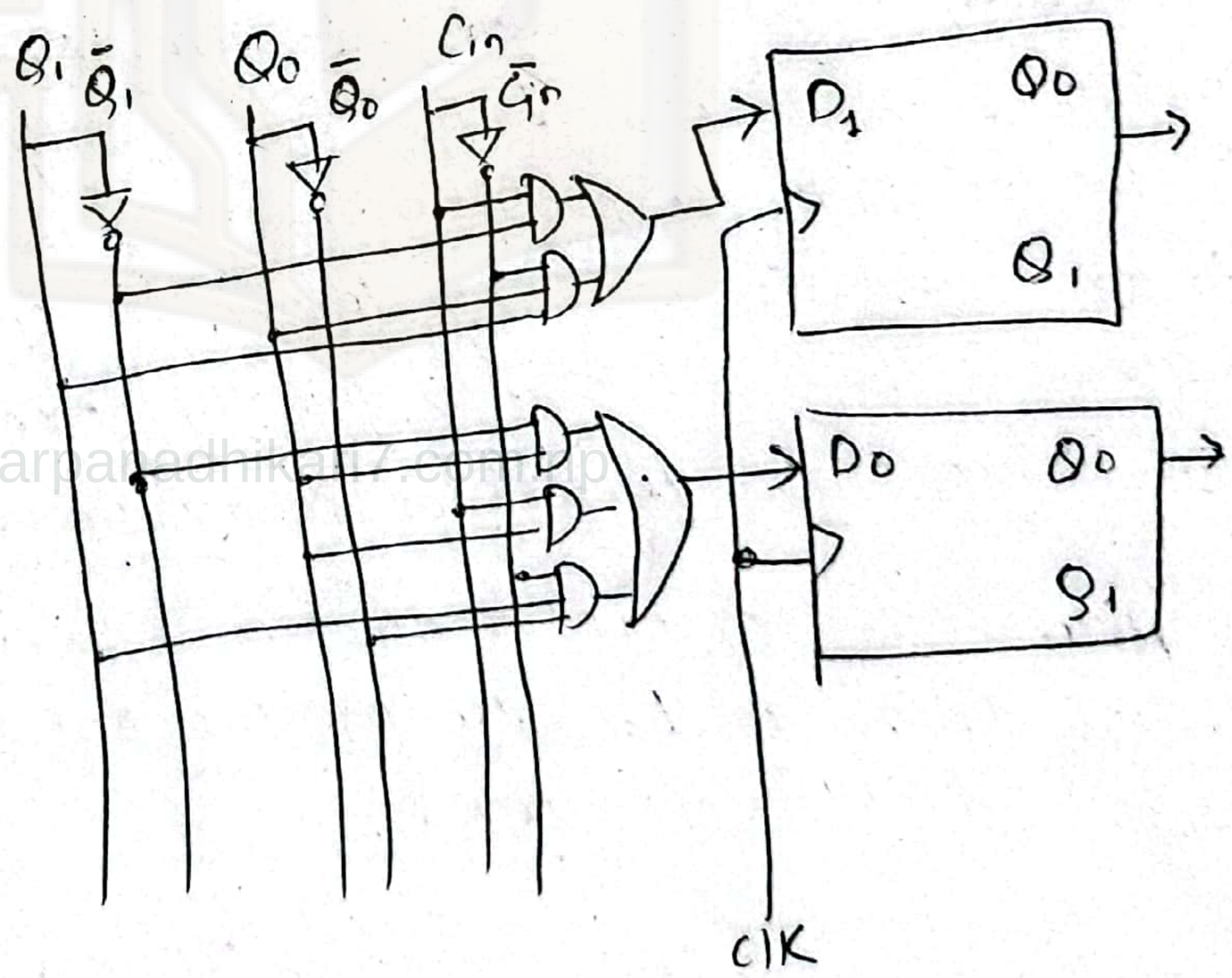
Q_1'	Q_0'	D_1	D_0	Sout
0	0	0	0	0
0	1	0	1	
0	1	0	1	0
0	1	1	0	
01	0	1	0	0
1	0	1	1	
1	1	0	0	
0	0	0	1	1
0	1	0	0	

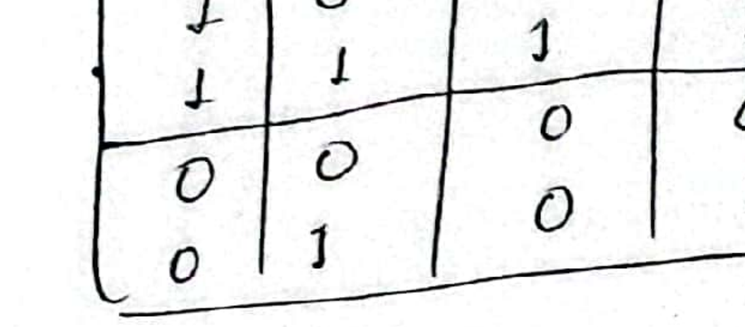
Truth Table

D_1	$\bar{Q}_1\bar{Q}_0$ (00)	$\bar{Q}_0Q_1$ (01)	Q_1Q_0 (11)	$Q_1\bar{Q}_0$ (10)
$\bar{C}_{in}$	0	0	(1)	0
C_{in}	(1)	1	0	0

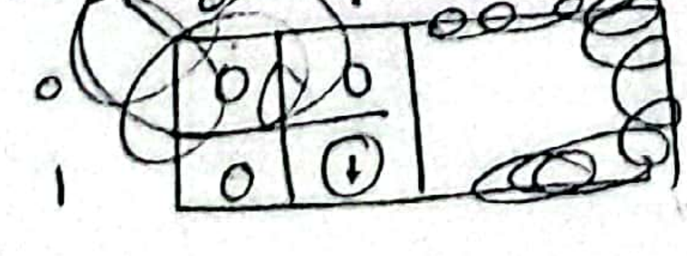
$\therefore D_1 = C_{in}\bar{Q}_1 + \bar{C}_{in}Q_0Q_1$

logic diagram

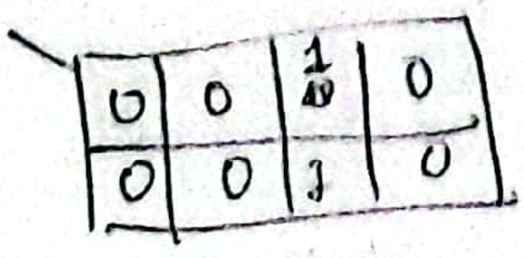




$D_0 = \bar{Q}_1Q_0 + C_{in}Q_0 + \bar{C}_{in}Q_1\bar{Q}_0$



Sout



$\therefore Sout = Q_0Q_1$

Available on notes.arpanadhikari7.com.np

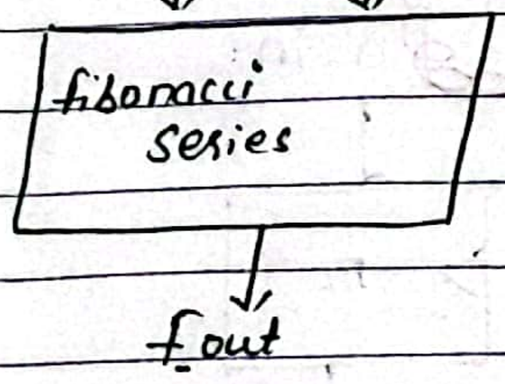
2018 spring: 2(a).

Design a custom single-purpose processor that generates fibonacci-series - up to 'n' places:

Sol^n [Note: fibonacci series is 0, 1, 1, 2, 3, 5, 8, 13, 21, ---]

Initially, let's draw black box for fibonacci series.

1. go_in n_in

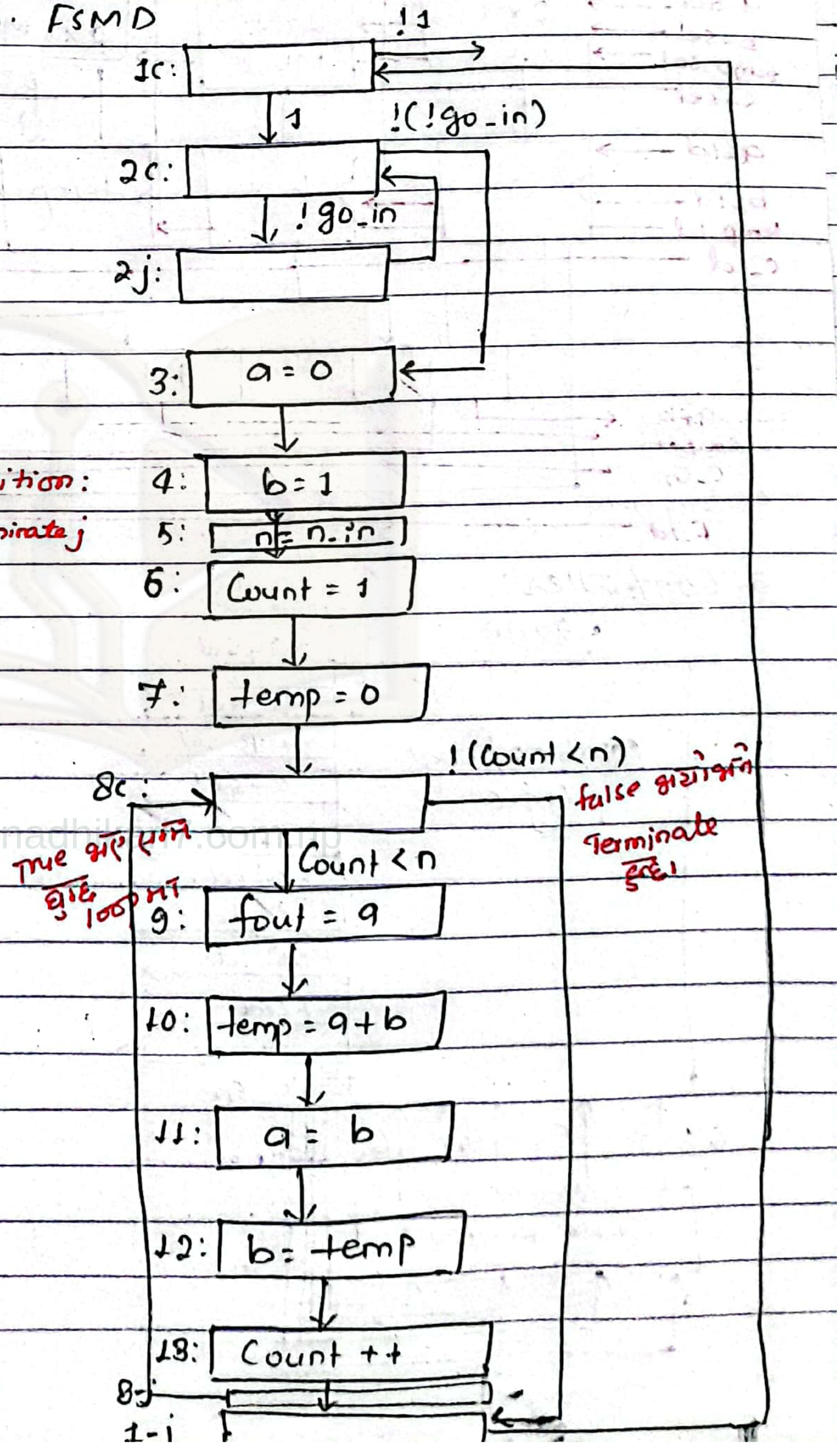


2. functionality Code:

```
0:  int a, b, n, count, temp;
1c: while (1) {
2c: while (!go_i)
3:  {a = 0;
4:  b = 1;
5:  n = n_in;
6:  count = 1;
7:  temp = 0;
8c: while (count < n)
    {
9:  fout = a;
10: temp = a + b;
11: a = b;
12: b = temp;
13: count ++;
    }
  }
}
```

c = condition: (c को terminate हो।)

3. FSMD



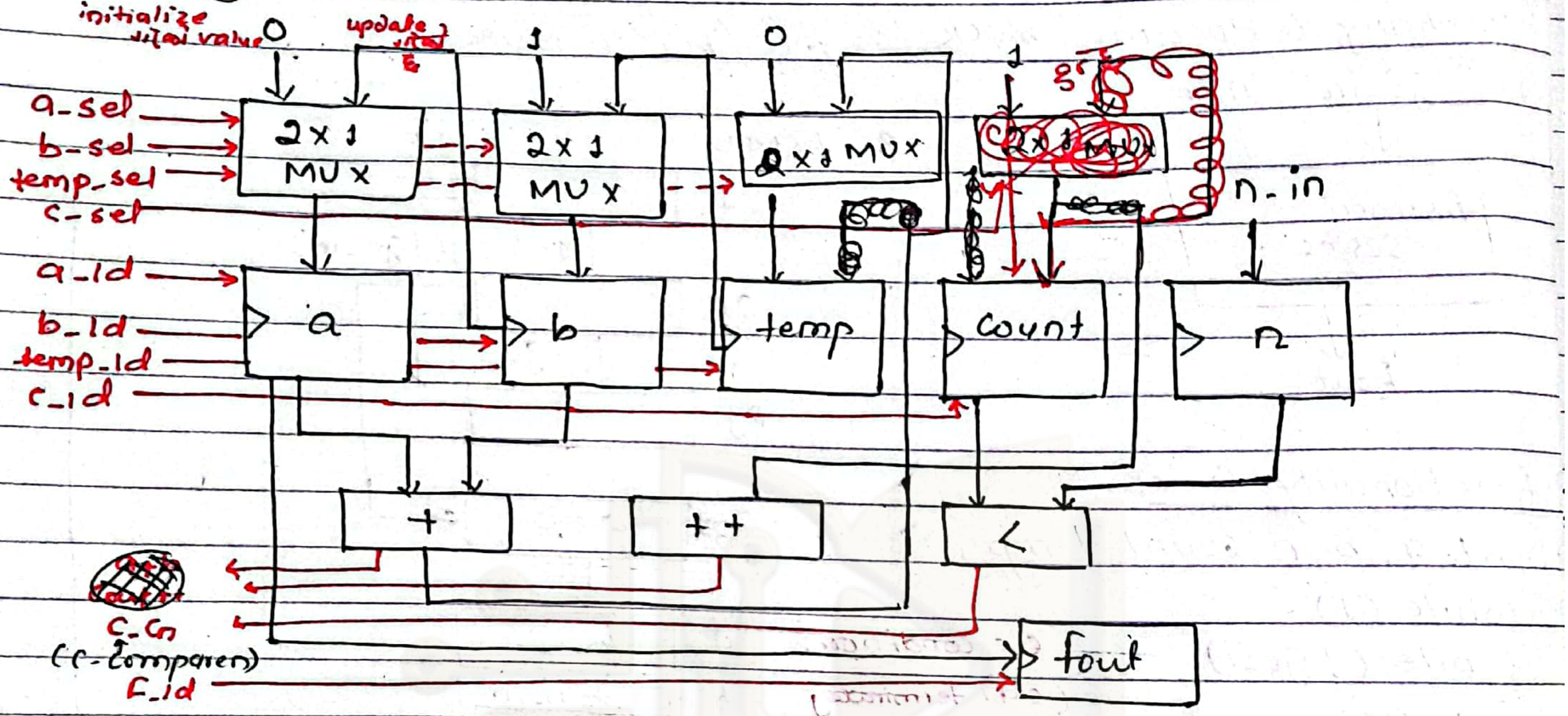
notes.arpanadhikari7.com.np

CamScanner

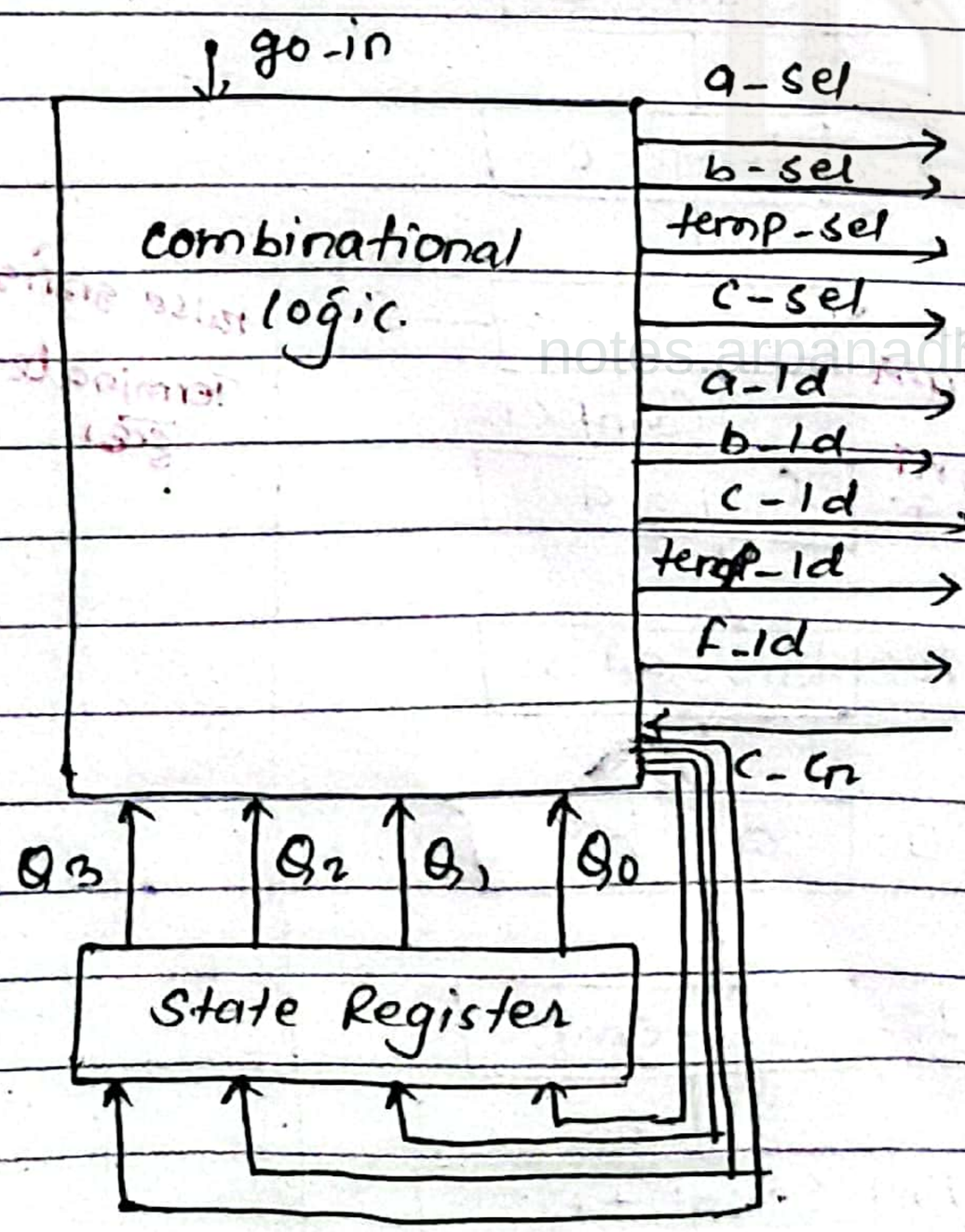
Available on notes.arpanadhikari7.com.np

4. Data Path

↳ Data path बनाउँदा सुरुमा program मा आती वेरा variable हु, त्यति नै register बनाउने ।



5. Controller:



Note:

formula to count no of state

$n = \log_2(N)$

Here,

$N = 13$

$\therefore n = \log_2(13)$

$= 4$ (ie. Q_3, Q_2, Q_1, Q_0)

notes.arpanadhikari7.com.np

CamScanner

Available on notes.arpanadhikari7.com.np

2019 Spring:

Design a custom single purpose processor to calculate LCM (least common multiple) between two integer.

Eqn Concept for LCM = $\frac{x \times y}{GCD}$ [for eg: LCM of 7 and 4 is 28

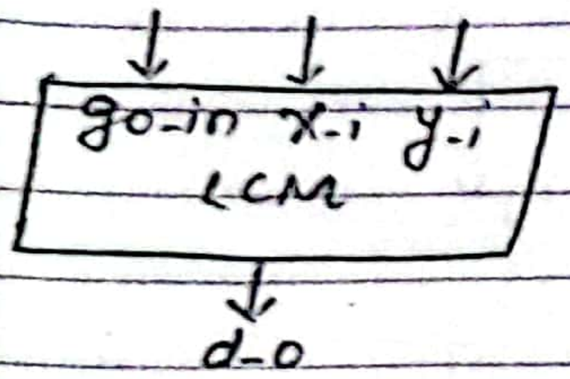
$x = 7$
$y = 4$

$\therefore LCM = \frac{7 \times 4}{GCD} = \frac{7 \times 4}{1}$
$= 28$

GCD: $7 = 7 \times 1$
$4 = 2 \times 2 \times 1$

$\therefore$ GCD of 7 and 4 = 1]

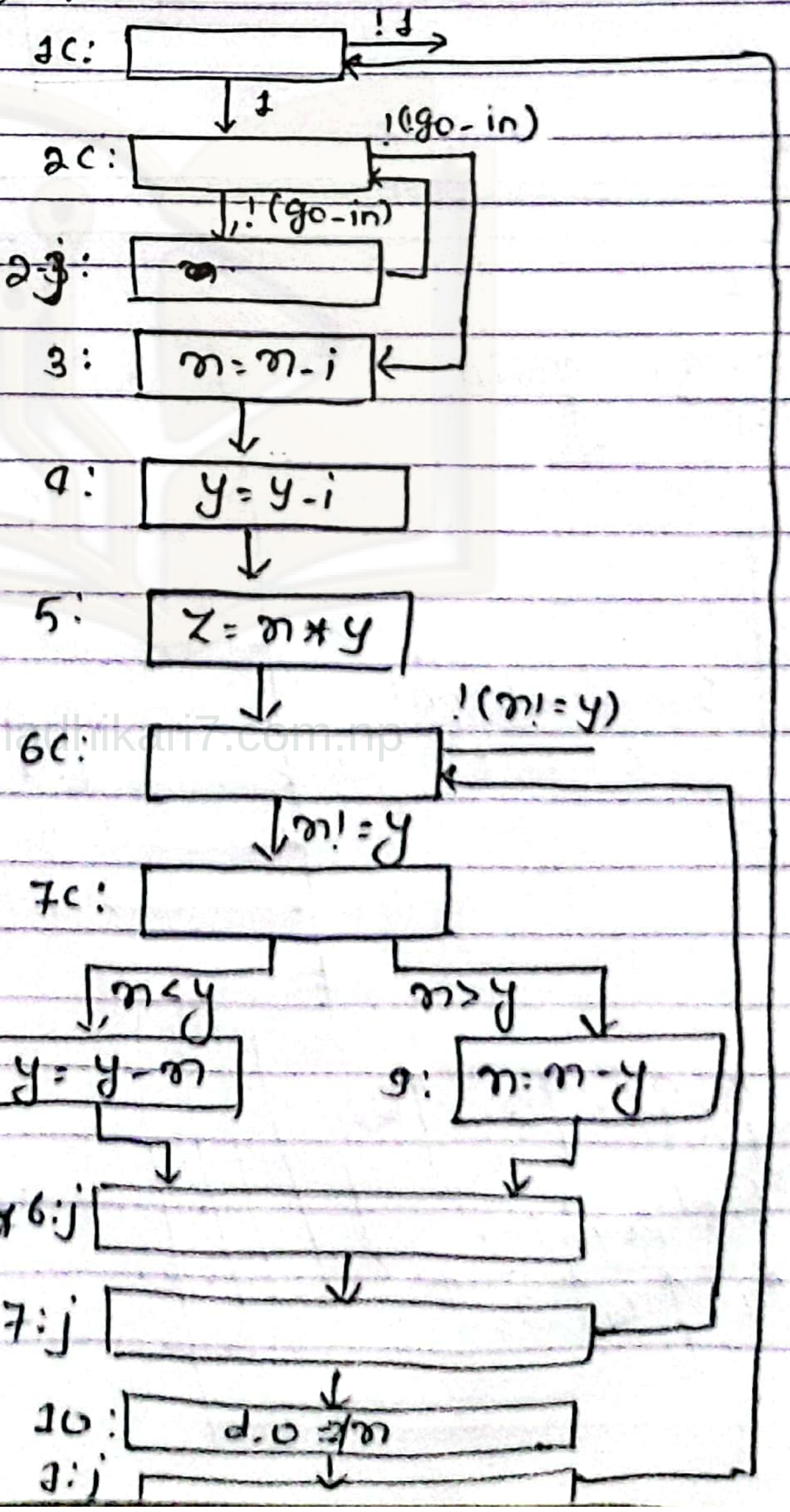
① Black Box:



② functionality code:

```
0:  int x,y,z;
1c: while(1){
2c: while (!go_in)
3 :  x = x_i;
4 :  y = y_i;
5 :  z = x * y;
6c: while (x != y){
7c: if (x<y)
8 :    y = y - x;
    else
9 :    x = x - y;
10:  d_o = z/x;
    }
  }
```

③ FSMD

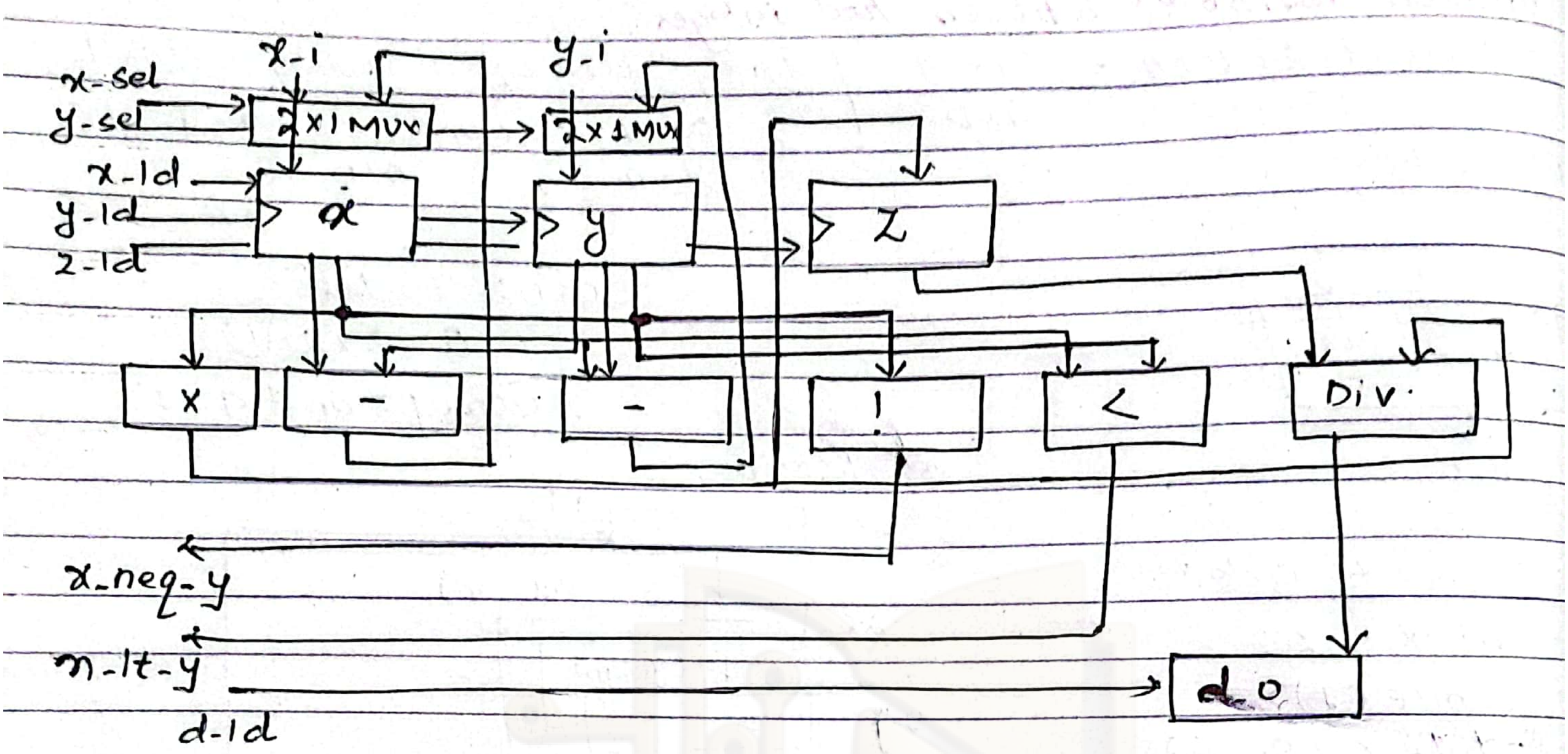


notes.arpanadhikari7.com.np

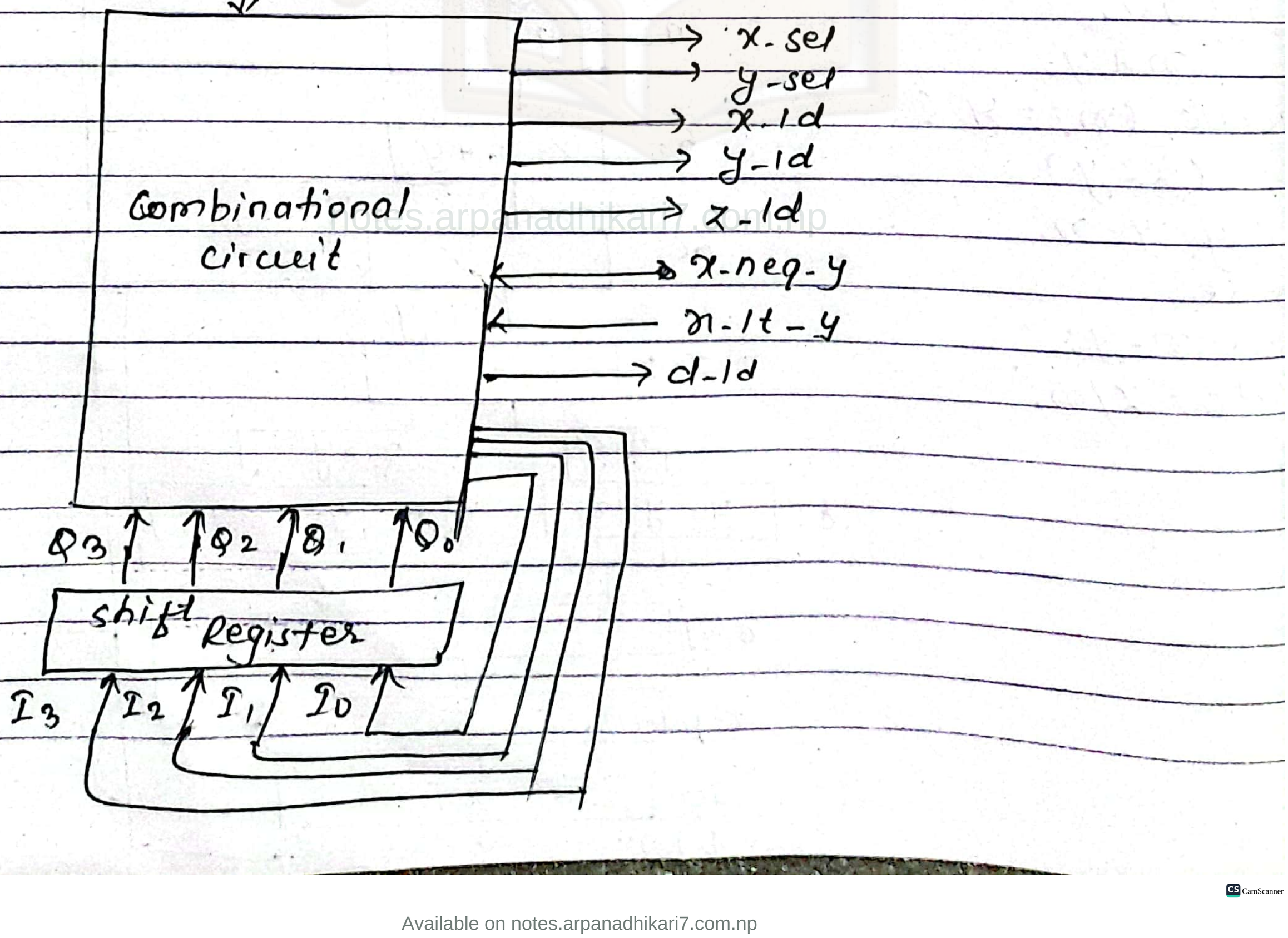
CamScanner

Available on notes.arpanadhikari7.com.np

4. Data Path:



5. Controller:



Available on notes.arpanadhikari7.com.np

2016 spring

Q. Explain with an example how to optimize custom single purpose processors.

Controller state table for the GCD

Inputs							Outputs								
Q3	Q2	Q1	Q0	x_neq_y	x_lt_y	go_i	I3	I2	I1	I0	x_sel	y_sel	x_ld	y_ld	d_ld
0	0	0	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	0	1	*	*	0	0	0	1	0	X	X	0	0	0
0	0	0	1	*	*	1	0	0	1	1	X	X	0	0	0
0	0	1	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	1	1	*	*	*	0	1	0	0	0	X	1	0	0
0	1	0	0	*	*	*	0	1	0	1	X	0	0	1	0
0	1	0	1	0	*	*	1	0	1	1	X	X	0	0	0
0	1	0	1	1	*	*	0	1	1	0	X	X	0	0	0
0	1	1	0	*	0	*	1	0	0	0	X	X	0	0	0
0	1	1	0	*	1	*	0	1	1	1	X	X	0	0	0
0	1	1	1	*	*	*	1	0	0	1	X	1	0	1	0
1	0	0	0	*	*	*	1	0	0	1	1	X	1	0	0
1	0	0	1	*	*	*	1	0	1	0	X	X	0	0	0
1	0	1	0	*	*	*	0	1	0	1	X	X	0	0	0
1	0	1	1	*	*	*	1	1	0	0	X	X	0	0	1
1	1	0	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	0	1	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	1	*	*	*	0	0	0	0	X	X	0	0	0

Optimization of Single Processor Design

→ terminate (विराम)

The finite state machine uses the number of states so thaht they can be reduced without halting the operation.
The machine can be optimized by optimizing different parameter as below:

- Original program
- FSMD (Finite State Machine with Datapath)
- Data path
- FSM for controller

notes.arpanadhikari7.com.np

1. Optimization of Original Program:

The program can be optimize by optimizing the no of computation, size of variable, time and space complexity, operation used like multiplication and division may have the higher cost etc. for example:

CamScanner

Available on notes.arpanadhikari7.com.np

Original program

```
0:    int x,y;
1:    while (1){
2:          while(!go_i);
3:                x=x_i;
4:                y=y_i;
5:                      while(x!=y){
6:                            if(x<y)
7:                                  y=y-x;
                              else
8:                                  x=x-y;
                              }
9:                      d_o=x;
          }
```

Let us see, how many iteration does it takes to find GCD of (42,8)

(42,8)→(34,8) → (26,8)→(18,8) → (10,8)→(2,8) →(2,6)→(2,4) →(2,2)

Thus, we can see it takes 9 iteration to complete. Now, let us refine our algorithm by using the next approach as shown below.

Optimized Program

```
0: int x, y, r;
1: while (1) {
2:   while (!go_i);
     // x must be the larger number
3:   if (x_i >= y_i) {
4:     x=x_i;
5:     y=y_i;
     }
6:   else {
7:     x=y_i;
8:     y=x_i;
     }
9:   while (y != 0) {
10:    r = x % y;
11:    x = y;
12:    y = r;
     }
13:  d_o = x;
   }
```

notes.arpanadhikari7.com.np

Again let's see how many iteration does it takes to find GCD of (42,8)

(42,8)→(8,2) → (2,0)

CamScanner

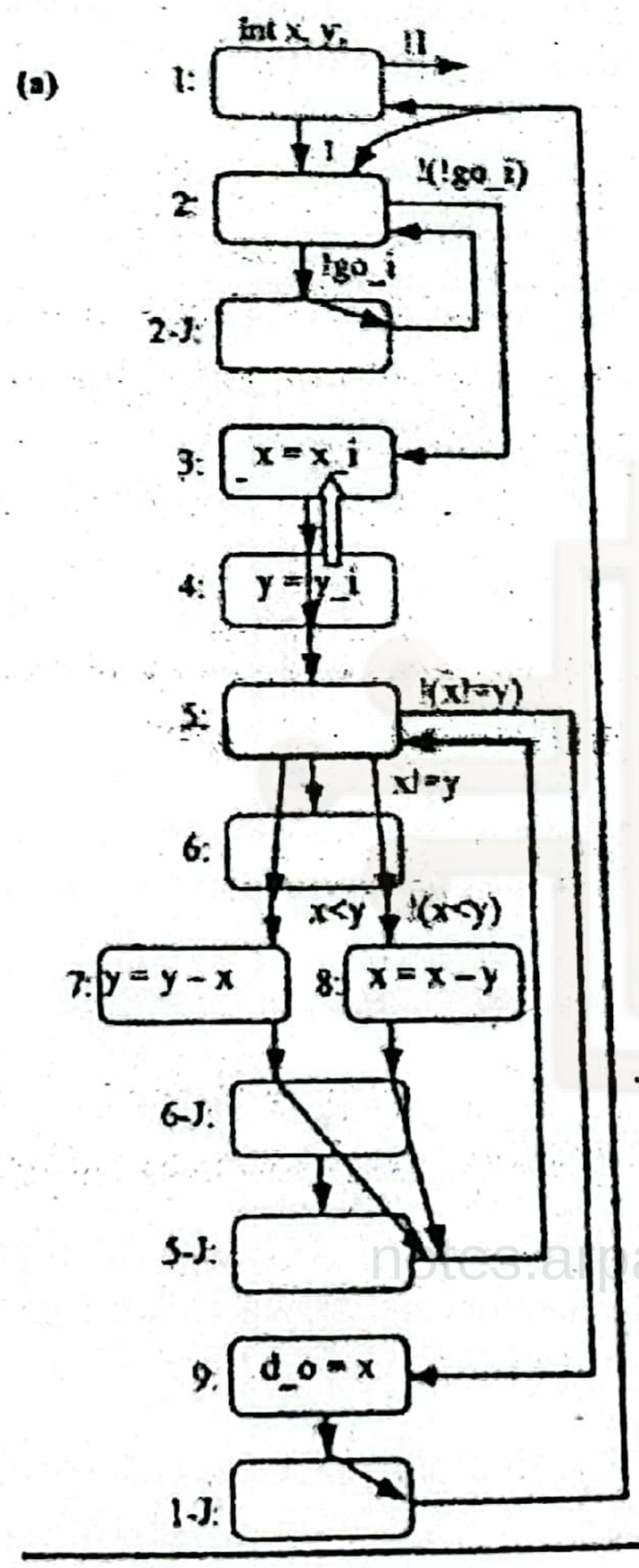
Available on notes.arpanadhikari7.com.np

Here subtract operation is replaced by modulo operation, the program be optimized in terms of number of computation as well as space and time complexity.

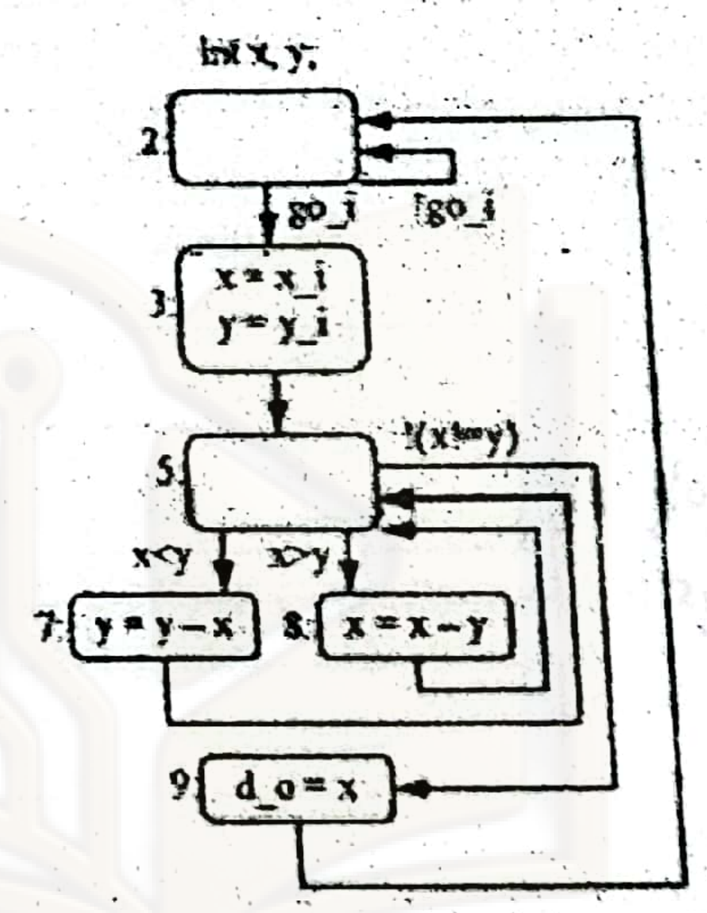
2. Optimization of FSMD:

FSMD can be optimized by using different concepts as merge state, separate state and scheduling. Those states they have constant value or independent with change then they can be merged and so called merged state. The state with complex logic can be replaces by no of sub operation with simpler logic that reduce the hardware complexity and so called as separate state. By optimization of scheduling time, we can optimize the FSMD. For example:

(a)



(b)



notes.arpanadhikari7.com.np

- Eliminate the state -1 that consists constant value.
- Merge the state 2 and 2-J since there is no loop operation in between them.
- Merge the state 3 and 4 since they are independent to each other.
- Merge the state 5 and 6 since transition from state 6 can be done from 5.
- Eliminate state 6-J and 5-J since it can be done from state 7 and 8.
- Eliminate the state 1-J since it can be done through state 9.

CamScanner

Available on notes.arpanadhikari7.com.np

3. Optimizing Data Path:

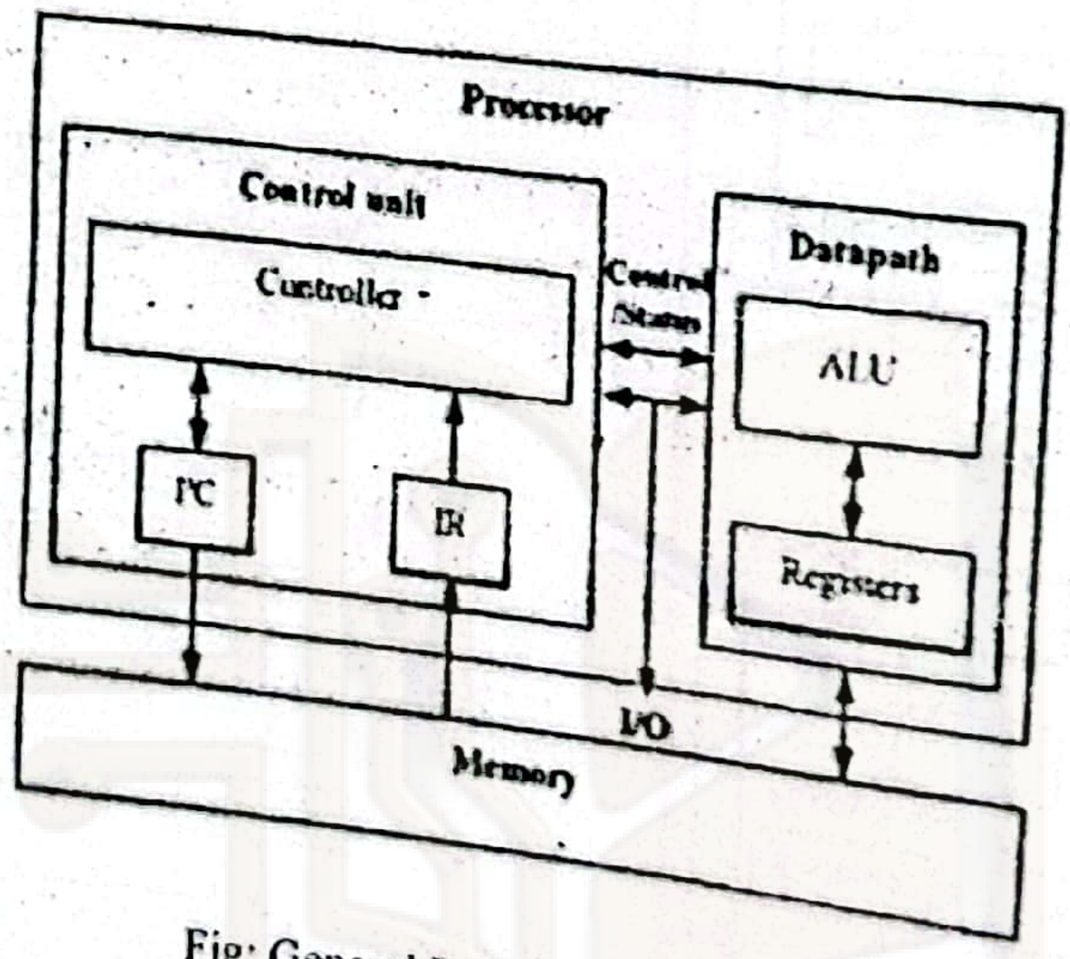
To optimize the data path, a sheared functional unit can be used such that there is a optimized datapath and able perform the verity of operation. For this purpose we can use the sheared ALU circuit that supports verity of state equation as well as state operations. For example: for operation X-Y and Y-X, instead of using two sub tractor we can generate the result.

4. Optimizing FSM for Controller:

The FSM of controller can be optimize by reducing the states and using efficient state encoding. The n bit encoded binary bits can define 2^n different states and n bits can generate the n! Combinations. The states can be merged into a single state such that it provides the optimize FSM.

Today's

Software Design Issues: General Purpose Processor



Basic fig: Architecture of Software Design Issue.

Fig: General Purpose Processor

The general purpose processor sometimes called as central processing unit (CPU) or a microprocessor. It consists of a datapath and a control unit, tightly coupled with memory as shown above fig. the general purpose basic architecture consists of:

- Datapath
- Control unit
- Memory

notes.arpanadhikari7.com.np

Data path:

The data path consists the circuitry to transfer the data from one place to another and storing the temporary data. The datapath contains the ALU capable of transferring the data from different operation like additions, subtraction, bitwise OR, AND etc. ALU also generates the status signals often stored in status register. These status bit conditions are known as flags. The flags may be zero, sign carry overflow etc. It also contains register and stores the data temporarily during ALU operation. The temporary data includes:

Available on notes.arpanadhikari7.com.np